

Лекции по дисциплине
«Технология хранения и архитектура
больших данных»

П. В. Купцов

Кафедра Информатики и программирования, факультет КНиИТ СГУ

2025-2026, осенний семестр

Рекомендованная литература

- Мамедли Р. Э., Казиахмедов Т. Б. Большие данные и NoSQL базы данных : учебное пособие для вузов — СПб.: Лань, 2024. — 92 с.
- Бурков А. Инженерия машинного обучения. — М.: ДМК Пресс, 2022. — 306 с.
- Бахши С. Power BI: моделирование на экспертном уровне. — М.: ДМК Пресс, 2022. — 490 с.
- Дейтел П., Дейтел Х. Python: Искусственный интеллект, большие данные и облачные вычисления. — СПб.: Питер, 2020. — 864 с.
- Дэви С., Арно Д., Мохамед А. Основы Data Science и Big Data. Python и наука о данных. — СПб.: Питер, 2017. — 336 с.
- Танимура К. SQL для анализа данных. — СПб.: БХВ-Петербург, 2024. — 384 с.
- Ланских Ю. В., Ланских В. Г., Родионов К. В. Введение в большие данные : учеб. пособие. — Киров: ВятГУ, 2023. — 172 с.
- Карау Х., Конвински Э., Венделл П., Захария М. Изучаем Spark: молниеносный анализ данных. — М.: ДМК Пресс, 2015. — 304 с.
- Перрен Ж.-Ж. Spark в действии. — М.: ДМК Пресс, 2021. — 636 с.

Содержание

Лекция 1. Понятие больших данных

- Предназначение данных
- Неформальное обсуждение понятия «большие данные»
- Определение и признаки больших данных
- Вопросы для конспекта

Лекция 2. Жизненный цикл больших данных

- Кластер больших данных и этапы работы с данными
- Специалисты по работе с данными
- Принципы организации хранения и обработки больших данных
- Классификация данных по наличию или отсутствию структуры
- Вопросы для конспекта

Лекция 3. Реляционные базы данных и распределённые хранилища

- Понятие реляционной базы данных
- ACID: базовые принципы работы реляционных баз данных
- Понятие распределённой системы хранения данных
- CAP-теорема для распределённых вычислительных систем
- Применимость реляционных баз для хранения больших данных. NoSQL-базы
- Принципы BASE для NoSQL-баз данных
- Вопросы для конспекта

Лекция 4. NoSQL хранилища больших данных

- Базы данных ключ-значение
- Документоориентированные базы данных
- Столбцовые или колонкоориентированные базы данных
- Графовые базы данных
- Технологии повышения эффективности распределённых хранилищ данных
- Вопросы для конспекта

Лекция 5. Столбцовая база данных ClickHouse

- Понятие веб-аналитики
- Основные особенности ClickHouse
- Особенности выполнения запросов в ClickHouse
- Вопросы для конспекта

Лекция 6. Документоориентированная база MongoDB

- Общие сведения о MongoDB
- Принципы организации хранения и обработки данных в MongoDB
- Выборка, фильтрация, агрегация
- Вопросы для конспекта

Лекция 7. Экосистема хранения и обработки больших данных Hadoop

- Общие сведения о Hadoop
- Взаимодействие Hadoop с другим инструментами для больших данных
- Архитектура кластера Hadoop
- Hadoop в настоящее время
- Вопросы для конспекта

Лекция 8. Распределённая файловая система, используемая в Hadoop

- Структура и основные принципы HDFS
- Работа с файлами в HDFS
- Требования к форматам файлов в HDFS
- Форматы файлов, используемых в HDFS
- Вопросы для конспекта

Лекция 9. Обработка данных с использованием MapReduce и Apache Hive

- Общие принципы работы MapReduce
- Иллюстрация подхода MapReduce при помощи программы на Python
- Реализация MapReduce на языках, отличных от Java при помощи Hadoop Streaming
- Основные сведения об Apache Hive

- Этапы работы с данными в Apache Hive: сопоставление файлам схемы данных в виде таблицы
- Этапы работы с данными в Apache Hive: выполнение запроса к таблице
- Вопросы для конспекта

Лекция 10. Общие сведения об Apache Spark

- Предназначение Apache Spark и его место среди систем для работы с большими данными
- Основные компоненты Spark
- Базовые структуры данных, используемые в Spark
- Структура Spark кластера и архитектура приложения
- Spark-приложение
- Вопросы для конспекта

Лекция 11. Работа с данными в Spark с использованием RDD

- Общие сведения о RDD
- Принципы работы с RDD
- Общие правила загрузки RDD из файлов и сохранения в файлы
- Выполнение DAG. Стадии и задачи
- Механизм взаимодействия драйвера и исполнителей
- Пример Spark-приложения и визуализация DAG
- Вопросы для конспекта

Лекция 12. Операции над RDD

- Загрузка и чтение текстовых файлов TXT
- Загрузка и чтение файлов JSON и CSV
- Операции преобразования RDD
- Операции действия над RDD
- Кеширование
- Глобальные переменные и работа механизма замыкания в Spark

- Аккумуляторы в Spark
- Широковещательные переменные
- Вопросы для конспекта

Лекция 1. Понятие больших данных

Предназначение данных

- **Данные** — совокупность сведений о реальном мире, представленных в форме, приемлемой для унифицированной обработки. В настоящее время обычно предполагается автоматическая обработка данных при помощи ЭВМ.

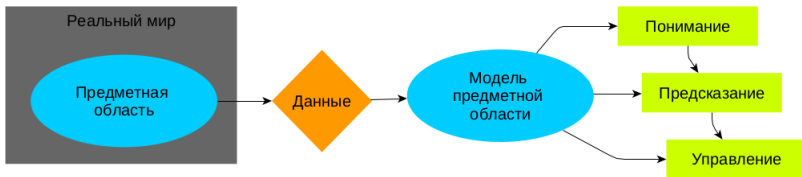
Данные собирают и обрабатывают для создания моделей каких-либо фрагментов реального мира (предметных областей).

Примечание

Унифицировать — от латинских «unus» (один) и «facere» (делать). Создавать единые требования, формы, методы или правила для разных объектов или процессов.

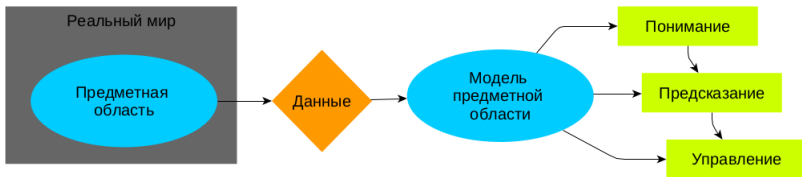
Модель — это описание некоторой части реального мира (предметной области), построенное для получения предсказания его поведения в будущем.

Люди всегда занимались сбором данных и построением на их основе моделей. И всегда это имело большое значение.



Движение небесных тел:

- Предметная область: планеты и звёзды.
- Данные — астрономические наблюдения Тихо Браге.
- Модель — законы Кеплера, потом обобщение в виде закона всемирного тяготения Ньютона $F = Gm_1m_2/R^2$.
- Понимание — притяжение зависит от масс и расстояния.
- Предсказание — можно предсказывать астрономические явления.
- Управление — вывод спутников на орбиту.



Работа фирмы:

- Предметная область — фирма, поставщики, клиенты, банки.
- Данные — сведения о доходах и расходах.
- Модель — бухгалтерская книга (таблица доходов и расходов).
- Понимание — видим полную картину поступлений и трат.
- Предсказание — можем рассчитать объём поставок в будущем, сумму необходимых кредитов, оптимальные цены.
- Управление — меняем поставщиков, оптимизируем расходы, выводим новые продукты на рынок.

До появление ЭВМ — сбор данных и построение моделей только «вручную».

Мало данных, мало сущностей, простые модели.

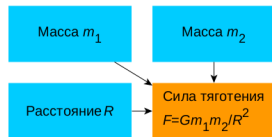
В бизнесе — рукописные гроссбухи.



Примечание

Гроссбук (нем. Großbuch) — это главная бухгалтерская книга, сводный документ, в котором отражаются итоговые данные по бухгалтерским отчетам и счетам.

В науке — базовые законы физики.



Начало использование ЭВМ — автоматизация сбора данных, стандартизация методов моделирования (электронные таблицы, базы данных, графики и гистограммы, модели машинного обучения).

Данных достаточно много (мегабайты и гигабайты), много сущностей (сотни и тысячи).

Но! Всё находится в пределах долговременной памяти одного компьютера или компьютера и сервера в локальной сети.

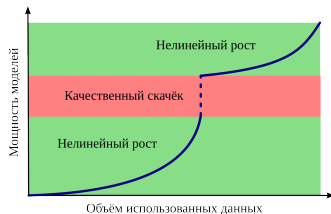


Вопросы для конспекта

1. Что такое данные?
2. С какой целью собирают данные?

Неформальное обсуждение понятия «большие данные»

Мощность моделей (качество понимания, точность прогнозов, возможности управления) растёт нелинейно с ростом объёма задействованных данных, а иногда делает качественные скачки.



Пример: ChatGPT учили на очень больших датасетах.

- GPT1. Датасет — 7 000 книг, 120 миллионов параметров.
- GPT2. Датасет — 40 Гб данных, 1.5 миллиарда параметров.
- GPT3. Датасет — 570 Гб данных, 175 миллиардов параметров.

Но выяснилось, что средства автоматического накопления данных развиваются быстрее, чем средства их обработки.

Регистрировать очень много данных достаточно легко, а хранить и обрабатывать их стандартными инструментами (электронные таблицы, реляционные базы данных) не получается — не хватает места в памяти, резко падает скорость обработки.

Причины появления понятия «большие данные»

- В науке и технологии: чем больше данных, тем интереснее результаты.
- В бизнесе: крупные организации накапливают огромные объёмы данных в процессе своей деятельности.
- Стандартные инструменты не работают там где интереснее и важнее всего.

Можно сказать, что данные считаются большими, когда для работы с ними недостаточно персонального компьютера или персонального компьютера и сервера в локальной сети.

Для больших данных требуются вычислительный кластер, каждый из узлов которого выполняет определённую функцию: хранения, управления, обработки.

Большие данные отличаются от «обычных» только тем, что их очень много. Никаких других принципиальных отличий в их структуре или в содержании нет.

В настоящее время для обеспечения функционирования крупных предприятий требуется работа с большими данными (очевидно, при условии что бизнес-процессы цифровизованы).

Примечание

ERP (Enterprise Resource Planning, планирование ресурсов предприятия) — интегрированная компьютерная система управления бизнес-процессами: финансы, производство, управление цепочками поставок, управление персоналом и другие. ERP-системы объединяют данные из разных отделов в единую базу, что позволяет компаниям функционировать более эффективно. Примеры — российская 1С:ERP, немецкая SAP ERP.

В 2018 году пользователи и предприятия по всему миру сгенерировали 33 зеттабайта информации, в 2025 году — 175 зеттабайт.

Примеры отраслей, где активно используются большие данные

- Банки.
- Страховые компании.
- Промышленность.
- Сотовые операторы.
- Развлекательные платформы.
- Системы видеонаблюдения.
- Интернет вещей.

Крупные научно-исследовательские проекты также работают с большими данными.

Пример — международная коллаборация, работающая на БАК, Большом адронном коллайдере.

Примечание

БАК — самый мощный ускоритель элементарных частиц, построенный на границе Швейцарии и Франции.



В каждом эксперименте на БАК записывается примерно 40 терабайт данных в секунду.

В конце сентября 2023 года общий объем накопленных данных достиг более одного эксабайта — это 1024 петабайт или 1 миллион терабайт.

В настоящее время большая часть данных, накопленных БАК, остаётся необработанной.

Пример масштабного научного проекта, который производит много данных, но это не «большие данные».

Проект Спектр-РГ (Спектр-Рентген-Гамма) — это российско-немецкая орбитальная астрофизическая обсерватория (проект Роскосмоса и DLR), предназначенная для построения **полной карты Вселенной** в рентгеновском диапазоне.

Примечание

Запуск космической обсерватории Спектр-Рентген-Гамма осуществлён 13 июля 2019 года. По состоянию на 2019 год это одна из лучших рентгеновских обсерваторий на ближайшие 10-15 лет. В отличие от предыдущих рентгеновских космических телескопов, поле зрения которых очень ограничено, «Спектр-РГ» способен сделать полный обзор неба с рекордной чувствительностью.



Передача данных на Землю производится со скоростью до 512 Кб/с.
Ежедневный объём передаваемых данных 500 Мб.

За год накапливается примерно 183 Гб данных. За 10 лет — около 1.8 Тб.

Это достаточно много, но эти данные могут храниться на одном жёстком диске и обрабатываться одной достаточно производительной рабочей станцией.

Поэтому их нельзя характеризовать как большие данные.

Вопросы для конспекта

3. Как мощность моделей зависит от объём использованных для их построения данных?
4. Перечислите причины появления понятие большие данные.
5. Приведите примеры использования больших данных в бизнесе.
6. Приведите примеры использования больших данных в науке.

Определение и признаки больших данных

Термин «большие данные» не имеет общепринятого строго определения. Более того, понимание больших данных меняется по мере развития вычислительного оборудования и технологий.

Впервые это понятие стали использовать в 70-х годах 20 века. Его стали использовать в связи с появлением реляционных баз данных и универсального языка запросов SQL.

Это позволило систематически обрабатывать большие, по тем временам, объёмы данных и извлекать из них новое знание.

Очевидно, что данные, считавшиеся большими в те времена, сегодня таковыми не являются.

Более того, в настоящее время считается, что традиционные реляционные базы данных непригодны для работы с большими данными из-за низкой эффективности.

Часто при описании больших данных пишут о влиянии их на экономику, социальную жизнь, развлечения, безопасность и т. д.

Можно встретить публикации, в которых пишут практически о чудесных возможностях анализа больших данных.

Это принципиальная неверно!

Чтобы получить значимый результат, данных должно быть просто достаточно и они должны быть адекватны поставленной задаче. Они не обязательно должны быть «большими».

Поэтому правильным будет определять большие данные только в контексте имеющихся в настоящее время технических возможностей по работе с ними.

- **Большие данные** — это данные, сбор, управление и обработку которых невозможно осуществить с помощью наиболее часто используемых в настоящее время вычислительных устройств и программных средств в течение допустимого для пользователя времени.

Выделяют три признака больших данных — 3V:

- **Volume (Объем)** — объём данных значительно больше чем типичная ёмкость долговременной памяти современной рабочей станции (типичный размер диска).
- **Velocity (Скорость)** — объём располагаемых данных всё время нарастает и скорость поступления новых данных существенно выше, чем скорость обработки данных на типичной современной рабочей станции (типичная скорость выполнения SQL запросов к реляционной базе данных).
- **Variety (Разнообразие)** — данные могут поступать одновременно из разных источников, могут иметь разный разную структуру, структура может меняться и структуры может не быть (получаем не записи с предварительно заданным набором полей, а записи, формат которых может измениться со временем и одновременно с этим произвольный текст, звук, видео).

В некоторых статьях, особенно в публикациях в Интернете, к признакам больших данных добавляют ещё два, три или даже четыре V:

- **Veracity (Достоверность)** — важно заранее удостовериться, что с данными все в порядке.
- **Variability (Изменчивость)** — значение одних и тех же данных может различаться в зависимости от контекста.
- **Visualization (Визуализация)** — визуализация делает данные доступными для человеческого восприятия.
- **Value (Ценность)** — анализа данных даёт ценные результаты.

Но это ошибка! Добавочные пункты — это не признаки именно больших данных.

Это список очевидных требований, предъявляемые к работе с данными вообще, независимо большие они или нет.

Когда пишут о больших данных часто рассказывают о невероятных возможностях, которые открываются в результате их анализа.

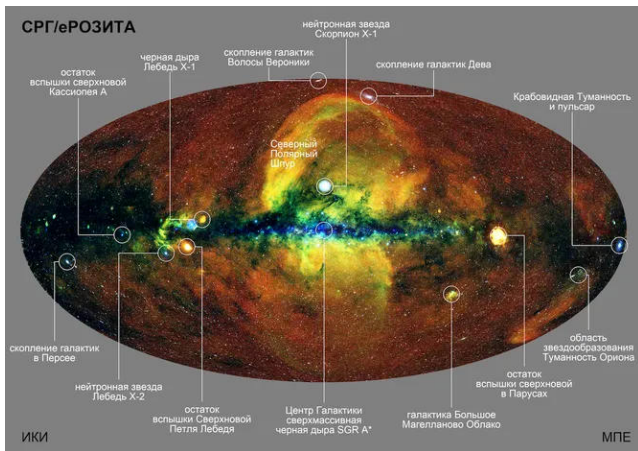
Но на самом деле это относится не к большим данным а к данным как таковым.

Естественно, большой объём данных даёт больше возможностей.

Но данные не обязательно должны удовлетворять критерию «большие данные» чтобы их анализ дал интересные, прорывные результаты.

Вспомним упоминавшийся выше проект Спектр-РГ.

Это очень масштабный проект. Его цель — съёмка космического пространства по всем направлениям, создание **полной карты звёздного неба** в рентгеновском спектре.



Много ли данных производит эта обсерватория?

Да, очень много.

Представляют ли они ценность?

Да, огромную. С их помощью делается и будет сделано много астрофизических открытий.

Будут ли с помощью этих данных обнаружены неочевидные, неявные закономерности и взаимосвязи?

С высокой долей вероятности да.



Но при этом данные этого проекта в настоящее время (по отношению к типичным современным вычислительным средствам) не являются «большими данными», так как для их обработки не требуется специализированная инфраструктура.

Плохо ли это? Делает ли это результаты менее ценными и интересными?

Нет, наоборот, это очень хорошо. Это пример того, как данные, адекватные поставленной задаче, позволяют получить ценные результаты. А так как это не «большие данные», стоимость получения результатов снижается потому что инфраструктура больших данных стоит дорого.

Таким образом, «большие данные» — это не о том, какие невероятные знания можно извлечь из данных, а о том, что когда данных становится слишком много, приходится привлекать специальные технические средства и организационные мероприятия, так как стандартные подходы становятся неэффективными.

Тем не менее очевидно, что анализ больших объёмов данных даёт наиболее интересные результаты.

В бизнесе и производстве при внедрении цифровых инструментов скапливаются всё большие объёмы данных и рано или поздно для работы с ними становится необходимой специальная инфраструктура.

В науке и технологии картина аналогична.

Поэтому в настоящее время разворачивание инфраструктуры больших данных — это практически неизбежный этап развития организаций и учреждений, так или иначе основывающих свою работы на использовании данных.

Вопросы для конспекта

7. Дайте определение понятию большие данные.
8. Приведите три признака больших данных.
9. Можно ли считать достоверность, изменчивость, визуализацию и ценность признаками именно больших данных и почему?
10. Должны ли данные обязательно удовлетворять критериям «больших данных» чтобы ожидать от их анализа ценных, прорывных результатов?

Вопросы для конспекта

- ① Что такое данные?
- ② С какой целью собирают данные?
- ③ Как мощность моделей зависит от объём использованных для их построения данных?
- ④ Перечислите причины появления понятие большие данные.
- ⑤ Приведите примеры использования больших данных в бизнесе.
- ⑥ Приведите примеры использования больших данных в науке.
- ⑦ Дайте определение понятию большие данные.
- ⑧ Приведите три признака больших данных.
- ⑨ Можно ли считать достоверность, изменчивость, визуализацию и ценность признаками именно больших данных и почему?
- ⑩ Должны ли данные обязательно удовлетворять критериям «больших данных» чтобы ожидать от их анализа ценных, прорывных результатов?

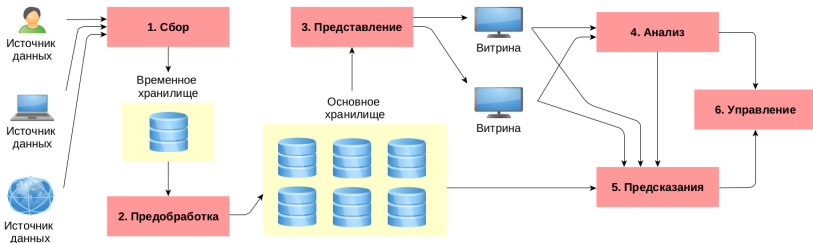
Лекция 2. Жизненный цикл больших данных

Кластер больших данных и этапы работы с данными

Для организации работы с большими данными должен быть развёрнут кластер, который имеет более или менее стандартную структуру.

Обычно в системах больших данных имеется непрерывный поток входящих данных, которые должны некоторым образом обрабатываться и использоваться.

На рисунке показана последовательность операций (пайплайн), которые выполняются на данными в процессе работы с ними, и общая схема функционирования кластера больших данных



① Сбор.

- Данные могут поступать из разных источников в разных форматах. Нужно организовать их сбор и перенос во временное хранилище.

② Предобработка.

- Проверка достоверности и целостности данных.
- Обогащение — подтягивание недостающих данных по ссылкам
- Объединение записей об одной и той же сущности в одной таблице (из разных источников пришли сведения об одном и том же пользователе — их нужно собрать в виде одной записи).
- Проверка и по возможности исправление ошибок.
- Возможно — слияние (Join) таблиц (нарушение требований нормализации с целью ускорения доступа).
- После предобработки данные переносятся из временного в основное хранилище, в котором они хранятся обычно неограниченно долго.

3 Представление.

- Из изначально широких (много столбцов) таблиц делаем тематические выборки (витрины) с необходимыми колонками, делаем фильтрацию, группировку, слияние (Join).
- Результаты этого кешируются в отдельных таблицах и регулярно обновляются.

4 Анализ (понимание).

- Строим сложные запросы к данным витрин (сколько вчерашних клиентов пришли на сайт сегодня, как распределены туристические направления по возрастным группам и т. п.).
- Строим визуальное представление данных (графики, гистограммы, тепловые карты и т. п.).

5 Предсказания.

- На данных основного хранилища или витрин строим модели машинного обучения, которые будут предсказывать процессы в предметной области.

6 Управление.

- Используем результаты анализа данных и предсказания для оптимизации и планирования работы.

Замечание: при работе с большими данными операции слияния по ключу (Join) могут быть очень ресурсоёмкими. По этой причине на этапе предобработки может быть принято решение об отходе от строгих требований нормализации таблицы в пользу эффективности — сохраняют таблицы с нарушением требований второй нормальной формы.

Замечание: для повышения эффективности выполнения аналитических запросов, на основе исходных таблиц больших данных строят витрины — закешированные выборки, необходимые для конкретных задачи (продажи, маркетинг, логистика — всем требуются разные коллекции столбцов).

Вопросы для конспекта

1. Перечислите основные этапы работы с данными. Дайте их описание.
2. Для чего предназначено временное хранилище данных?
3. Для чего предназначено основное хранилище данных?
4. Для чего нужны витрины данных?

Специалисты по работе с данными

За реализацию стандартных этапов работы с данными отвечают разные специалисты, которые перечислены ниже.

● **Инженер по данным** — проектирует, разворачивает и поддерживает инфраструктуру для сбора, хранения, обработки и доставки данных.



- Выбор и настройка технологий для хранения и обработки.
- Развёртывание и управление инфраструктурой в облаке или on-premise (локально).
- Проектирование структуры хранилищ данных и витрин.
- Разработка и поддержка пайплайнов (цепочек операций) по переносу входящих сырых данных в постоянное хранилище.
- Оптимизация производительности (оптимизация запросов к базам, разбиения данных на партиции, масштабирование оборудования).
- Обеспечение механизмов отказоустойчивости и восстановления после сбоев (резервирование данных, горячая замена вышедшего из строя оборудования).

● **Инженер по качеству данных** — обеспечивает точность, полноту, согласованность, надежность и пригодность данных. Его задача — предотвратить попадание «мусорных» данных в аналитику, отчёты и модели машинного обучения.



- Профилирование данных, т. е. анализ на предмет выбросов и систематических пропусков, выявление аномальных статистик (среднее значение слишком велико, дисперсия нереалистично мала).
- Разработка процедур вычисления метрик качества данных (процент пропущенных значений, количество дубликатов, соответствие формату адресов и телефонов, отклонение от исторических норм, соответствие справочникам).
- Организация автоматического вычисления метрик качества.
- Постоянное отслеживание метрик качества данных в реальном времени или по расписанию.
- Расследование причин появления ошибок, поиск путей их устранения.

● **Аналитик данных** — описательный анализ данных без привлечения сложных моделей, в основном на основе запросов к базам и разнообразных средств их визуализации.



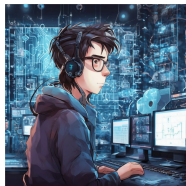
- Извлечение данных, обычно из подготовленных для них витрин, при помощи достаточно сложных запросов.
- Качественный анализ данных — выявление паттернов, трендов, аномалий, поиск закономерностей.
- Вычисление базовых статистических метрик (средние, медианы, персентили, корреляции и т. п.).
- Создание визуального представления данных в виде дашбордов (интерактивная визуальная панель, которая отображает ключевые показатели и данные, собранные из разных источников, в удобном для анализа виде).
- Мониторинг метрик (конверсия, средний чек и т. п.).
- Предложение управленческих решений на основе выполненного анализа.

● **Учёный по данным** — извлекает из данных скрытые знания, закономерности и ценность для целей решения задач прогнозирования будущего и поддержки принятия решений.



- Взаимодействие с заказчиками по формализации требований, преобразования их в виде конкретной задачи математической статистики или машинного обучения.
- Сбор и разведочный анализ данных о предметной области.
- Преобразование данных к пригодному для использованию виду.
- Выбор и построение модели, организация её обучения.
- Оценка качества обучения.
- Подготовка модели к работе в реальных условиях (создание API, контейнеризация).
- Коммуникация с заказчиками по наглядному представлению полученных результатов.

● **Инженер по машинному обучению** — превращает прототипы моделей машинного обучения в производственные системы. Его задача — создать инфраструктуру для жизненного цикла ML-моделей (от разработки до мониторинга).



- Внедрение обученных моделей, созданных учёными по данным, в продакшен-среду: создание API, микросервисов, пакетных обработчиков.
- Создание сквозных пайплайнов для обучения, валидации, тестирования и деплоя моделей.
- Отслеживание метрик: контроль точности, задержек, нагрузки на систему в реальном времени.
- Масштабирование инфраструктуры для работы с большими объёмами данных и запросов.

В реальности функции специалистов могут пересекаться, один и тот же человек может выполнять функции нескольких специалистов.

Вопросы для конспекта

5. Опишите, чем занимается инженер по данным.
6. Опишите, чем занимается инженер по качеству данных.
7. Опишите, чем занимается аналитик данных.
8. Опишите, чем занимается учёный по данным.
9. Опишите, чем занимается инженер по машинному обучению.

Принципы организации хранения и обработки больших данных

- **Локальность.** Задачи обработки должны данных выполняться как можно ближе к физическому месту хранения данных, на которых они работают (предпочтительно на том же узле). Вместо перемещения данных по сети к вычислительным ресурсам, код отправляется на узлы, где данные уже находятся.
- **Неизменность.** Данные, однажды записанные в основное хранилище, не изменяются «на месте». Вместо изменения создаются новые версии данных.
- **Идемпотентность** (от лат. *idem* — «тот же самый» и *potens* — «способный»). Задачи по обработки данных должны быть такими, чтобы многократные их применения к тем же самым исходным данным всегда давали бы один и тот же результат.
- **Параллелизм.** Задачи обработки данных должны допускать разбиение на множество подзадач, которые могут выполняться независимо и параллельно на разных узлах.

- **Масштабируемость.** Система должна увеличивать производительность пропорционально добавлению ресурсов — наращивание числа узлов (горизонтальное масштабирование), увеличения быстродействия процессоров и объёма памяти (вертикальное масштабирование).
- **Отказоустойчивость.** Система должна автоматически продолжать корректную работу или быстро восстанавливаться при сбоях отдельных узлов.

Примечание

Идемпотентность похожа на понятие чистоты функции, но это не совсем одно и то же. Идемпотентность относится к свойству операции, когда повторное её выполнение с теми же входными данными даёт тот же результат, что и однократное выполнение.

Чистая функция, в свою очередь, не имеет побочных эффектов и всегда возвращает один и тот же результат для одних и тех же входных данных. Все чистые функции идемпотентны в отношении побочных эффектов, но не все идемпотентные операции являются чистыми функциями.

Например функция, которая просто возвращает квадрат числа, является и чистой, и идемпотентной. Функция, которая увеличивает счетчик на сервере, может быть идемпотентной, если она возвращает текущее значение счетчика, но она не является чистой, так как изменяет состояние сервера.

Однако учитывая принцип неизменности можно сказать, что применительно к большим данным идемпотентные операции будут также и чистыми.

Вопросы для конспекта

10. Перечислите и опишите принципы организации хранения и обработки больших данных.

Классификация данных по наличию или отсутствию структуры

- **Структурированные данные** — это данные, имеющие строгую, заранее определенную схему (модель) и организованные в виде таблиц.

Схема (модель) данных в таблице описывает имена столбцов, типы данных для каждого столбца, а также любые ограничения или правила, применяемые к данным в этой таблице.

time	event	ad_id	platform	cost
2025-04-01 00:00:48	click	1221	android	211,12
2025-07-24 13:21:15	view	58121	pc	123,11
2025-08-16 09:12:53	view	1121	ios	65,91

Фрагмент таблицы, хранящей структурированные данные.

time	DateTime	Дата и время события
event	VarChar	Тип события, click или view
ad_id	BigInt	Id рекламного банера, >= 0
platform	VarChar	Платформа пользователя, android, pc, ios
cost	Float	Цена, >0

Схема данных таблицы.

Структурированные данные хорошо поддаются анализу. Например, можно подсчитывать средние, медианы, персентили и т. д.

Структурированные данные хранятся в виде таблиц, а для работы с ними используются специализированные системы управления базами данных.

- **Полуструктурированные данные** — это данные, не имеющие строгой схемы, но содержащие метаданные, внутренние метки или маркеры, разделяющие элементы и иерархии.

Пример — тексты, снабжённые тегами. Каждый текст имеет некоторые набор тегов, но у всех количество тегов различное.

Ещё пример — Файлы JSON и XML. Они содержат иерархическую структуру, но допускают вариации в данных.

Пример: два JSON-файла, полученные с датчиков сети IoT.

```
{
  "sensor_id": "temp-sensor-7",
  "timestamp": 1698234567,
  "readings": {
    "temperature_C": 23.4,
    "humidity": 45
  },
  "location": [55.751244, 37.618423]
}
```

```
{
  "sensor_id": "air-quality-3",
  "timestamp": 1698234591,
  "readings": {
    "pm2.5": 12.8,
    "pm10": 24.1,
    "co2_ppm": 420
  },
  "status": "OK",
  "battery_level": 78.5
}
```

- Есть одинаковые поля: "sensor_id", "timestamp" и "readings".
- Типы данных полей "readings" отличаются, т. к. вложенные объекты имеют разные поля: "temperature_C" и "humidity" слева и "pm2.5", "pm10" и "co2_ppm" справа.
- Есть несовпадающие поля: "location" слева и "status" и "battery_level" справа.

Характеристики полуструктурированных данных

- **Гибкая схема** — записи могут иметь разные поля.
- **Иерархическая вложенность** — поля записей могут содержать вложенные записи.
- **Динамические типы данных** — одно поле может содержать разные типы значений в разных записях.
- **Самоописательность** — ключи, например "email", "phone", описывают смысл данных.
- **Необязательные поля** — отсутствие обязательной проверки на наличие всех атрибутов.
- **Эволюция схемы** — возможность добавлять новые поля без изменения обработки старых записей.

Полуструктурированные данные в некоторых случаях можно привести к структурированной форме.

Например, все варианты несовпадающих полей можно указать в схеме данных и разрешить пустые (NULL) значения.

Иногда в этом есть смысл, так как структурированные данные обрабатываются проще (и значит быстрее) всех других.

Но с другой стороны структурирование может привести к таблицам, в которых будет много пустых значений что будет причиной снижения эффективности. Универсального рецепта не существует.

Полуструктурированные данные могут храниться как в виде файлов файловой системы, так и в виде элементов хранения специализированных систем управления базами данных.

- **Неструктурированные данные** не имеют установленной схемы данных или сырые данные, которые пока не упорядочены.

Распространённые примеры неструктурированных данных: текстовые файлы, записи видеонаблюдений.

Неструктурированные так же как и полуструктурированные данные могут храниться как в виде файлов файловой системы, так и в виде элементов хранения специализированных систем управления базами данных.

Вопросы для конспекта

11. Что такое структурированное данные? Приведите пример. В каком виде они хранятся?
12. Что такое полуструктурированное данные? Приведите пример. В каком виде они хранятся?
13. Что такое неструктурированное данные? Приведите пример. В каком виде они хранятся?

Вопросы для конспекта

- 1 Перечислите основные этапы работы с данными. Дайте их описание.
- 2 Для чего предназначено временное хранилище данных?
- 3 Для чего предназначено основное хранилище данных?
- 4 Для чего нужны витрины данных?
- 5 Опишите, чем занимается инженер по данным.
- 6 Опишите, чем занимается инженер по качеству данных.
- 7 Опишите, чем занимается аналитик данных.
- 8 Опишите, чем занимается учёный по данным.
- 9 Опишите, чем занимается инженер по машинному обучению.
- 10 Перечислите и опишите принципы организации хранения и обработки больших данных.
- 11 Что такое структурированные данные? Приведите пример. В каком виде они хранятся?
- 12 Что такое полуструктурированные данные? Приведите пример. В каком виде они хранятся?
- 13 Что такое неструктурированные данные? Приведите пример. В каком виде они хранятся?

Лекция 3. Реляционные базы данных и распределённые хранилища

Понятие реляционной базы данных

- **Реляционные базы данных** — предназначены для хранения структурированных данных в виде таблиц.

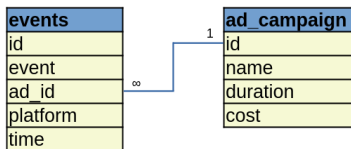
Данные представлены в виде таблиц.

Каждая строка (запись) — это отдельный объект хранения.

Каждая колонка описывает некоторое свойство, присущее объекту.

Важное понятие — это первичный ключ, который представляет собой уникальный идентификатор каждого данного объекта.

У реляционных баз данных между таблицами могут быть установлены отношения (реляции): один к одному, один ко многим.



Работа с реляционными базами данных производится через SQL — Structured Query Language — язык структурированных запросов.

Данные таблиц в реляционных базах хранятся по строкам, т. е. объекты хранятся последовательно, один за другим. По этой причине реляционные базы данных называют строковыми.

Примеры систем управления реляционными базами данных:
PostgreSQL, MySQL, MSSQL Server.

Вопросы для конспекта

1. Что такое реляционные базы данных? Приведите основные характеристики.
2. Какого типа данных хранят реляционные базы данных? В какой форме они хранят данные?
3. К какому типу, согласно CAP-классификации, относятся реляционные базы данных?

ACID: базовые принципы работы реляционных баз данных

Все реляционные базы данных функционируют в соответствии с принципами ACID.

- **Атомарность (Atomicity)** — операции над данными выполняются по правилу «всё или ничего». Если набор некоторых данных нужно поместить в таблицу или выполнить их модификацию то гарантируется, что либо будут совершены все необходимые действия, либо ничего не будет сделано вообще. Если в процессе работы происходит сбой, то производится автоматический откат в исходное состояния. Такие операции называют транзакциями. Все операции над базой данных выполняют как транзакции.

- **Согласованность (Consistency)** — это означает целостность данных. Каждая запись, помещённая в данную не может противоречить выбранной схеме данных — в ней не могут отсутствовать обязательные поля, типы данных должны быть правильными. Кроме того, записи не нарушают связей между таблицами — в таблице `events` не может быть записи с `ad_id`, значения которого нет в связанной с ней таблице `ad_campaign`. Никакая транзакция не может нарушить согласованность.
- **Изолированность (Isolation)**. Если над базой выполняются несколько транзакций параллельно, то они не могут влиять друг на друга. Каждая должна выполняться так, как если бы она была единственной, работающей в базе данных.
- **Долгосрочность (Durability)** — после успешного завершения транзакций, данные сохраняются и остаются в неизменном виде даже при сбоях системы. (Разумеется за исключением ситуаций физического уничтожения носителей информации.)

Вопросы для конспекта

4. Расскажите о принципах ACID, в соответствии с которыми строится работа реляционных баз данных.

Понятие распределённой системы хранения данных

Центральное место в инфраструктуре, обеспечивающей работу с большим данными, занимает основное хранилище данных

В хранилище данные попадают после этапа предобработки и находятся там обычно неограниченно долго.

В некоторых случаях устанавливают временные лимиты, после которых старые данные уничтожаются, но желательно этого не делать, так исторические данные также могут иметь существенную ценность.

Хранилище больших данных реализуется в виде распределённой вычислительной системы.

Распределённая вычислительная система хранения данных — совокупность сетевых узлов, предназначенная для высокодоступного хранения данных, функционирующая как единый логический ресурс.

- **Данные хранятся на множестве компьютеров (узлов).** Это значит, что вместо одного централизованного сервера, данные разделяются на несколько независимых физических или виртуальных машин, объединенных в сеть, часто географически распределённую.
- **Система функционирует как единое целое.** Несмотря на физическое распределение данных, система предоставляет пользователям и приложениям единую точку доступа и единое логическое представление данных в виде единой файловой системы или базы данных. Сложность распределения скрыта от конечного пользователя.
- **Предназначена для хранения данных.** Предназначение такой системы — надёжное, эффективное и масштабируемое хранение больших объёмов данных и обеспечение доступа к ним.

Основные характеристики распределённых хранилищ данных.

- **Масштабируемость.** Возможность легко увеличивать ёмкость хранилища и пропускную способность (производительность) путём простого добавления новых узлов в систему (горизонтальное масштабирование).
- **Отказоустойчивость.** Способность системы продолжать работу (хотя бы в деградированном режиме) и не терять данные при отказе одного или даже нескольких узлов или сетевых компонентов. Достигается в основном через репликацию данных (хранение нескольких копий на разных узлах) и механизмы восстановления.
- **Доступность.** Обеспечение доступа к данным для клиентов большую часть времени (измеряется в процентах от общего времени функционирования), даже в условиях частичных отказов. Высокая доступность напрямую связана с отказоустойчивостью.

- **Согласованность.** Гарантия того, что все клиенты видят одни и те же актуальные данные в определённый момент времени. Высокий уровень согласованности может достигаться за счёт снижения других характеристик хранилища, см. далее CAP-теорему.
- **Производительность.** Способность обрабатывать большие объёмы операций чтения/записи с низкой задержкой. Распределение данных и нагрузки помогает достичь высокой производительности.
- **Географическое распределение.** Возможность размещать узлы в разных датацентрах или регионах для снижения задержки для удалённых пользователей, соответствия регуляторным требованиям (данные в определённой юрисдикции) и повышения отказоустойчивости (устойчивость к сбоям целого региона).

Вопросы для конспекта

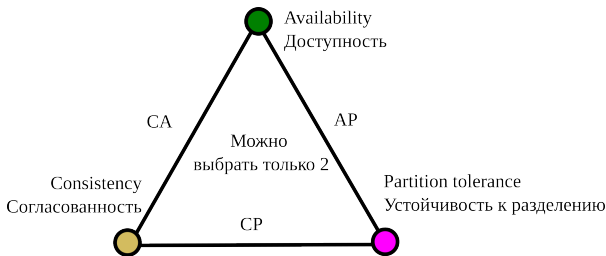
5. Дайте определение распределённого хранилища данных.
6. Перечислите и поясните характеристики распределённых хранилищ данных.

CAP-теорема для распределённых вычислительных систем

Работа с большим данными осуществляется на распределённой вычислительной системе, представляющей собой кластер взаимодействующих вычислительных систем.

Для таких систем выполняется эвристическое правило, получившее название CAP-теоремы или теоремы Брюера.

- **CAP-теорема:** в любой реализации распределённых вычислений возможно обеспечить не более двух из трёх следующих свойств:



- **С, Согласованность (Consistency)** — любые изменения данных распространяются между всеми узлами системы таким образом, что все клиенты в каждый момент времени видят одни и те же данные.
- **А, Доступность (Availability)** — все рабочие узлы всегда выполняют запросы и предоставляют ответы, не содержащие ошибок, т. е. клиент гарантированно получит ответ на запрос даже если один или несколько узлов не доступны; система всегда доступна для операций чтения и записи.
- **Р, Устойчивость к разделению сети (Partition tolerance)** — система должна продолжать работать, даже если соединение между двумя и более узлами потеряно или осуществляется со значительными задержками.

Варианты поведения распределённых систем:

- **СА-система.** Фокус делается на обеспечении высокой доступности и согласованности данных, в то время как при нарушении коммуникации между узлами либо будет приостановлена обработка запросов (для поддержания согласованности), либо запросы продолжат обрабатываться, но тогда пострадает согласованность данных.
- **АР-система.** В приоритете доступность и устойчивость к нарушениям коммуникации между узлами. Допускается нарушение согласованности данных, так как запросы продолжают обрабатываться даже в случае нарушения коммуникации между узлами. Когда коммуникация восстанавливается, узлы синхронизируются, но до этого может оказаться так, что они содержат разные версии данных.
- **СП-система.** Обеспечивается согласованность и устойчивость к нарушению коммуникации. Но доступность не гарантируется — при нарушении коммуникации и во время выполнения обновления и синхронизации обработка запросов приостанавливается.

CAP-теорема помогает разработчикам грамотно расставлять приоритеты в проектируемом распределённом приложении.

При этом нужно помнить, что CAP-теорема часто подвергается из-за того, что сильно упрощает реальность.

В реальности CAP-свойства могут присутствовать в большей или в меньшей степени. Например, доступность может быть частичной, т. е. с бóльшими задержками.

Существует теорема, которая выступает в роли более гибкой рекомендации по разработке распределённых систем.

• **PACELC-теорема.** Если происходит сетевое разделение (partition), система должна выбирать между доступностью (availability) и согласованностью (consistency). В противном случае (else), система должна выбирать между задержками (latency) и согласованностью (consistency)

Этот вариант позволяет разработчикам обратить внимание на большее количество факторов и более приближен к реальной жизни.

Вопросы для конспекта

7. Сформулируйте CAP-теорему для распределённых систем. В чём недостатки этой теоремы?
8. Опишите поведение CA-системы.
9. Опишите поведение AP-системы.
10. Опишите поведение CP-системы.
11. Сформулируйте PACELC-теорему для распределённых систем.

Применимость реляционных баз для хранения больших данных.
NoSQL-базы

Реляционные базы данных плохо подходят для больших данных по следующим причинам.

- Реляционные базы данных реализуют поведение СА (consistency, availability). Такие системы плохо приспособлены для горизонтального масштабирования, т. е. производительность не растёт пропорционально росту числа узлов.
- СА-поведение означает чувствительность к сбою коммуникации узлов кластера. При этом сбои сети в распределенных кластерах — ежедневная норма (например, в дата-центрах на разных континентах).
- Реляционные базы нарушают принцип локальности данных — данные для обработки перемещаются к вычислителю, а не код к данным. Это создаёт нагрузку на сеть и замедляет работу.

- Неэффективность при обработке запросов с небольшим числом полей. Объекты хранения больших данных могут иметь очень много полей (соответствует большому числу столбцов в таблице). При строковом хранении для доступа к одному или нескольким полям приходится считывать из долговременной памяти в оперативную (возможно, предварительно переслав это по сети) строку целиком и только потом выделять из неё нужные поля. Это крайне неэффективно.
- Неэффективность для хранения полуструктурированных и неструктурированных данных.
- Хранилища больших данных могут накапливать данные в течении долгого времени и поставщик может изменить схему отправляемых данных. Потребуется поменять схему в базе что будет очень затратно.

Язык SQL — это золотой стандарт работы с данными. Поэтому даже разрабатывая новые хранилища данных, стараются сохранить возможность использования SQL или хотя бы близкого аналога.

Альтернативные базы данных называют NoSQL-базами — «Not Only SQL». Также их называют нереляционными.

Типы нереляционных баз данных.

- Ключ-значение
- Документориентированные.
- Столбцовые.
- Графовые.

Вопросы для конспекта

12. Опишите причины, по которым реляционные базы данных плохо подходят для хранения больших данных.

Принципы BASE для NoSQL-баз данных

Принципы ACID — это наиболее строгая модель работы с данными, гарантирующая отсутствие ошибок в работе, таких как логические противоречия, потери данных, выдача разным пользователям разных версий и т. п.

К сожалению, при переходе к работе с большими данным на распределённой системе и вынужденным отказом от реляционных баз, приходится отказываться и от ACID, так как распределённые NoSQL базы технически не в состоянии обеспечивать эти строгие требования.

Для NoSQL-баз разработан набор более мягких требований, которые называются BASE.

BASE жертвует строгой согласованностью данных в пользу гибкости, производительности и отказоустойчивости.

- **Базовая доступность (Basically Available).** Система гарантирует доступность данных даже в условиях сбоев (например, при отказе части узлов). Запросы к базе всегда получают ответ (успешный или с ошибкой), но не обязательно самые актуальные данные. Например, при разделении (сбое) сети внутри кластер часть узлов может обслуживать запросы, используя устаревшие данные, но система не «зависает».
- **Мягкое или неустойчивое состояние (Soft State).** Состояние системы может меняться со временем даже без внешних воздействий из-за внутренних процессов (например, репликации или устаревания данных). Данные не являются «истиной в последней инстанции» в каждый момент времени. Например, реплики данных на разных узлах могут временно расходиться, пока не синхронизируются.

■ Согласованность в конечном счете (Eventual Consistency).

Если не поступают новые данные, то со временем все узлы системы придут к согласованному состоянию. Данные будут одинаковыми на всех репликах, но не мгновенно. Например, после внесения изменения в одну копию данных другие узлы обновятся через несколько миллисекунд, секунд или минут.

Сравнение ACID и BASE

Параметр	ACID	BASE
Согласованность	Строгая (всегда актуальные данные)	Слабая (данные со временем согласуются)
Доступность	Может снижаться при сбоях	Приоритет даже в ущерб свежести данных
Подход	Пессимистичный (блокировки)	Оптимистичный (разрешение конфликтов постфактум)
Идеология	«Точность здесь и сейчас»	«Работать всегда, уточнить позже»

Вопросы для конспекта

13. Что собой представляют принципы BASE, в соответствии с которым строится работа NoSQL баз данных?
14. Проведите сравнение принципов ACID и BASE.

Вопросы для конспекта

- 1 Что такое реляционные базы данных? Приведите основные характеристики.
- 2 Какого типа данных хранят реляционные базы данных? В какой форме они хранят данные?
- 3 К какому типу, согласно CAP-классификации, относятся реляционные базы данных?
- 4 Расскажите о принципах ACID, в соответствии с которыми строится работа реляционных баз данных.
- 5 Дайте определение распределённого хранилища данных.
- 6 Перечислите и поясните характеристики распределённых хранилищ данных.
- 7 Сформулируйте CAP-теорему для распределённых систем. В чём недостатки этой теоремы?
- 8 Опишите поведение CA-системы.
- 9 Опишите поведение AP-системы.
- 10 Опишите поведение CP-системы.
- 11 Сформулируйте PACELC-теорему для распределённых систем.
- 12 Опишите причины, по которым реляционные базы данных плохо подходят для хранения больших данных.
- 13 Что собой представляют принципы BASE, в соответствии с которым строится работа NoSQL баз данных?
- 14 Проведите сравнение принципов ACID и BASE.

Лекция 4. NoSQL хранилища больших данных

Базы данных ключ-значение

Базы данных ключ-значение — это NoSQL-системы, в которых данные хранятся в виде пар ключ-значение.

По классификации CAP-теоремы чаще всего это AP-системы, но есть CP-системы или гибриды, где режим выбирается при настройке.

Ключевые характеристики:

- Простая модель данных:

- Ключ — уникальный идентификатор (строка, число, двоичные данные).
- Значение — любые данные (строка, JSON, бинарный объект, изображение).

- Высокая производительность. База поддерживает небольшое число операций, которые выполняются за время $O(1)$. Основные операции:

- `Get(key)` — возвращает значение, связанное с переданным ключом.
- `Put(key, value)` — связывает значение с ключом.
- `Delete(key)` удаляет запись для ключа из хранилища.

- Простое горизонтальное масштабирование.
- Отсутствие схемы данных.
- Нет возможность строить сложные запросы: нет операции JOIN, нет агрегаций и т. д.

Примечание

Время выполнения $O(1)$ или константное время — это значит, что время, необходимое для выполнения алгоритма, не зависит от размера входных данных. Другими словами, независимо от того, сколько данных обрабатывает алгоритм, время его работы остается одним и тем же, или, по крайней мере, не увеличивается пропорционально размеру входных данных.

В базах ключ-значение это достигается за счёт того, что при доступе к данным применяется хеш-функция, которая позволяет напрямую рассчитать адрес ячейки памяти, где хранится значение, связанное с данным ключом.

Базы ключ-значение используются для хранения изображений, создания специализированных файловых систем, в качестве кешей, хранения сессий пользователей.

Примеры таких хранилищ — Riak, Redis, DynamoDB, Memcached.

Вопросы для конспекта

1. Опишите принципы работы баз данных типа ключ-значение.
2. К какому типу относятся базы ключ-значение, согласно CAP-классификации?
3. Для каких задач наилучшим образом подходят базы данных ключ-значение?

Документоориентированные базы данных

Документоориентированные базы данных — это NoSQL системы, чаще всего типа AP. Но CP режим можно задать в настройках.

База представляет собой набор коллекций, каждая из которых хранит сущности определённого типа. В реляционных базах коллекциям соответствуют таблицы.

Каждая сущность — это документ. Документ — это единица хранения, также как строка таблицы — единица хранения реляционной базы.

Документ обычно имеет древовидную структуру, описывающую его свойства. Т. е. у документа может быть набор свойств (атрибутов), каждый из которых может быть также документом.

Для описания документов часто используют JSON, XML или BSON.

Примечание

BSON и JSON — это форматы представления данных, но BSON является двоичным форматом, в то время как JSON — текстовым. BSON — это Binary JSON, двоичный JSON. Например в JSON числовые типы данных хранятся в строк, а в BSON — непосредственно в двоичном виде. BSON разработан для более эффективного хранения и обработки данных, особенно в базах данных.

Пример документа:

```
{
  "_id": "user-789",
  "name": "Анна Петрова",
  "email": "anna@mail.ru",
  "address": {
    "city": "Москва",
    "street": "Тверская, 15"
  },
  "orders": ["ORD-100", "ORD-102"]
}
```

В реляционных базах все записи имеют одинаковую структуру, которая задаётся схемой данных. В документоориентированных базах в одной коллекции документы могут иметь разные поля, т. е. коллекция не имеет жёстко заданной схемы данных.

```
// Документ 1
{ "title": "Книга", "pages": 200 }
// Документ 2
{ "name": "Наушники", "price": 5990, "warranty": 2 }
```

Документоориентированные базы данных поддерживают глубокие запросы, т. е. поиск по полям внутри документов, включая включая вложенные объекты.

Документоориентированные базы не поддерживают JOIN между коллекциями, поэтому данные приходится хранить в денормализованном виде. Это приводит к избыточности.

Типичные сценарии использования:

- Каталоги продуктов с вариативными атрибутами

```
// Коллекция "products"
{
  "type": "laptop",
  "brand": "Lenovo",
  "specs": {"cpu": "i7", "ram_gb": 32}
}
{
  "type": "book",
  "author": "Достоевский",
  "genres": ["классика", "философия"]
}
```

- Профили пользователей со сложной структурой

```
{
  "user_id": "u-123",
  "preferences": {
    "theme": "dark",
    "notifications": {
      "email": true,
      "sms": false
    }
  },
  "activity_log": [{"action": "login", "time": "2023-10-25T12:00:00Z" }]
}
```

- Хранение статей с метаданными и мультимедиа:

```
{
  "slug": "big-data-guide",
  "title": "Руководство по Big Data",
  "tags": ["аналитика", "технологии"],
  "attachments": [{"type": "video", "url": "..."}]
}
```

Примеры документоориентированных баз: MongoDB, CouchDB.

Вопросы для конспекта

4. Опишите принципы организации хранения данных в документоориентированных базах.
5. К какому классу по CAP-классификации относятся документоориентированные базы данных?
6. Опишите основные сценарии использования документоориентированных баз данных.

Столбцовые или колонкоориентированные базы данных

Столбцовые базы данных, также как и реляционные, предназначены для хранения данных, представленных в виде таблиц.

Это NoSQL системы, которые могут быть как типа AP, так и CP.

Принципиальное отличие от реляционных — данные хранятся по столбцам, а не по строкам.

Главное преимущество столбцовой базы: очень высокая производительность для запросов на больших объёмах данных.

Строковое хранилище:

Row1: [UserID:101, Name:Иван, Age:30]

Row2: [UserID:102, Name:Мария, Age:25]

Записано на диске: 101, Иван, 30, 102, Мария, 25

Столбцовое хранилище:

Column UserID: [101, 102]

Column Name: ["Иван", "Мария"]

Column Age: [30, 25]

Записано на диске: 101, 102, Иван, Мария, 30, 25

Ключевые особенности столбцовых баз данных.

■ **Радикальное ускорение выполнения запросов.**

Хранение по столбцам позволяет радикально ускорить выполнение запросов к таблицам с большим данным.

Таблицы в столбцовых базах обычно широкие (очень много столбцов) и длинные (очень много записей).

Запрос обычно включает данные нескольких столбцов и при хранении по строкам приходится считывать строки целиком, чтобы извлечь нужные столбцы.

Когда данные записаны по столбцам, извлечь нужные значение из небольшого количества столбцов получается намного быстрее.

■ **Эффективное сжатие.**

Данные в столбцах сжимаются лучше чем строки.

Причина в том, что при хранении по столбцам рядом друг с другом оказываются однотипные данные.

■ Неэффективное слияние таблиц.

Операция JOIN либо не поддерживается, либо выполняется очень медленно.

В силу этого таблицы в столбцовых базах обычно сильно денормализованы, т. е. широкие.

Перед записью данных в столбцовую базу выполняются все необходимые слияния (JOIN) данных из разных таблиц.

■ Неэффективное удаление.

Удаление записей — это очень медленная операция и обычно при работе со столбцовой базой не применяется вообще.

Причина в том, что нужно удалить строку целиком, а её разные столбцы могут оказаться в разных местах хранилища, вплоть до разных узлов кластера, находящихся в разных датацентрах.

■ Неэффективное обновление.

Также медленно выполняется точечное обновление данных (в одной или нескольких записях) и также обычно не выполняется.

Вопросы для конспекта

7. Опишите принципы работы столбцовых баз данных. К какому классу по CAP-классификации они относятся?
8. Чем обусловлена эффективность столбцовых баз данных при работе с большими данными?
9. Какие операции над данными столбцовые базы выполняют не эффективно и почему?

Графовые базы данных

Графовые базы данных — это NoSQL-системы, предназначенные для работы со связанными данными.

Согласно CAP-классификации большинство графовых баз данных относятся к CP-системам.

Данные хранятся в виде узлов (сущностей), рёбер (связи) и свойств, что позволяет эффективно моделировать сложные сети.

Компонент	Описание	Пример
Узел	Объект	Пользователь, Товар, Город
Ребро	Связь между узлами	дружит_с, купил, находится
Свойство	Атрибуты узлов и рёбер	name:"Анна", weight:5, date:2023-01-01

Графовые БД решают задачи, в которых связи важнее самих данных.

- Социальные сети: поиск друзей, рекомендации («друзья друзей»).
- Рекомендательные системы: «Люди, купившие этот товар, также брали...».
- Обнаружение мошенничества: выявление цепочек транзакций.
- Знаниевые графы (Knowledge Graphs): медицинские онтологии (формализованные представления знаний в области медицины).

Пример: социальная сеть на Cypher (язык Neo4j)

```
CREATE // Создание узлов
  (anna:Person {name: "Анна", age: 30}),
  (boris:Person {name: "Борис", age: 25}),
  (tv:Product {name: "Телевизор"})
CREATE // Создание связей
  (anna)-[:FRIEND]->(boris),
  (boris)-[:BOUGHT {date: "2023-10-25"}]->(tv)
// Запрос: "Что купили друзья Анны?"
MATCH (anna:Person {name: "Анна"})-[:FRIEND]->(friend)-[:BOUGHT]->(product)
RETURN friend.name, product.name;
```

Результат: friend.name="Борис", product.name="Телевизор".

Задачи, для которых используются графовые базы данных.

- Связи имеют глубину больше двух шагов: «Показать друзей, потом друзей их друзей, затем их общие интересы».
- Связи могут достаточно часто меняться, как в соцсетях или при построении сетевых маршрутов.
- Выполняется исследование паттернов: поиск циклов, кластеризация сообществ и т. д.

Примеры графовых баз данных: Neo4j, Amazon Neptune, JanusGraph, ArangoDB.

Вопросы для конспекта

10. Опишите принципы работы графовых баз данных. К какому классу по CAP-классификации они относятся?

11. Опишите круг задач, для решения которых предназначены графовые базы данных.

Технологии повышения эффективности распределённых хранилищ данных

В распределённом хранилище большое количество данных должно быть распределено между большим числом взаимодействующих вычислительных узлов.

Чтобы обеспечить эффективное функционирование такой системы применяют различные технологии оптимизации.

- **Репликация** — это создание и поддержание в актуальном состоянии идентичных копий данных на нескольких физических серверах, входящих в распределённую систему хранения.

Репликация позволяет решить следующие задачи.

- **Повышение доступности и отказоустойчивости хранилища.**

Если основной сервер (мастер) падает, то происходит автоматическое переключение на одну из реплик.

- **Масштабирование чтения.** Запросы на чтение распределяются балансировщиком нагрузки между мастером и несколькими репликами. Это позволяет обрабатывать гораздо больший объем запросов на чтение, чем способен один сервер.
- **Геораспределение и уменьшение задержки.** Пользователи обращаются к географически ближайшей реплике для чтения данных, что резко снижает сетевую задержку и улучшает отзывчивость приложения.
- **Выделение реплик под специальные задачи.** Определённые реплики могут быть настроены для специфических задач, например подготовка отчётов (выделенные реплики выполняют тяжёлые аналитические запросы), резервное копирование (резервные копии создаются с выделенных для этого реплик).

- **Партиционирование.** В базах, в которых данные представлены в виде таблиц (реляционные и столбцовые), содержимое таблиц подвергается партиционированию — их разделяют на блоки по заданному критерию.

Например — посуточно, т. е. данные, относящиеся к одним и тем же суткам собраны в одну партицию.

Правило партиционирования задаётся при создании таблицы.

Разные партиции хранятся отдельно: в отдельных файлах или каталогах. Это значит, что при использовании для хранения базы распределённой файловой системы, партиции физически могут оказаться на разных узлах кластера, а возможно — в разных датацентрах.

Если партиционирование задано правильно, т. е. соответствует наиболее часто выполняемым фильтрациям и агрегациям (например, посуточной), то это даёт очень большое ускорение работы так как при обращении к данным считывается только необходимая партиция.

Аналогия: библиотека, где книги расставлены в алфавитном порядке авторов, а по годам издания. Если нужны книги 2023 года то легко сразу найти нужный раздел.

Ошибочное партиционирование при создании таблицы приводит к снижению эффективности.

- **Ошибочный критерий разбиения.** Например задано посуточное партиционирование, но наиболее часто данные фильтруются и агрегируются по фамилиям клиентов. В этом случае запросы будут выполняться существенно медленнее т. к. люди с одинаковыми фамилиями будут находиться в разных партициях.
- **Ошибочная гранулярность.** Например вместо посуточного выбрано почасовое разбиение — тогда метаданных, описывающих структуру партиций будет больше, чем данных. Наоборот вместо посуточного выбрано помесечное разбиение — будут проблемы с памятью при чтении партиций слишком большого размера.

- **Упорядоченное хранение.** В столбцовых базах данные внутри партиций почти всегда (не обязательно, но крайне желательно) хранятся в отсортированном виде. Критерии, по которым осуществляется сортировка, задаются при создании таблицы и не могут быть изменены впоследствии.

Операции поиска, фильтрации, агрегации данных по критериям, по которым предварительная выполнена сортировка, выполняются существенно быстрее.

В случае ошибки проектирования (выбран не самый часто применяемый критерий сортировки) эффективность работы с таблицей падает.

Партиционирование и предварительная сортировка приводят к тому, что обновление и удаление данных в таких таблицах либо невозможны, либо крайне затруднены. Например, типичная ситуация — разрешено удалить только партицию целиком.

Поэтому при появлении в базе ошибочных данных их просто отфильтровывают при выполнении запросов.

По этой причине в строчных (реляционных) базах упорядоченное хранение используется достаточно редко.

Наряду с партиционированием для распределения нагрузки на базу применяют шардирование (shard — осколок).

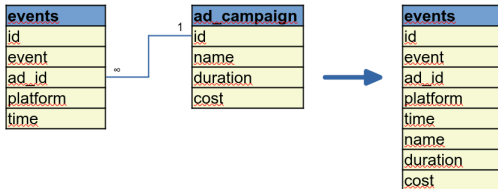
- **Шардирование** — это разделение коллекции записей (обычно это применяют к таблицам) на несколько частей (шард) по некоторому критерию и передача каждой полученной шарды под управления одного из нескольких физических серверов баз данных.

Например шардирование может производиться по регионам, к которым принадлежат пользователи, или по значениям `user_id`. Для равномерного распределения записей между серверам обычно вычисляется хеш от одного или нескольких значений (`user_id`, `platform`, и т. п.), затем вычисляется остаток от хеша деления на

число имеющихся серверов. Полученное значение — это номер шарды, в которую должна быть помещена запись.

При шардировании базы дополнительно задействуется диспетчеризация запросов между имеющимися физическими серверами. Таким образом серверы работают со своим шардами незаметно для пользователя — ему вся их совокупность представлена как единая база.

- **Денормализация.** Объединения данных из нескольких таблиц (JOIN), типичные для реляционных баз, выполняются неэффективно на больших таблицах, которые хранятся распределённо. Поэтому такие таблицы хранят в денормализованном виде — операция слияния выполнена перед сохранением.



Это приводит к появлению в базе широких (много столбцов) таблиц с большой избыточностью данных.

Эффективно работать с такими таблицами позволяют столбцовые базы. Поэтому денормализация применяется в основном для них.

- **Кеширование** — хранение часто запрашиваемых данных или результатов сложных запросов в очень быстрой памяти, например, непосредственно в оперативной памяти сервера или во вспомогательных базах ключ-значение.
- **Материализованные представления** — сохранение на диске результата предварительно вычисленного запроса, например сложного объединения или агрегации. Обновляются периодически или по триггеру. Пример этого — витрины данных, которые упоминались выше.

Вопросы для конспекта

12. Что такое репликация и как она помогает повысить эффективность работы распределённого хранилища данных?
13. Что такое партиционирование и как оно помогает повысить эффективность работы хранилища данных?
14. Что такое упорядоченное хранение? Для каких типов баз данных оно применяется и почему позволяет повысить эффективность их работы?
15. Что такое шардирование и как оно помогает повысить эффективность работы распределённого хранилища данных?
16. В каких случаях денормализация таблиц позволит повысить эффективность работы хранилища данных?
17. Зачем применяются кеширование и материализованные представления и как это помогает повысить эффективность работы хранилища данных?

Вопросы для конспекта

- 1 Опишите принципы работы баз данных типа ключ-значение.
- 2 К какому типу относятся базы ключ-значение, согласно CAP-классификации?
- 3 Для каких задач наилучшим образом подходят базы данных ключ-значение?
- 4 Опишите принципы организации хранения данных в документоориентированных базах.
- 5 К какому классу по CAP-классификации относятся документоориентированные базы данных?
- 6 Опишите основные сценарии использования документоориентированных баз данных.
- 7 Опишите принципы работы столбцовых баз данных. К какому классу по CAP-классификации они относятся?
- 8 Чем обусловлена эффективность столбцовых баз данных при работе с большими данными?
- 9 Какие операции над данными столбцовые базы выполняют не эффективно и почему?
- 10 Опишите принципы работы графовых баз данных. К какому классу по CAP-классификации они относятся?
- 11 Опишите круг задач, для решения которых предназначены графовые базы данных.
- 12 Что такое репликация и как она помогает повысить эффективность работы распределённого хранилища данных?
- 13 Что такое партиционирование и как оно помогает повысить эффективность работы хранилища данных?
- 14 Что такое упорядоченное хранение? Для каких типов баз данных оно применяется и почему позволяет повысить эффективность их работы?
- 15 Что такое шардирование и как оно помогает повысить эффективность работы распределённого хранилища данных?

- 16 В каких случаях денормализация таблиц позволят повысить эффективность работы хранилища данных?
- 17 Зачем применяются кеширование и материализованные представления и как это помогает повысить эффективность работы хранилища данных?

Лекция 5. Столбцовая база данных ClickHouse

Понятие веб-аналитики

- **Веб-аналитика** — это процесс сбора, анализа и интерпретации данных о поведении пользователей на веб-сайтах или в веб-приложениях. Её основная цель — понять, как пользователи взаимодействуют с сайтом, чтобы оптимизировать его работу, улучшить пользовательский опыт и достичь бизнес-целей.

Входные данные для веб-аналитики — логи всех действия пользователей на сайте — переходов по ссылкам, кликов мышкой по управляющим элементам, кликов по рекламным баннерам.

Для высоконагруженных сайтов, вроде соцсетей, таких данных чрезвычайно много.

Стандартное решение, т. е. сохранение данных в «обычных» SQL-базах, не годится. Такие базы будут работать очень медленно.

Компромиссное решение проблемы большой поток входных данных / медленное хранилище — агрегация.

- **Преагрегация** — метод понижения объёма входящих данных за счёт их предварительной обработки.

Например, регистрируются показы пользователям рекламных баннеров (ситуация, когда пользователь открыл страничку с рекламой) и переходы (клики) по ним. А в базе сохраняется только CTR (Click-Through Rate) — процент всех кликов по отношению к количеству всех показов.

Ещё пример: регистрируется время захода на страничку каждого пользователя, а в базе сохраняется только количество посетителей в день.

Этот подход позволяет уменьшить размер хранимых данных, однако имеет ряд недостатков:

- необходимость заранее зафиксировать список доступных для пользователя отчётов (отсутствие возможности построить произвольный отчёт);

- агрегации по большому количеству ключей или по ключам высокой кардинальности (количество возможных значений), таким как URL, может приводить к обратному эффекту, т. е. к увеличению объёма данных;
- поддержание логической целостности при хранении большого количества разных агрегаций затруднительно.

Лучший подход — хранить все сырые данные в том виде, как они поступают и производить необходимые вычисления в момент выполнения запроса к базе.

Для этого требуется хранилище данных, способное в реальном времени принимать очень большие объёмы данных и с очень высокой эффективностью обрабатывать их — с интерактивными задержками, т. е. достаточно быстро, чтобы режим работы можно было рассматривать как интерактивный.

Набор технических и технологических решений такого рода называют OLAP.

- **OLAP (Online Analytical Processing)** — совокупность инструментов и методов, предназначенных для сложного анализа больших объёмов данных в реальном времени (т. е. запросы выполняются быстро).

Этот инструментарий позволяет быстро и гибко «разрезать и исследовать» огоромные наборы данных по множеству измерений (время, география, источник трафика, тип устройства, продукт и т.д.), агрегировать их (суммы, средние, уникальные пользователи) и отвечать на сложные бизнес-вопросы на лету, т. е. без существенных задержек.

В частности, SQL запросы, применяемые в рамках OLAP — это запросы на выборку данных (не ввод и не модификация), которые отличаются сложными вычислениями (например, агрегациями, обработкой строк, арифметическими операциями) по огромным наборам данных.

Для OLAP скорость обработки является наивысшим приоритетом, даже в ущерб надёжности. Нередкая ситуация при выполнении OLAP-запросов — потеря части данных.

OLAP противопоставляют методам транзакционных запросов **OLTP**, Online Transaction Processing, используемых в реляционных базах.

Вспомним, что транзакция — это операция, которая выполняется с соблюдением требований ACID, т. е. либо выполняется целиком либо не выполняется вовсе. Потеря данных в ходе транзакции исключена.

- **OLTP (Online Transaction Processing)** — запросы выполняются с соблюдением ACID, затрагивает небольшое количество строк и столбцов, может включать не только чтение, но и модификацию данных и при этом выполняется с максимальной надёжностью.

Коротко:

- **OLAP** — чтение и агрегация гигантских массивов данных, в основном только дозапись крупных блоков данных в таблицы.
- **OLTP** — много коротких операций, обновлений, удалений.

Вопросы для конспекта

1. Какие задачи решает веб-аналитика?
2. Что такое агрегация данных? Для чего предназначена, плюсы и минусы?
3. Что такое OLAP?
4. Что такое OLTP?

Основные особенности ClickHouse

ClickHouse — это столбцовая СУБД, разработанная в Яндексе и имеющая открытый исходный код (opensource).

Представляет собой хранилище данных для решения задач веб-аналитики на основе подхода OLAP.

Исходное предназначение ClickHouse — обеспечение работы Яндекс Метрики.

Изначально в Яндекс Метрике для построения отчётов использовались предварительно агрегированные данные.

Для преодоления недостатков такого подхода была разработана ClickHouse.

Первый прототип ClickHouse появился в 2009 году, к концу 2014 года была запущена Метрика 2.0, работающая на базе ClickHouse.

- **Структурированные данные.** ClickHouse хранит структурированные данные, представленные в виде таблиц.

- **Расширенный SQL.** Управление данными в ClickHouse производится при помощи расширенной версии SQL.
- **Денормализованные таблицы.** Хотя слияние JOIN поддерживается в ClickHouse, её использование не рекомендуется, так как это очень медленная и ресурсоёмкая операция. Поэтому таблицы стараются строить в денормализованном виде.
- **Широкие таблицы.** В силу денормализации таблицы обычно имеют очень много столбцов: от нескольких десятков до нескольких сотен.
- **Хранение данных по столбцам.** В отличие от реляционных баз, в ClickHouse данные хранятся по столбцам.

Почему это позволят существенно ускорить выполнение запросов?

Основное бутылочно горлышко при выполнении запроса к базе — чтение данных из долговременной памяти (магнитные или твердотельные диски) в оперативную.

При выполнении запросов к строчной базе (таковы все реляционные) простейший и самый неэффективный вариант — считывать строки таблиц целиком, а потом выбирать нужные значения.

Это очевидным образом можно оптимизировать.

Так как типы данных в столбцах строго заданы, для каждого столбца ширина в байтах известна .

Следовательно, можно рассчитать смещение относительно начала строки для каждого столбца и считывать не всё строку, а только нужные фрагменты.

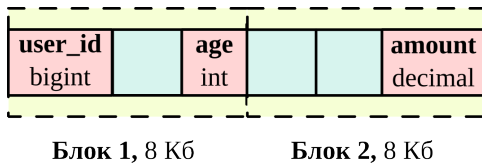
Однако все накопители долговременной памяти — блочные устройства. Это значит, что минимальная единица хранения — блок в несколько килобайт. Данные можно считывать и записывать только блоками.

Даже если нужно получить одно целое число, приходится считывать блок целиком.

Поэтому даже используя смещения и пытаясь считывать только нужные данные из строк, придётся считывать много лишнего.

Пусть имеется таблица, записанная по строкам, и из неё требуется считать три поля: `user_id` (bigint, 8 байт), `age` (int, 4 байта), `amount` (decimal, 8 байт).

Если они расположены в строке как показано на рисунке, для их чтения (всего $8+4+8=20$ байт) потребуется считать целиком два блока (всего $8+8=16$ Кбайт=16384 байт).

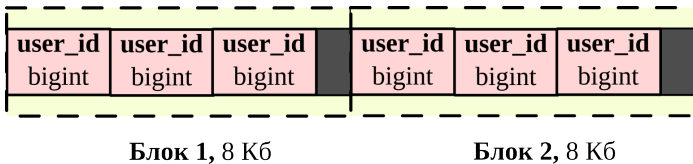


Из общего объёма данных, полученного из долговременной памяти, приходится отбрасывать

$$(1 - 20/16384) \times 100\% = 99.9\%.$$

Представим, что наша таблица очень большая — в таком режиме нужно прочитать не одну а несколько сотен миллионов записей (а бывает и несколько миллиардов). Ясно, что работа в таком режиме крайне неэффективна.

Теперь предположим, что данные хранятся в по столбцам. На рисунке показано как записаны данные столбца `user_id` и точно также хранятся все остальные столбцы.



Возможно в один блок не будет уместиться целое число значений и часть блока окажется не занятой. На рисунке незанятая часть блока помечена тёмно-серой заливкой. (Это не обязательно должно быть так. Возможно байты каких-то значений будут распределены между соседними блоками.)

Например, пусть имеется столбец текстовых данных VARCHAR(10) — каждое занимает 10 байт. В одном блоке уместится

$$\lfloor 8 \times 1024 / 10 \rfloor = 819$$

значений и останется не занятым

$$8 \times 1024 - 10 \times 819 = 2$$

байта. Получается, что при чтении одного блока впустую будет прочитано

$$(2/8192) \times 100\% = 0.024\%.$$

Хорошо видно, что объёмы накладных расходов при чтении базы по строкам и по столбцам различаются очень сильно.

Характерные числа выигрыша в производительности (цитата):

*«Нам требовалось анализировать петабайты данных о сетевом трафике, и наше строчное хранилище просто не справлялось — аналитические запросы выполнялись часами. После миграции на столбцовую архитектуру произошло то, что команда называла "эффектом волшебства" — типичный запрос на агрегацию по 5 миллиардам записей сократился с 2.5 часов до 17 секунд. Это было не просто количественное, а качественное изменение — аналитики впервые смогли работать в интерактивном режиме. Ключевым техническим аспектом успеха стал тот факт, что 80% наших запросов использовали только 6-8 полей из таблиц, содержащих 40+ столбцов. Колоночное хранение позволяло считывать с диска только 15-20% объема данных по сравнению с построчным подходом.»**

* <https://sky.pro/wiki/analytics/stolbtsovye-bazy-dannyh-osobennosti-preimushchestva-i-primenenie>

- **Репликация данных и целостность.** ClickHouse использует асинхронную схему репликации. Это означает, что данные, выставленные на одну из реплик, не сразу становятся доступны на всех репликах, а репликация происходит в фоновом режиме. Восстановление после большинства сбоев осуществляется автоматически или полуавтоматически в сложных случаях.
- **CP-тип.** ClickHouse, согласно CAP-теореме, можно отнести к CP типу. Это означает, что в ситуации сетевого раздела (Partition) ClickHouse выберет целостность (Consistency), отказавшись от доступности (Availability) для операций записи, если не может гарантировать целостность данных.
- **Распараллеливание запросов.** Так как данные хранятся по столбцам, при выполнении запросов каждый столбец таблицы можно обрабатывать независимо от других. В силу этого запросы естественным образом распараллеливаются.

Например, одно ядро читает столбец денежных сумм и выполняет группировку и агрегацию, а в второе независимо от первого делает тоже самое со столбцом, например, возрастов клиентов.

При строчном хранении также можно ввести параллелизм, но это сложнее, а значит будет больше накладных расходов. На первом этапе разные ядра читают подмножества строк и выделяют нужные значения. Затем эти данные объединяются и раздаются по ядрам для вычисления группировок и агрегация по разным столбцам.

- **Векторизация вычислений.** Благодаря столбовому хранению, данные естественным образом представлены в виде однородных массивов (столбцов). В ClickHouse для их обработки используются SIMD-инструкции (векторные инструкции).

Примечание

SIMD-инструкции (Single Instruction, Multiple Data) — это команды машинного языка процессора, которые позволяют одновременно выполнять одно и то же действие сразу над несколькими данными одновременно. Такие инструкции называются векторными. Они в той или иной мере поддерживаются всеми современным процессорами.

- **Эффективное сжатие данных.** При столбцовом хранении данных компрессия работает более эффективно чем при строчном хранении.

Данные в одном столбце часто имеют однородный тип (все числа, даты или строки из одной предметной области) и содержат много повторяющихся или близких значений.

Например, в столбце `age` будут только числа, в столбце `city_name` — повторяющиеся названия городов, а в столбце `price` — числовые значения из одного диапазона.

Алгоритмам сжатия гораздо проще находить и эффективно упаковывать такие последовательности однотипных данных, чем разнородные данные в строке, где число `user_id` может соседствовать со строкой `username` и датой `timestamp`.

- **Целые числа лучше вещественных.** Колонки с целочисленными и категориальными данными (принимают одно из нескольких значений, пример «android», «ios», «pc») обрабатываются быстрее чем колонки

вещественных чисел. В первую очередь — из-за более быстрой компрессии.

- **Планирование таблиц.** При создании таблицы требуется продумать её структуру.

Нужно задать, по какому полю будет выполняться партиционирование таблиц. Компрессия выполняется по партициям. Данные одной партии хранятся в одной папке. Поэтому важно задать партиционирование по столбцу, по которому чаще всего планируется агрегация и отбор. Обычно — по дням.

Нужно продумать правильную гранулярность — по дням, по часам и т. д. Первичный ключ присваивается не каждой записи как в реляционных базах, а блокам строк (гранулам). Это нужно, чтобы ключ не был чрезвычайно длинным.

Нужно задать критерий сортировки при хранении — данные столбцов хранятся в отсортированном виде, что ускорят поиск при выполнении

запросов. Следовательно, нужно задать сортировку по тому столбцу, по которому чаще всего выполняется отбор. Обычно — даты и время.

• **Правильный выбор движка таблицы.** При создании таблицы нужно выбрать движок в соответствии с планируемым сценарием использования таблицы.

Движок определяет

- Как и где хранятся данные (в долговременной памяти, в ОЗУ, с репликацией или без, в распределённой файловой системе).
- Стратегии обработки запросов (возможность параллелизма, оптимизация для частых вставок данных, возможность использования индексов).

Наиболее часто используемый движок называется «MergeTree».

В OLTP-базах также используются разные движки (например, «InnoDB» и «MyISAM» в MySQL). Но в ClickHouse движков значительно больше и их влияние на работу значительно выше.

Вопросы для конспекта

5. Поддерживает ли ClickHouse работу с таблицами при помощи SQL?
6. Почему таблицы в ClickHouse обычно широкие?
7. Какой способ хранения данных применяется в ClickHouse? В чём преимущества столбцового хранения данных?
8. К какому типу, согласно CAP-классификации, относится ClickHouse?
9. Почему ClickHouse хорошо адаптирована для параллельного выполнения запросов?
10. Как и для чего в ClickHouse используются векторные вычисления?
11. Какой способ хранения табличных данных обеспечивает более эффективную компрессию данных и почему?
12. Какой тип данных предпочтительнее для хранения в таблицах ClickHouse с точки зрения эффективности?
13. Что нужно задать при создании таблицы в ClickHouse.

14. С какой целью при создании таблицы в ClickHouse требуется выбрать движок?

Особенности выполнения запросов в ClickHouse

В этом разделе мы будем обсуждать работу с ClickHouse на примере таблицы trips, описывающей поездки такси Нью-Йорка.

```
CREATE TABLE nyc_taxi.trips (  
    trip_id                UInt32,  
    pickup_datetime        DateTime, -- дата и время начала поездки  
    dropoff_datetime       DateTime, -- дата и время окончания поездки  
    pickup_longitude       Nullable(Float64), -- GPS-долгота точки начала поездки  
    pickup_latitude        Nullable(Float64), -- GPS-широта точки начала поездки  
    dropoff_longitude      Nullable(Float64), -- GPS-долгота точки окончания поездки  
    dropoff_latitude       Nullable(Float64), -- GPS-широта точки окончания поездки  
    passenger_count        UInt8, -- количество пассажиров  
    trip_distance          Float32, -- длина поездки  
    fare_amount            Float32, -- тариф поездки  
    extra                  Float32, -- доп. в час пик $0.5/миля, ночью $1/миля  
    tip_amount            Float32, -- чаевые  
    tolls_amount          Float32, -- дорожный сбор  
    total_amount           Float32, -- полная сумма оплаты  
    payment_type           Enum('CSH'=1, 'CRE'=2, 'NOC'=3, 'DIS'=4, 'UNK'=5),  
    pickup_ntaname        LowCardinality(String), -- район точки начала поездки  
    dropoff_ntaname       LowCardinality(String) -- район окончания поездки  
)  
ENGINE = MergeTree  
PRIMARY KEY (pickup_datetime, dropoff_datetime);
```


Расшифровка аббревиатур в поле payment_type, тип оплаты:

- CSH — наличные
- CRE — кредитная карта
- NOC — отсутствие оплаты (может означать отказ от оплаты, бесплатный проезд или ошибку)
- DIS — скидка
- UNK — неизвестный тип оплаты или неизвестное значение

В таблице содержится около миллиона записей.

- **Отказ от ACID-транзакций.** ACID-транзакции выполняются медленно и в ClickHouse поддерживаются частично. Например, запись пакета данных запросом INSERT выполняется атомарно. Невозможна ситуация, когда записывается только часть записей из пакета, но в редких случаях весь пакет может быть не записан.

Поэтому хотя ClickHouse вполне надёжная база, она не годится для работы с чувствительными данными, например, с банковскими.

- **Чтение чаще чем запись.** Штатный режим работы ClickHouse, на который он ориентирован — данные добавляются в таблицы крупными пакетами и почти никогда не изменяются и не удаляются.

- **Сложность удаления и обновления данных.** Удаление и модификация данных — нежелательные и очень тяжёлые операции. Операции удаления DELETE и изменения UPDATE записей нельзя применять для отдельных данных. Можно обработать только целый блок — партицию данных (партиционирование данных обсуждалось в предыдущих лекциях).

Поэтому если при работе с ClickHouse в базу попадают ошибочные данные, они обычно остаются там навсегда и при выполнении запросов их просто отфильтровывают.

- **Материализованные представления — это триггеры.** Вспомним, что материализованные представления — это «легкие» таблицы, созданные в результате выполнения запросов к основным таблицам, чтобы ускорить работу с часто используемыми данными.

Материализованные представления хранятся в долговременной памяти, а не перевычисляются при каждом обращении. В ClickHouse перевычисление материализованных представлений производится автоматически при обновлении исходной таблицы.

- **Не применять запросы без ограничений выдачи.** Обычно в таблицах ClickHouse много столбцов (десятки или сотни) и очень много строк (миллион строк считается небольшим размером).

Поэтому нельзя выполнять запросы без ограничения объёма вывода из-за потенциально огромного объёма получаемой информации. Часто такие запросы будут блокироваться на уровне политик работы с базой.

```
SELECT
    * -- Так делать нельзя!
FROM trips;
```

Практически никогда нет необходимости выполнять запросы без агрегации и фильтрации. В процессе отладки запросы желательно выполнять с ограничением вывода с использованием LIMIT. И крайне желательно всегда перечислять требуемые столбцы.

```
SELECT
    pickup_ntaname, -- Всегда нужно указывать требуемые столбцы
    dropoff_ntaname
FROM trips LIMIT 10; -- В процессе отладки устанавливаем лимит выдачи
```

● **Тип данных Array.** В ClickHouse кроме стандартных типов данных, ячейки таблицы могут хранить массивы. Элементы массива должны быть одного типа. Таблицу с массивами в ячейках можно получить в результате операции группировки.

Функция `groupUniqArray` представит результаты группировки в виде массива и удалит повторяющиеся значения.

В примере ниже показано, компании из какого количества людей уезжали из каждого из районов.

```
SELECT
  pickup_ntaname,
  groupUniqArray(passenger_count) as pc
FROM trips
WHERE pickup_ntaname != '' GROUP BY pickup_ntaname ORDER BY length(pc)
```

	A-Z pickup_ntaname	pc
> 16	Brighton Beach	> 1
> 17	West Brighton	> 4
> 18	Woodlawn-Wakefield	> 1
▼ 19	East Flatbush-Farragut	▼ [2]
19.1		5
19.2		1
▼ 20	Pelham Bay-Country Club-City Island	▼ [2]
20.1		1
20.2		3
▼ 21	Parkchester	▼ [2]
21.1		2
21.2		1
> 22	West Farms-Bronx River	> 2 [+1]
> 23	Queens Village	> 2 [+1]
> 24	East Flushing	> 2 [+1]
> 25	Belmont	> 2 [+1]

- **Расширенный набор функций агрегации.** В стандартном SQL имеются пять функций агрегации: COUNT, MIN, MAX, SUM, AVG.

В примере ниже предполагается использование таблицы trips с информацией о поездках такси в Нью-Йорке.

```
SELECT
    pickup_ntaname, -- район начала поездки
    COUNT(*) AS cnt,
    MIN(trip_distance) AS min_distance,
    MAX(trip_distance) AS max_distance,
    SUM(trip_distance) AS sum_distance,
    AVG(trip_distance) AS avg_distance
FROM trips WHERE pickup_ntaname != '' GROUP BY pickup_ntaname;
```

	A: pickup_ntaname	123 cnt	123 min_distance	123 max_distance	123 sum_distance	123 avg_distance
1	Maspeth	55	0	18,77	295,1099997759	5,3656363596
2	Clinton Hill	530	0	18,86	1 717,9399991781	3,2413962249
3	Schuylerville-Throgs Neck-Edgewater Park	11	0,5	26,2	78,510001719	7,1372728835
4	North Side-South Side	3 674	0	21,1	12 029,7600094769	3,2742950489
5	park-cemetery-etc-Brooklyn	101	0	24,05	361,1499985792	3,5757425602
6	Woodhaven	22	0	10,52	95,0899996161	4,3222727098

ClickHouse поддерживает стандартные функции агрегации, а также вводит дополнительные.

- Приближённые функции.
 - Функции с параметром, позволяющим настроить их работу.
 - Функции с комбинатором (суффиксом после имени), модифицирующем поведение функций.
- **Приближённые агрегатные функции.** ClickHouse позволяет жертвовать точностью ради производительности и при агрегации выполнять вычисления приближённо.

Приближенные вычисления организованы на основе семплирования данных (рассматривается не весь набора данных, а его случайное подмножество) и использования вероятностных методов.

Такой подход оправдан и погрешность незначительна, если полный набор данных содержит много значений — это как раз типично для таблиц ClickHouse.

Функция `uniq` — найти примерное количество уникальных элементов в каждой группе. Точно это вычисляются функцией `uniqExact`.

Функция `median` — найти приближённое значение медианы. Точное значение вычисляется при помощи `medianExact`.

SELECT

```
pickup_ntaname, -- район начала поездки
uniq(dropoff_ntaname) AS u, -- район окончания поездки
uniqExact(dropoff_ntaname) AS uE,
median(trip_distance) AS m,
medianExact(trip_distance) AS mE
```

FROM trips **WHERE** pickup_ntaname != '' **GROUP BY** pickup_ntaname;

	A: pickup_ntaname	123 u	123 uE	123 m	123 mE
1	Maspeth	26	26	4,8499999046	4,85
2	Clinton Hill	72	72	2,40000000954	2,4
3	Schuylerville-Throgs Neck	8	8	3,40000000954	3,4
4	North Side-South Side	115	115	2,70000000477	2,7
5	park-cemetery-etc-Brookl	42	42	2,22000000286	2,22
6	Woodhaven	17	17	3,95000000477	4,46

• **Функции с параметром.** Такие функции имеют две пары круглых скобок. Первая пара задаёт параметры, вторая — столбец для обработки.

Функция вычисления гистограммы `histogram`. Первый параметр — количество ячеек (столбцов) гистограммы. Второй — столбец для обработки.

Возвращается массив кортежей следующего формата:

$$[(lw1, up1, ht1), \dots, (lwN, upN, htN)]$$

где: `lw` — нижняя граница ячейки, `up` — верхняя граница ячейки, `ht` — вычисленная высота ячейки.

В примере ниже построена гистограмма числа поездок в зависимости от числа пассажиров. Нецелые значения числа поездок получаются из-за того, что гистограмма вычисляется приближённо.

```
SELECT histogram(10)(passenger_count)
FROM trips;;
```

histogram(10)(passenger_count)		
l23 1	l23 2	l23 3
10	10	10
0	0,5	87 670,375
0,5	1,5	543 541,375
1,5	2,5	200 482,25
2,5	3,5	53 373,875
3,5	4,5	28 722,375
4,5	5,5	48 496
5,5	6,5	33 871
6,5	7,5	4 499,875
7,5	8,5	1,75
8,5	9	1,125

● **Комбинатор функций агрегации -If** может быть добавлен к имени любой агрегатной функции. В этом случае агрегатная функция принимает дополнительный аргумент — условие . Функция обрабатывает только строки, которые удовлетворяют условию. Если условие не срабатывало ни разу, она возвращает значение по умолчанию (обычно нули или пустые строки).

Функции `avgIf`, `maxIf`, `minIf` вычисляют соответствующие значения только для тех значений, для которых в ячейке второго столбца записано значение «Истина».

В примере ниже вычисляется средняя и максимальная сумма чаевых, которые были заплачены компаниями из нескольких пассажиров при условии, что длина поездки была более 5 миль.

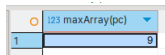
```
SELECT
    passenger_count,
    avgIf(tip_amount, trip_distance > 5) as avg_tip,
    maxIf(tip_amount, trip_distance > 5) as max_tip
FROM trips
GROUP BY passenger_count
```

	123 passenger_co	123 avg_tip	123 max_tip
1	0	0	0
2	1	4,371732018	105
3	2	4,0386576416	75
4	3	3,7147705637	120
5	4	3,2320888259	35
6	5	4,3105422787	80
7	6	4,166239354	35
8	7	не число	0
9	8	не число	0
10	9	не число	0

- **Комбинатор агрегатных функций -Array.** Функция с таким суффиксом предназначена для обработки столбцов, в которых хранятся массивы ячеек. Если агрегатная функция принимает несколько аргументов, это должны быть массивы одинаковой длины.

В примере ниже функция `maxArray` применена к результатам выполнения запроса, который рассматривался выше, см. функцию `groupUniqArray`. Результат этого запроса — максимальное значение по всем массивам.

```
SELECT
    maxArray(pc)
FROM (
    SELECT
        groupUniqArray(passenger_count) as pc
    FROM trips
    WHERE pickup_ntaname != '' GROUP BY pickup_ntaname
)
```



123 maxArray(pc)
9

Вопросы для конспекта

15. Поддерживает ли ClickHouse ACID-транзакции?
16. На какой режим работы с данными (чтение, запись, обновление) ориентирован ClickHouse?
17. Что обычно делают, если в таблицу ClickHouse попадают ошибочные данные?
18. Всегда ли вычислительные операции над данными выполняются в ClickHouse точно? Приведите примеры.
19. Запросы какого вида крайне нежелательно применять к таблицам ClickHouse? Почему?
20. Что собой представляют данные типа Array в таблицах ClickHouse?
21. Какие агрегатные функции имеются в ClickHouse?
22. Что собой представляют функции агрегации с параметром?
23. Что такое комбинатор функций агрегации? Приведите примеры.

Вопросы для конспекта

- 1 Какие задачи решает веб-аналитика?
- 2 Что такое агрегация данных? Для чего предназначена, плюсы и минусы?
- 3 Что такое OLAP?
- 4 Что такое OLTP?
- 5 Поддерживает ли ClickHouse работу с таблицами при помощи SQL?
- 6 Почему таблицы в ClickHouse обычно широкие?
- 7 Какой способ хранения данных применяется в ClickHouse? В чём преимущества столбцового хранения данных?
- 8 К какому типу, согласно CAP-классификации, относится ClickHouse?
- 9 Почему ClickHouse хорошо адаптирована для параллельного выполнения запросов?
- 10 Как и для чего в ClickHouse используются векторные вычисления?
- 11 Какой способ хранения табличных данных обеспечивает более эффективную компрессию данных и почему?
- 12 Какой тип данных предпочтительнее для хранения в таблицах ClickHouse с точки зрения эффективности?
- 13 Что нужно задать при создании таблицы в ClickHouse.
- 14 С какой целью при создании таблицы в ClickHouse требуется выбрать движок?
- 15 Поддерживает ли ClickHouse ACID-транзакции?
- 16 На какой режим работы с данными (чтение, запись, обновление) ориентирован ClickHouse?
- 17 Что обычно делают, если в таблицу ClickHouse попадают ошибочные данные?
- 18 Всегда ли вычислительные операции над данными выполняются в ClickHouse точно? Приведите примеры.
- 19 Запросы какого вида крайне нежелательно применять к таблицам ClickHouse? Почему?

- 20 Что собой представляют данные типа Array в таблицах ClickHouse?
- 21 Какие агрегатные функции имеются в ClickHouse?
- 22 Что собой представляют функции агрегации с параметром?
- 23 Что такое комбинатор функций агрегации? Приведите примеры.

Лекция 6. Документоориентированная база MongoDB

Общие сведения о MongoDB

MongoDB — это документоориентированная СУБД класса NoSQL с открытым исходным кодом.

Вместо хранения данных в таблицах со строками и столбцами, как в реляционных СУБД, MongoDB хранит данные в виде коллекций документов в формате BSON (Binary JSON).

Реляционная база данных

- В базе несколько таблиц. Каждая таблица состоит из записей (строк). Запись — единица хранения, информация об одном объекте.
- Строгая схема данных: количество полей и типы данных одинаковые для всех записей одной таблицы.
- Язык запросов SQL.

MongoDB

- В базе несколько коллекций. Каждая коллекция состоит из документов в формате JSON. Документ — единица хранения, информация об одном объекте.
- Схема данных отсутствует (scheme-less): документы в одной коллекции могут иметь неодинаковые поля.
- Язык запросов JavaScript.

- **Применение.** Большие объемы неструктурированных или быстро меняющихся данных.
- **CP-система** В стандартной конфигурации MongoDB является CP-системой (Consistency, Partition Tolerance). Она жертвует доступностью (Availability) в момент сетевого раздела (Partition), чтобы обеспечить консистентность данных.
- **Иерархическая модель данных.** В MongoDB полем документа может быть другой документ. Поэтому модель данных в общем случае — иерархическая.

Однако, при необходимости можно воссоздать табличную модель данных, как в реляционных базах.

Для этого требуется поддерживать строгую схему данных на уровне пользовательского кода, т. е. в каждой коллекции создавать документы с только с заданной структурой.

- **Поддержка горизонтального масштабирования.** Документы хранятся независимо друг от друга и поэтому горизонтальное

масштабирование за счёт шардирования (распределения данных по множеству серверов) поддерживается естественным образом.

- **Ограниченная поддержка ACID-транзакций** Поддерживаются ACID-транзакции на уровне отдельного документа. Многодокументные транзакции также поддерживаются, но работают менее эффективно.
- **Нет связей между коллекциям.** Связи и объединения (JOIN) в обычном понимании не поддерживаются. Используется денормализация данных и вложение документов.

Но при необходимости можно выполнить поиск документов по заданному критерию \$lookup — аналог JOIN.

- **Сравнение с базами ключ-значения.** Базы ключ-значения просто хранят пары, которые могут быть чем угодно.

MongoDB до некоторой степени похожа на такие базы: JSON также представляет собой набор пар ключ-значение.

Отличие в том, что MongoDB хранит структурированные и полуструктурированные документы с возможностью сложных запросов по содержимому, т. е. в глубь иерархии.

У баз ключ-значение есть возможность искать только по содержанию ключей или значений но без учёта наличия у значений собственной структуры типа ключ-значение.

- **Сравнение со столбцовыми базами.** MongoDB не так эффективна при поиске документов со заданным значением в заданном поле как столбцовые.

Однако также допускает достаточно эффективную работу с большим данными на кластере и при этом более адаптирована к изменяющимся данным, так как не требует строго продуманной структуры хранения.

- **Сравнение с графовыми базами.** MongoDB неэффективна для анализа сложных (например, многоступенчатых) связей между сущностями. Графовые СУБД лучше справляются с подобными задачами.

- **Сильные стороны MongoDB**

- **Гибкость схемы.** Быстрое прототипирование и лёгкая адаптация структуры данных под изменяющиеся требования.
- **Горизонтальная масштабируемость.** Отличная поддержка шардирования для распределения нагрузки и данных по множеству серверов.
- **Высокая производительность записи.** Оптимизирована для сценариев с большим объемом операций вставки и обновления.
- **Отсутствие JOIN и денормализация.** Это часто приводит к более быстрому чтению данных, так как вся необходимая информация находится в одном документе.
- **Удобство для разработчиков.** Данные представлены в формате, очень близком к объектам в объектно-ориентированных языках программирования, что уменьшает затраты на преобразование.

- Слабые стороны MongoDB

- **Денормализация.** Денормализация приводит к избыточности данных и проблемам с согласованностью, если не управлять этим правильно.
- **Отсутствие JOIN** Хотя операция поиска по значению \$lookup существует, сложные связи между коллекциями обрабатываются менее эффективно, чем в SQL.
- **Потребление памяти.** Требуется много оперативной памяти для эффективной работы, так как активно использует кеширование и память для хранения данных.
- **Сложность транзакций.** Хотя транзакции поддерживаются, их использование в распределенной среде (шарды) сложнее и может негативно влиять на производительность по сравнению с реляционными базами.

Вопросы для конспекта

1. Что является аналогом таблиц и записей в MongoDB?
2. Чем объекты хранения MongoDB отличаются от объектов хранения реляционных баз данных?
3. Что в MongoDB используется в качестве языка запросов?
4. Какая модель данных используется в MongoDB? Чем она отличается от модели данных в реляционных базах?
5. Как в MongoDB поддерживается масштабирование?
6. Имеется ли возможность поддерживать связи между объектами базы, аналогично тому, как это сделано в реляционных базах?
7. В чём сходства и отличия MongoDB и баз ключ-значение?
8. Сравните MongoDB со столбцовыми базами.
9. Сравните MongoDB с графовыми базами.

10. Сформулируйте сильные и слабые стороны MongoDB.

Принципы организации хранения и обработки данных в MongoDB

Одни из сценариев использования MongoDB — код JavaScript на Web-сервере (бекэнд), исполняемый под управлением Node.js. MongoDB в этом случае отображается как подключаемая библиотека со своим API.

Имеются драйверы (интерфейсы) для использования MongoDB с другим языкам программирования, такими как C++, Python и Java. Также имеются графические оболочки для управления базой.

Сервер MongoDB может хранить несколько баз данных (так же как и серверы других баз данных).

Единица хранения базы — документ. Документы объединены в коллекции. Для пользователя документ представлен в виде объекта JSON, а коллекция — это список (массив) объектов.

Каждый документ имеет одно обязательное поле `_id`. Это идентификатор документа, уникальный в пределах коллекции, к которой документ принадлежит.

Поле `_id` выполняет функцию первичного ключа реляционных баз.

Пример документа:

```
{  
  "_id": "ObjectId('68aaf97083426b4f1289b03d')",  
  "title": "Big Data",  
  "type": "Лекции"  
}
```

Здесь `ObjectId()` — это тип данных для идентификатора.

Так как в одной коллекции могут быть разные документы, то кажется, что возможность создания нескольких коллекций избыточна. Однако на самом деле коллекции очень важны.

Коллекция предназначена для хранения полуструктурированных данных — документов, которые имеют почти одинаковую структуру (совпадающие поля), но, возможно, с небольшими отличиям.

Группировка одинаковых по смыслу документов в одной коллекции обеспечивает локальность данных. Получение нескольких постов в блоге из коллекции, содержащей только посты, потребует меньше

операций поиска, чем получение тех же постов из коллекции, где содержатся как посты, так и данные об авторах.

Рассмотрим несколько примеров работы с базой данных под управлением MongoDB на примере демонстрационной базы данных `sample_supplies`, предоставляемой MongoDB.

www.mongodb.com/docs/atlas/sample-data/sample-supplies/

В этой базе имеется одна коллекция, в которой содержатся сведения о поставщиках канцелярский товаров.

Ниже приведён пример документа из базы. Видно, что значение поля `customer` — это тоже документ, а поле `items` хранит массив документов.

Поля, начинающиеся со знака доллара «\$», обозначают зарезервированные ключевые слова. При импорте этого документа в базу они будут трансформированы в вызовы системных функций преобразования типа данных. Например `$oid` будет преобразован в вызов `ObjectId()`.

```
{
  "_id": {"$oid": "5bd761dcae323e45a93ccfe8"},
  "saleDate": {"$date": {"$numberLong": "1427144809506"}},
  "items": [
    {
      "name": "notepad",
      "tags": ["office", "writing", "school"],
      "price": {"$numberDecimal": "35.29"},
      "quantity": {"$numberInt": "2"}
    }, {
      "name": "pens",
      "tags": ["writing", "office", "school", "stationary"],
      "price": {"$numberDecimal": "56.12"},
      "quantity": {"$numberInt": "5"}
    }
  ],
  "storeLocation": "Denver",
  "customer": {
    "gender": "M", "age": {"$numberInt": "42"}, "email": "cauho@witwuta.sv",
    "satisfaction": {"$numberInt": "4"}
  },
  "couponUsed": true,
  "purchaseMethod": "Online"
}
```

Данные в базу MongoDB могут быть импортированы из разных форматов, в том числе из списка объектов JSON, пример которого показан выше.

Один из способов импорта — утилита `mongoimport`. Если сервер базы запущен на локальном компьютере с настройками по умолчанию, в частности без требования аутентификации, то импорт выполняется следующим образом:

```
mongoimport --db sample_supplies --collection sales --type json --file sales.json
```

Если сервер находится в сети, то потребуется указать адрес и порт базы сервера при помощи ключей `--host` и `--port`. Для того, чтобы указать пароль и имя пользователя, можно использовать ключи `--username` и `--password`.

Базовый инструмент для работы с базами MongoDB — оболочка командной строки `mongosh`. При необходимости можно указать в качестве параметров этой команды адрес сервера и учётные данные пользователя.

После того как мы подключились к серверу, можно увидеть список имеющихся баз, выполнив следующую команду:

```
show dbs
```

```
root@8128b7036886:/# mongosh
Current Mongosh Log ID: 68ac58d2e9b2e0c63089b03c
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.6
Using MongoDB:      8.0.13
Using Mongosh:       2.5.6

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
  The server generated these startup warnings when booting
  2025-08-25T07:26:13.703+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage
  engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2025-08-25T07:26:14.231+00:00: Access control is not enabled for the database. Read and write access to dat
  a and configuration is unrestricted
  2025-08-25T07:26:14.231+00:00: Soft rlimits for open file descriptors too low
  2025-08-25T07:26:14.231+00:00: For customers running the current memory allocator, we suggest changing the
  contents of the following sysfsFile
  2025-08-25T07:26:14.231+00:00: We suggest setting the contents of sysfsFile to 0.
  2025-08-25T07:26:14.231+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance p
  roblems.
  -----

test> show dbs
admin                40.00 KiB
config               116.00 KiB
local                72.00 KiB
sample_airbnb        52.13 MiB
sample_analytics     9.43 MiB
sample_geospatial   788.00 KiB
sample_supplies      968.00 KiB
sample_weatherdata   2.55 MiB
учебная             72.00 KiB
test> |
```


Для начала работы с базой, нужно выполнить команду `use` плюс имя базы, например:

```
use sample_supplies
```

При этом подсказка в командной строке (промпт) меняется на имя текущей базы.

Далее можно вывести на экран список имеющихся в текущей базе коллекций:

```
show collections
```

A terminal window with a dark background. The first three lines show a list of collections: 'sample_supplies' with size '968.00 KiB', 'sample_weatherdata' with size '2.55 MiB', and 'учебная' with size '72.00 KiB'. The next line is a prompt 'test>' followed by the command 'use sample_supplies'. The following line shows the prompt changed to 'switched to db sample_supplies'. Then, the command 'sample_supplies> show collections' is entered. The final line shows the prompt 'sample_supplies>' with a cursor, and the word 'sales' is visible on the line above it, indicating it's the first collection in the list.

```
sample_supplies 968.00 KiB
sample_weatherdata 2.55 MiB
учебная 72.00 KiB
test> use sample_supplies
switched to db sample_supplies
sample_supplies> show collections
sales
sample_supplies> █
```

Дальнейшая работа с базой производится в стиле объектно-ориентированного программирования. При выборе базы она сопоставляется переменной `db`. Каждая имеющаяся в базе коллекция — это объект, являющийся полем (атрибутом) объекта `db`. Инструменты для работы с базой — это методы объекта-коллекции.

Вопросы для конспекта

11. В каких вычислительных средах можно использовать базу MongoDB?
12. Для чего предназначено поле `_id` у документов MongoDB?
13. Для чего в MongoDB нужны коллекции, если в одной коллекции не запрещено помещать совершенно разные документы?
14. Как называются утилита для работы с базами MongoDB в консольном режиме?
15. Как в консоли MongoDB получить список имеющихся баз данных, выбрать базу для работы и посмотреть имеющиеся в ней коллекции?

Выборка, фильтрация, агрегация

Вывести на экран коллекцию документов можно при помощи метода `.find()` с пустым списком параметров. Например, команда ниже покажет все документы из коллекции `sales`:

```
db.sales.find()
```

Если в коллекции много документов, то вывод будет слишком объёмным. Чтобы «заглянуть» в коллекцию можно использовать метод, возвращающий один, первый попавшийся документ:

```
db.sales.findOne()
```

```
sample_supplies> db.sales.findOne()
{
  _id: ObjectId('5bd761dcae323ae45e93ccfd'),
  saleDate: ISODate('2015-09-02T16:11:59.565Z'),
  items: [
    {
      name: 'binder',
      tags: [ 'school', 'general', 'organization' ],
      price: Decimal128('13.44'),
      quantity: 8
    },
    {
      name: 'binder',
      tags: [ 'school', 'general', 'organization' ],
      price: Decimal128('16.66'),
      quantity: 10
    }
  ],
  storeLocation: 'London',
  customer: { gender: 'M', age: 44, email: 'owtar@pu.cd', satisfaction: 2 },
  couponUsed: false,
  purchaseMethod: 'In store'
}
sample_supplies> █
```

Ещё один вариант получить ограниченный список документов — задать ограничение при помощи метода `.limit()`:

```
db.sales.find().limit(3)
```

Функция `.skip()` позволят пропустить несколько первых документов.

```
db.sales.find().skip(5).limit(3)
```

Выборка по заданным критериям: в качестве аргумента `find()` или `findOne()` нужно задать JSON-объект, содержащий имена полей и требуемые значения:

```
db.sales.find({"storeLocation": "Denver", "purchaseMethod": "Online"})
db.sales.findOne({"storeLocation": "Denver", "purchaseMethod": "Online"})
```

```
customer: { gender: 'M', age: 32, email: 'kamavbe@po.jkl.dj', satisfaction: 5 },
couponUsed: false,
purchaseMethod: 'Online'
},
{
  _id: ObjectId('5bd761dcae323e45a93cd8b8'),
  saleDate: ISODate('2014-07-28T15:28:59.274Z'),
  items: [
    {
      name: 'envelopes',
      tags: [ 'stationary', 'office', 'general' ],
      price: Decimal128('5.4'),
      quantity: 8
    },
    {
      name: 'notepad',
      tags: [ 'office', 'writing', 'school' ],
      price: Decimal128('7.31'),
      quantity: 3
    }
  ],
  storeLocation: 'Denver',
  customer: {
    gender: 'M',
    age: 32,
    email: 'kamavbe@po.jkl.dj',
    satisfaction: 5
  },
  couponUsed: false,
  purchaseMethod: 'Online'
}
```

Первый метод вернёт все документы коллекции, удовлетворяющие критериям, второй — первый подходящий документ.

Для указания при поиске вложенных документов, нужно указывать просматриваемые подчинённые поля обычном образом, через точку. В примере ниже мы найдём поставщиков из Лондона, у которых есть ноутбуки и их клиенты остались довольны заказом:

```
db.sales.find({"items.name": "laptop", "customer.satisfaction": 1,  
              "storeLocation": "London"})
```

```
    price: Decimal128( '785.36' ),  
    quantity: 1  
  },  
  {  
    name: 'laptop',  
    tags: [ 'electronics', 'school', 'office' ],  
    price: Decimal128( '785.36' ),  
    quantity: 1  
  },  
  {  
    name: 'pens',  
    tags: [ 'writing', 'office', 'school', 'stationary' ],  
    price: Decimal128( '65.12' ),  
    quantity: 1  
  }  
],  
storeLocation: 'London',  
customer: {  
  gender: 'F',  
  age: 41,  
  email: 'behe@benmuzku.qa',  
  satisfaction: 1  
},  
couponUsed: true,  
purchaseMethod: 'In store'  
}
```

Условия меньше или больше задаются при помощи операторов, которые записываются со знаком доллара. Запрос ниже вернёт документы, в которых клиентами были женщины моложе 30 лет:

```
db.sales.find({"customer.gender": "F", "customer.age": {$lt: 30}})
```

```
{
  name: 'binder',
  tags: [ 'school', 'general', 'organization' ],
  price: Decimal128( '29.17' ),
  quantity: 3
},
{
  name: 'printer paper',
  tags: [ 'office', 'stationary' ],
  price: Decimal128( '41.86' ),
  quantity: 2
}
],
storeLocation: 'London',
customer: { gender: 'F', age: 19, email: 'gl@binvozfo.at', satisfaction: 4 },
couponUsed: true,
purchaseMethod: 'In store'
}
```


Можно показать только отдельные свойства документов. За ограничение показа свойств отвечает второй параметров метода `.find()`. В запросе ниже выведены только наименования канцтоваров:

```
db.sales.find({"customer.gender": "F", "customer.age": {$lt: 30}},  
              {"items.name": true})
```

```
{  
  _id: ObjectId('5bd761dcae323e45a93cd0f3'),  
  items: [ { name: 'pens' }, { name: 'notepad' }, { name: 'envelopes' } ]  
},  
{  
  _id: ObjectId('5bd761dcae323e45a93cd0f0'),  
  items: [  
    { name: 'pens' },  
    { name: 'notepad' },  
    { name: 'envelopes' },  
    { name: 'binder' },  
    { name: 'laptop' },  
    { name: 'pens' },  
    { name: 'binder' },  
    { name: 'printer paper' }  
  ]  
}  
]
```

В пример выше видно, что кроме запрошенного свойства `items.name` показан также ключ `_id`. Ключ выводится по умолчанию и чтобы его скрыть, это нужно потребовать явным образом:

```
db.sales.find({"customer.gender": "F", "customer.age": {$lt: 30}},  
              {"items.name": true, "_id": false})
```

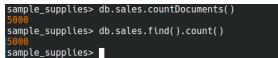
```
{  
  items: [  
    { name: 'binder' },  
    { name: 'notepad' },  
    { name: 'printer paper' },  
    { name: 'envelopes' },  
    { name: 'notepad' }  
  ],  
  {  
    items: [ { name: 'pens' }, { name: 'notepad' }, { name: 'envelopes' } ]  
  },  
  {  
    items: [  
      { name: 'pens' },  
      { name: 'notepad' },  
      { name: 'envelopes' },  
      { name: 'binder' },  
      { name: 'laptop' },  
      { name: 'pens' },  
      { name: 'binder' },  
      { name: 'printer paper' }  
    ]  
  }  
}
```

Можно выполнить подсчёт количества документов, возвращённых выполненной командой. Для этого используется метод `.count()`:

```
db.sales.find().count()
```

Команда выше возвращает полное число документов в коллекции. Для этой операции есть также специальная функция `.countDocuments()`:

```
db.sales.countDocuments()
```



```
sample_supplies> db.sales.countDocuments()  
5000  
sample_supplies> db.sales.find().count()  
5000  
sample_supplies> |
```

Ещё пример: количество женщин клиентов моложе 30 лет:

```
db.sales.find({"customer.gender": "F", "customer.age": {$lt: 30}}).count()
```

Имеется возможность отбора уникальных документов при помощи метода `.distinct()`

Также можно выполнять вычисления минимальных и максимальных значений: `.min()` и `.max()`.

Вопросы для конспекта

16. Как в базе данных MongoDB получить список всех документов в коллекции?
17. Как в базе данных MongoDB отобрать документы из коллекции по заданным критериями?
18. Как в базе данных MongoDB из числа найденных документов вернуть их ограниченное количество?
19. Как в базе данных MongoDB найти только один документ по заданным критериями?
20. Как в базе данных MongoDB при выводе документов получить только требуемые поля?
21. Как в базе данных MongoDB получить количество найденных документов?

Вопросы для конспекта

- 1 Что является аналогом таблиц и записей в MongoDB?
- 2 Чем объекты хранения MongoDB отличаются от объектов хранения реляционных баз данных?
- 3 Что в MongoDB используется в качестве языка запросов?
- 4 Какая модель данных используется в MongoDB? Чем она отличается от модели данных в реляционных базах?
- 5 Как в MongoDB поддерживается масштабирование?
- 6 Имеется ли возможность поддерживать связи между объектами базы, аналогично тому, как это сделано в реляционных базах?
- 7 В чём сходства и отличия MongoDB и баз ключ-значение?
- 8 Сравните MongoDB со столбцовыми базами.
- 9 Сравните MongoDB с графовыми базами.
- 10 Сформулируйте сильные и слабые стороны MongoDB.
- 11 В каких вычислительных средах можно использовать базу MongoDB?
- 12 Для чего предназначено поле `_id` у документов MongoDB?
- 13 Для чего в MongoDB нужны коллекции, если в одной коллекции не запрещено помещать совершенно разные документы?
- 14 Как называются утилита для работы с базами MongoDB в консольном режиме?
- 15 Как в консоли MongoDB получить список имеющихся баз данных, выбрать базу для работы и посмотреть имеющиеся в ней коллекции?
- 16 Как в базе данных MongoDB получить список всех документов в коллекции?
- 17 Как в базе данных MongoDB отобрать документы из коллекции по заданным критериями?
- 18 Как в базе данных MongoDB из числа найденных документов вернуть их ограниченное количество?

- 19 Как в базе данных MongoDB найти только один документ по заданным критериями?
- 20 Как в базе данных MongoDB при выводе документов получить только требуемые поля?
- 21 Как в базе данных MongoDB получить количество найденных документов?

Лекция 7. Экосистема хранения и обработки больших данных Hadoop

Общие сведения о Hadoop

Hadoop — это технология, сформировавшая современную индустрию больших данных.

Исходные идеи, которые привели к созданию появлению индустрии больших данных и Hadoop появились в начале нулевых. Google искал возможность проиндексировать весь Интернет. Это связано с решением следующих проблем.

- Как разместить сотни терабайт данных на тысячах дисков, установленных в более, чем тысяче машин, без простоев на ремонт и обслуживание и потерь информации?
- Как распараллелить вычисление эффективным и отказоустойчивым способом для обработки всех этих данных на всех машинах?

В 2003-2006 годах сотрудники Google опубликовали три исследовательские работы, посвящённые решению этих проблем. В этих статьях были описаны GFS (Google File System, распределённая файловая система для хранения больших данных), MapReduce (модель построения программ для обработки больших распределённых данных) и BigTable (NoSQL СУБД для работы с большим данными).

Программное обеспечение на основе этих идей остались в Google закрытыми, но на их основе сотрудники Yahoo! (поисковая система, была очень популярна в девяностые и в начале нулевых) начали разрабатывать проект с открытым исходным кодом, который получил название Apache Hadoop.

Название «Hadoop» — это имя жёлтого слона сына Дуга Каттинга, первого разработчика Hadoop. Apache — это название фонда поддержки свободного ПО «Apache Software Foundation» под эгидой которого разрабатывается Hadoop.



- **Hadoop** — это открытый фреймворк (набор программных библиотек и утилит), предназначенный для распределённой обработки и хранения огромных объёмов данных (петабайты и более) на кластерах стандартных серверов.

Ключевая идея Hadoop — возможность горизонтального масштабирования на недорогом оборудовании: вместо того чтобы использовать мощное и дорогое серверное оборудование, объединяется множество относительно простых и дешёвых компьютеров в кластер, и Hadoop автоматически распределяет между ними данные и вычислительные задачи.

Ядро Hadoop состоит из четырёх основных модулей:

- **Hadoop Distributed File System (HDFS)** — распределённая, отказоустойчивая файловая система, предназначенная для работы на стандартном оборудовании. HDFS предназначена для хранения очень больших файлов (гига-, тера-, петабайты) с высокой пропускной способностью.
- **Hadoop MapReduce** — это вычислительная модель и её реализация в Hadoop для параллельной обработки больших данных. Предназначена для выполнения распределённых вычислений над данными, хранящимися в HDFS. По сути предназначен для решения задач, аналогичных тем, которые решает SQL.
- **Hadoop YARN (Yet Another Resource Negotiator)** — менеджер ресурсов. Предназначен для планирования заданий и управления кластером.
- **Hadoop Common** — это набор инструментов (библиотек и утилит), которые используются тремя другими модулями Hadoop.

Сильные стороны Nadoop.

- Кластер может быть построен на основе стандартного оборудования. Нет необходимости приобретать дорогие высокопроизводительные серверы. В принципе, будут работать даже обычные, несерверные вычислительные устройства, хотя это не рекомендуется из-за более низкой надёжности.
- Кластер легко масштабируется «горизонтально» простым добавлением новых узлов. Не требуется дорогая модификация существующего оборудования и сложное переконфигурирование программного обеспечения.
- Данные автоматически реплицируются на несколько узлов. При сбое одного узла его работу автоматически перенимают другие, а данные восстанавливаются из реплик. Вычисление продолжается без потери данных.

- Так как данные хранятся в виде файлов, можно хранить и обрабатывать данные произвольного типа: структурированные (таблицы), полуструктурированные (JSON, XML) и неструктурированные (логи, текст, видео). Не требуют предварительной подготовки данных как это требуется в СУБД.
- Вычисления автоматически распараллеливаются по множеству узлов, что позволяет обрабатывать петабайты данных за приемлемое время.

Слабые стороны Hadoop.

- MapReduce наилучшим образом подходит для несложных по устройству и массивных по объёмы обрабатываемых данных операций. Задачи на небольших объёмах данных а также на большом количестве данных малого объёма выполняются неэффективно из-за высоких накладных расходов.
- Задачи MapReduce не самые быстрые, они не предназначены для обработки в реальном времени или интерактивных задач с малой задержкой. Ответ на запрос может занимать минуты или часы. В значительной степени это связано с тем, что в модели MapReduce минимизировано использование оперативной памяти и все промежуточные данные хранятся во временных файлах.
- MapReduce-задачи пишутся на Java, но устроены достаточно сложно. Java программистов много, но тех кто пишет MapReduce найти гораздо сложнее. Гораздо проще найти специалиста по SQL.

- В ранних версиях Hadoop не уделялось внимание безопасности. Поэтому эти функции были добавлены, но всё равно правильная настройка безопасности кластера остаётся сложной задачей.
- HDFS оптимизирована для записи файла один раз и чтения многократно (Write-Once Read-Many). Частые обновления или произвольная запись в маленькие файлы неэффективны.

Кроме основных, существует достаточно большое количество дополнительных модулей, которые интегрируются в Hadoop.

Некоторые важные дополнительные модули в экосистеме Hadoop

- **Apache Zookeeper** — сервис для поддержания конфигурации, синхронизации и обеспечения координации между распределёнными процессами в кластере. Он координирует действия множества независимых компонентов и позволяет избегать конфликтов. Используется такими компонентами, как HBase и Kafka.

- **Apache HBase** — распределенная столбцовая база данных, построенная поверх HDFS. Может использоваться для работы с данными в реальном времени.
- **Apache Spark** — в настоящее время один из самых популярных фреймворков для распределенной обработки данных. Значительно превосходит классический MapReduce по скорости для многих задач благодаря работе с данными в оперативной памяти, без выгрузки в файлы, как это делает MapReduce. Поддерживает пакетную обработку, стриминг, машинное обучение и графовую обработку. В настоящее время **MapReduce часто заменяется** более быстрыми и удобными фреймворками, например **Apache Spark**.

- **Apache Tez** — открытый фреймворк для выполнения распределённых вычислений, построенный поверх Apache Hadoop YARN. Его основная цель — значительно ускорить обработку больших данных, предоставив более гибкую и эффективную альтернативу традиционной модели выполнения MapReduce. Tez позволяет представлять сложные задачи обработки данных в виде графов, где вершины обозначают этапы преобразования данных, а рёбра — пути перемещения данных между этими этапами.
- **Apache Flink** — современный фреймворк для обработки потоковых данных с высокой пропускной способностью и низкой задержкой. Конкурирует со Spark Streaming в экосистеме.
- **Apache Hive** — выполнение SQL-подобных запросов (язык HiveQL) к данным, хранящимся в HDFS или других распределённых хранилищах. Преобразует эти запросы в задачи MapReduce, Tez или Spark.

- **Apache Pig** — фреймворк и платформа для анализа больших данных, работающая поверх Hadoop. Он предоставляет язык сценариев под названием Pig Latin, который упрощает создание сложных программ обработки данных, автоматически транслируя их в последовательность MapReduce-задач или других вычислительных моделей, например, с использованием Tez или Spark для выполнения на Hadoop-кластере. В отличие от Hive (HiveQL) язык Pig Latin ближе к процедурным языкам и поэтому обладает большей гибкостью.
- **Presto** — распределённый движок SQL для выполнения быстрых интерактивных запросов к различным источникам данных. Он позволяет выполнять SQL-запросы к различным источникам данных, как реляционным (PostgreSQL, MySQL), так и нереляционным (HDFS, MongoDB), без необходимости их перемещения в отдельную систему, без преобразования в MapReduce. Оптимизирован для аналитических запросов с низкой задержкой.

- **Apache Sqoop** — инструмент для передачи объемных данных между Hadoop и структурированными реляционными базами данных.
- **Apache Flume** — сервис для сбора, агрегации и перемещения больших объемов потоковых лог-данных (например, логи серверов) в HDFS.
- **Apache Kafka** — используется для построения конвейеров потоковых данных и приложений потоковой передачи данных в реальном времени. Конвейер данных обрабатывает и перемещает данные из одной системы в другую, а потоковое приложение использует их потоки. Хотя формально это независимый проект, он стал стандартом для построения высокомасштабируемых конвейеров потоковых данных и часто интегрируется с компонентами экосистемы Hadoop (такими как Spark Streaming или Flink) для реального времени обработки.

- **Apache Oozie** — система для планирования и координации сложных конвейеров данных, состоящих из последовательных задач Hadoop (например, запуск цепочки Hive → Pig → Sqoop).
- **Apache Airflow** — популярный инструмент для оркестрации конвейеров данных, который не является частью проекта Hadoop, но часто используется вместе с его экосистемой для управления, мониторинга и планирования задач.

Таким образом, Hadoop представляет собой развитую и зрелую основу для построения современных платформ для работы с данными, поверх которой строятся высокоуровневые инструменты для аналитики, машинного обучения и потоковой обработки.

Вопросы для конспекта

1. С какими проблемами столкнулись Интернет-компании, решение которых привело к появлению индустрии больших данных?
2. Перечислите и опишите четыре основных элемента ядра Hadoop.
3. Перечислите и опишите сильные стороны Hadoop.
4. Перечислите и опишите слабые стороны Hadoop.
5. Сделайте обзор дополнительных модулей и программ, интегрируемых в Hadoop. Какие функции выполняют, какие задачи решают? Приведите примеры.

Взаимодействие Hadoop с другим инструментами для больших данных

Hadoop был первым программным решением промышленного уровня для работы с большими данными. Затем появились другие подходы.

Например, хранилища ClickHouse и MongoDB представляют собой самостоятельные решения по организации работы с большими данными.

Hadoop, ClickHouse и MongoDB решают похожие задачи но имеют разные сильные и слабые стороны. Hadoop, ClickHouse и MongoDB часто сосуществуют в одной организации, обмениваясь данными для достижения общих целей.

Они не встраиваются друг в друга, а взаимодействуют через коннекторы, общие хранилища (HDFS,) и очереди сообщений (Kafka), образуя гибкий и мощный конвейер для работы с данными любого масштаба и назначения.

ClickHouse может напрямую читать данные из HDFS с помощью специального табличного движка HDFS. Это позволяет выполнять

быстрые SQL-запросы к данным, хранящимся в Hadoop, без их предварительного перемещения.

Например в HDFS могут накапливаться сырые логи сервера которые затем передаются для обработки в ClickHouse.

В MongoDB для взаимодействия с экосистемой Hadoop существует MongoDB Connector for Hadoop. Этот коннектор позволяет использовать MongoDB в качестве источника данных для задач MapReduce или Spark. Также можно получать данные из Hadoop обратно в MongoDB.

Например, операционные данные из MongoDB (например, пользовательские транзакции) можно агрегировать в Hadoop для построения моделей машинного обучения при помощи SparkML, а результаты вернуть в MongoDB для использования в реальном времени.

Пример конвейера (пайплайна) данных с применением всех трёх технологий

- Операционные данные (данные, получаемые напрямую из повседневной деятельности) записываются в MongoDB.
- CDC-инструмент (Change Data Capture, инструмент мониторинга) обнаруживает новые данные в MongoDB и отправляет их в Kafka.
- Hadoop (через Spark Streaming или Flume) потребляет данные из Kafka для пакетной обработки, очистки и обогащения.
- Обработанные и агрегированные данные загружаются из Hadoop в ClickHouse.
- Аналитики и бизнес-пользователи выполняют быстрые интерактивные запросы к данным в ClickHouse через BI-инструменты (Business Intelligence, бизнес-аналитика, т.е. инструменты анализа и визуализации данных, например, Tableau или Superset).
- Обученные в Hadoop ML-модели или ключевые агрегаты могут быть записаны обратно в MongoDB, чтобы обогащать операционные приложения (например, предлагать персональные рекомендации в реальном времени).

Таким образом, экосистема Hadoop, ClickHouse и MongoDB — это не взаимозаменяемые, а комплементарные технологии. При их совместном использовании они образуют мощный симбиоз в архитектуре больших данных:

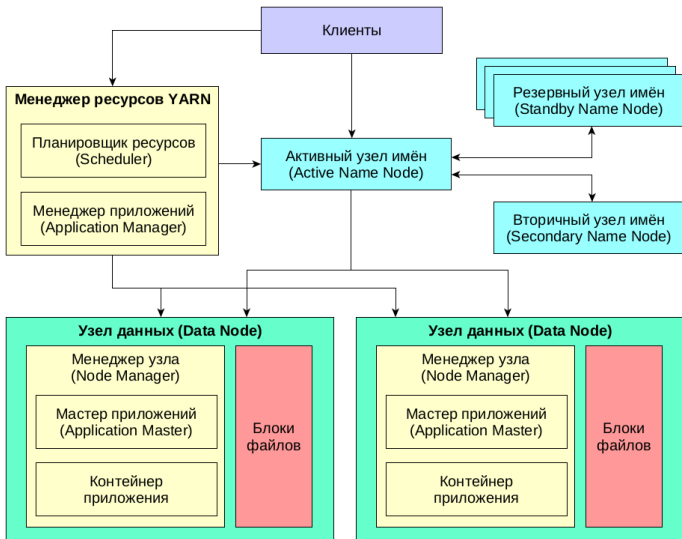
- Hadoop выступает как платформа для хранения и глубокой пакетной обработки больших объемов сырых данных.
- ClickHouse обеспечивает скорость выполнения аналитических запросов для интерактивного анализа.
- MongoDB поддерживает операционную работу приложений с гибкой схемой данных.

Вопросы для конспекта

6. Опишите, как Hadoop может использоваться совместно с ClickHouse и MongoDB.

Архитектура кластера Hadoop

Базовая конфигурация кластера Hadoop включает в себя узлы имён, узлы данных и узел менеджера ресурсов YARN.



Узлы имён и данных отвечают за хранение данных в HDFS, а YARN обеспечивает работу задач обработки данных (приложений).

Для операций с файлами клиенты кластера обращаются к активному узлу имён, а для запуска задач (приложений) — к YARN.

- **Активный узел имён** хранит метаинформацию о данных в HDFS — имена файлов, атрибуты файлов, структуру каталогов, карту расположения отдельных блоков на разных узлах данных. Также он отвечает за выполнение действий над файлами в HDFS.

В кластере всегда только один активный узел имён.

- **Резервные узлы имён** хранят копии активного узла имён на случай выхода его из строя. Кластер может функционировать без них, но в таком случае сбой активного узла приводит полной недоступности HDFS. Поэтому на рабочих кластерах под резерв выделяют несколько узлов.

- **Вторичный узел имён** предназначен для снижения нагрузки на активный узел имён. Также благодаря ему сокращается время простоя кластера после сбоя активного узла имён.

Не является критически необходимым компонентом для работы кластера, но на рабочих кластерах его наличие настоятельно рекомендуется.

- **Узел данных** — узлы, в долговременной памяти которых хранятся блоки файлов, записанных в HDFS. Также узлы данных выполняют задачи обработки данных, которые получают от YARN.

- **Менеджер ресурсов YARN** управляет выполнением задач на кластере в целом. Ему подчинены менеджеры узлов (Node Manager), управляющие своими узлами.

Каждому пользовательскому приложению, запущенному на кластере, выделяется собственный менеджер (Application Manager).

Этот менеджер запускает контейнеры приложения на тех узлах данных, на которых лежат фрагменты необходимых приложению данных.

Далее под управлением локальных мастеров приложений (Application Master) контейнеры выполняют свои задачи на нужных узлах данных.

Так реализуется принцип локальности данных — приложение доставляется к месту хранения данных и там они обрабатываются.

Менеджеры узлов (Node Manager) отвечают за организацию вычислений и поэтому должны быть развёрнуты на каждом узле данных — так обеспечивается выполнение принципа локальности данных.

Но для упрощения, например в учебных целях, в кластере может быть выделен единственный узел менеджера узлов.

Такой кластер тоже будет работать, но эффективность вычислений у него будет ниже — узлам данных придётся обмениваться информацией с менеджером узла по сети.

Вопросы для конспекта

7. Перечислите типы узлов кластера Hadoop и их предназначение.
8. Для чего в кластере Hadoop предназначены активный, вторичный и резервные узлы имён?
9. Для чего в кластере Hadoop предназначены узлы данных? Какие функции они выполняют?
10. Где в кластере Hadoop хранится информация о расположении файлов? Где хранятся сами данные файлов?
11. Какие функции в кластере Hadoop выполняет YARN?
12. Как в кластере Hadoop реализуется принцип локальности данных?

Hadoop в настоящее время

В настоящее время Hadoop всё ещё достаточно широко применяется в инфраструктуре больших данных.

Однако Hadoop не стоит рассматривать в качестве основного инструмента для новых проектов.

Наибольшую актуальность из всех экосистемы сохраняют HDFS и YARN.

Эти компоненты остаются надёжной основой для on-premise (локальных) решений, когда к безопасности и суверенитету предъявляются жёсткие требования (в госкомпаниях).

HDFS предлагает проверенную отказоустойчивость, а YARN — эффективное управление ресурсами кластера для разнородных рабочих нагрузок.

Однако современные архитектуры все чаще предпочитают разделение вычислительных ресурсов и хранилища.

Объектные хранилища, такие как Amazon S3 или его on-premise-аналог Apache Ozone, становятся стандартом де-факто для данных в силу своей масштабируемости, экономической эффективности и гибкости.

Инструменты экосистемы Hadoop (Spark, Hive) научились эффективно работать напрямую с такими хранилищами через коннекторы (например, S3A).

Базовая для Hadoop модель обработки данных MapReduce в настоящее время не рекомендуется для новых разработок.

- **Низкая производительность:** MapReduce активно работает с долговременной памятью (много операций чтения/записи файлов), что делает его на порядки медленнее современных фреймворков для итеративных задач (машинное обучение, графовые алгоритмы) и интерактивной аналитики.
- **Сложность программирования:** Модель MapReduce довольно низкоуровневая и зачастую приводит к созданию сложного и объемного кода по сравнению с более высокоуровневыми API.

- **Ограниченность сценариев:** Далеко не все задачи обработки данных можно решить в парадигме MapReduce или решение оказывается неуклюжим. Кроме того MapReduce плохо подходит для обработки потоковых данных в реальном времени.

Современные альтернативы MapReduce: Apache Spark (универсальный фреймворк) и Apache Flink (потоковые данные).

Вопросы для конспекта

13. Какие компоненты Hadoop сохраняют наибольшую актуальность в настоящее время? Расскажите почему.

14. Почему MapReduce в настоящее время не стоит задействовать для новых проектов? Какие существуют альтернативы?

Вопросы для конспекта

- 1 С какими проблемами столкнулись Интернет-компании, решение которых привело к появлению индустрии больших данных?
- 2 Перечислите и опишите четыре основных элемента ядра Hadoop.
- 3 Перечислите и опишите сильные стороны Hadoop.
- 4 Перечислите и опишите слабые стороны Hadoop.
- 5 Сделайте обзор дополнительных модулей и программ, интегрируемых в Hadoop. Какие функции выполняют, какие задачи решают? Приведите примеры.
- 6 Опишите, как Hadoop может использоваться совместно с ClickHouse и MongoDB.
- 7 Перечислите типы узлов кластера Hadoop и их предназначение.
- 8 Для чего в кластере Hadoop предназначены активный, вторичный и резервные узлы имён?
- 9 Для чего в кластере Hadoop предназначены узлы данных? Какие функции они выполняют?
- 10 Где в кластере Hadoop хранится информация о расположении файлов? Где хранятся сами данные файлов?
- 11 Какие функции в кластере Hadoop выполняет YARN?
- 12 Как в кластере Hadoop реализуется принцип локальности данных?
- 13 Какие компоненты Hadoop сохраняют наибольшую актуальность в настоящее время? Расскажите почему.
- 14 Почему MapReduce в настоящее время не стоит задействовать для новых проектов? Какие существуют альтернативы?

Лекция 8. Распределённая файловая система, используемая в Hadoop

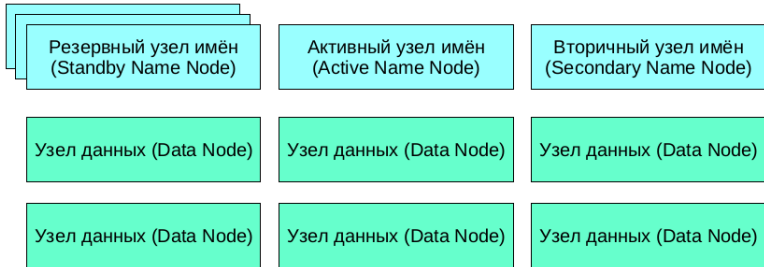
Структура и основные принципы HDFS

- **Распределённая файловая систем** — система организации хранения данных в виде файлов в которой задействовано множество накопителей данных на множестве вычислительных узлов (компьютеров) в сети.

Количество узлов может быть очень большим, их географическое расположение может быть произвольным. Для пользователя работа с распределённой файловой системой происходит прозрачно — пользователь «видит» файлы и папки как на локальном компьютере независимо от их физического расположения на узлах сети.

- **HDFS (Hadoop Distributed File System)** — это распределённая файловая система, используемая для хранения больших данных на кластере Hadoop.

Хранение файлов организовано с использованием узлов имён и узлов данных.



Вспомним, что узлы имён хранят информацию о размещении файлов на узлах данных. Имеется единственный активный узел имён, несколько резервных и единственный вторичный узел имён, который используется для снижения нагрузки на активный узел имён.

Файлы в HDFS разделены на блоки, которые хранятся на узлах данных.

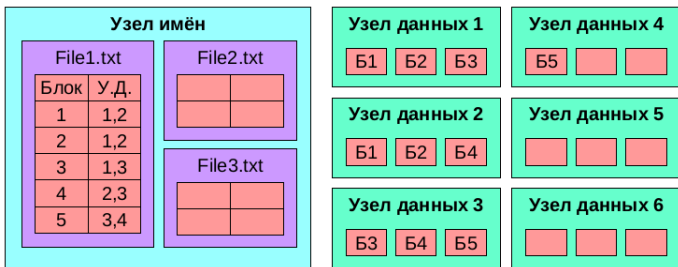
Узлы данных «не знают» что за блоки данных они хранят, к какому файлу в HDFS они относятся и в случае выхода из строя всех узлов имён собрать блоки в единый файл не получится.

При сохранении файла в HDFS можно задать размер блока. Но обычно используются блоки не более 128 Мб. Это значит, что при необходимости последний блок будет меньше.

Предположим, что имеется файл 600 Мб. Такой файл будет разбит на 5 блоков. Первые четыре по 128 Мб и последний размером 88 Мб.

Блок — это файл в локальной файловой системе узла данных.

Хранится несколько копий каждого блока на разных узлах. Количество копий задаётся коэффициентом репликации в настройках HDFS.



- **Репликация** — автоматическое создание нескольких копий каждого блока на разных узлах данных.

По умолчанию коэффициент репликации равен трём — хранятся по три копии каждого блока. Этот коэффициент можно менять в настройках.

Если какая-то из копий теряется из-за сбоя, узел имён автоматически создаёт новую копию на другом узле данных.

Также файловой системе можно сообщить на какой стойке («rack») находится каждый узел данных — rack awareness. Тогда копии блоков будут записываться на узлы, находящиеся в разных стойках.



Это важно потому, что вероятность выхода из строя одновременно нескольких узлов на одной стойке выше чем на разных. Поэтому надёжнее хранить копии блоков на разных стойках.

При этом обмен данными внутри одной стойки происходит быстрее, поэтому выгоднее хранить на одной стойке разные блоки одного файла.

Узлы данных можно добавлять или удалять в процессе работы кластера. Благодаря репликации потеря данных не происходит.

После изменения числа узлов данных происходит автоматическое перераспределение блоков данных между имеющимися узлами.

Процедура чтения файла из HDFS

- ① HDFS-клиент обращается к узлу имён и открывает соединение.
- ② HDFS-клиент запрашивает информацию о файле у узла имён.
- ③ Узел проверяет существует ли файл и какие у него права доступа.
- ④ Если проверка пройдена, клиент получает информацию о расположении блоков на узлах данных.
- ⑤ HDFS-клиент читает блоки файла с указанных узлов данных.

Процедура записи файла в HDFS

- 1 HDFS-клиент обращается к узлу имён и открывает соединение..
- 2 HDFS-клиент запрашивает разрешение о записи файла у узла имён.
- 3 Узел проверяет существует ли такой файл и есть ли у клиента права на запись в этот файл.
- 4 Если проверки прошли успешно, то узел в своих метаданных создаёт новую запись для файла и сообщает клиенту имена узлов данных, выделенных для блоков файла.
- 5 HDFS-клиент делит файл на несколько пакетов согласно установленному размеру блока и собирает их в виде очереди.
- 6 HDFS-клиент отправляет блоки на указанные узлы данных.
- 7 Запись происходит в один узел, а затем узел имён организует репликацию на несколько других.
- 8 По окончании записи клиент получает подтверждение.

Клиенты пишут напрямую в узлы данных, минуя узел имён. Благодаря этому обеспечивается высокая пропускная способность.

Есть возможность либо удалить файл, либо записать файл заново, либо записать что-то в конец файла.

Нет возможности изменить содержимое блока. Поэтому при записи в конец файла создается отдельный новый блок, даже если последний блок не достигает предельного размера.

Также нельзя записать что-то в середину в файла. Можно только создать файл заново.

- **HDFS плохо работает с небольшими файлами.** Узел имён хранит информацию о каждом файле (метаданные) в оперативной памяти.

Если создать много маленьких файлов (размер файла меньше размера блока), то получится, что объём метаданных больше чем сами данные.

Это приведёт к перерасходу оперативной памяти узла имён и к снижению эффективности работы кластера в целом.

Ещё одна причина снижения эффективности — при обработке файлов для каждого файла создаётся некоторая служебная структура. Если файлов много — этих структур будет тоже много и они будут использоваться неэффективно.

Вопросы для конспекта

1. Дайте определение распределённой файловой системы.
2. Что такое блок данных в HDFS?
3. Что такое репликация блоков в HDFS?
4. Для чего в HDFS используется параметр осведомлённости о стойке?
5. Что происходит когда в кластере Hadoop отключаются или подключаются узлы данных?
6. Опишите процедуру чтения файла в HDFS.
7. Опишите процедуру записи файла в HDFS.
8. Почему HDFS плохо работает с небольшими файлами?

Работа с файлами в HDFS

Базовый инструмент для манипуляций с файлами в HDFS — консоль (командная строка).

Так как узлы кластера Hadoop работают под управлением Linux, то работа с HDFS осуществляется при помощи команд, выполняемых в Linux-консоли узла имён кластера.

Команды обращения к файлам Hadoop состоят из нескольких частей.

Существует два варианта команды: `hadoop fs ...` и `hdfs dfs ...`.

Обе команды являются обёртками над одним и тем же Java-кодом. Они вызывают один и тот же исполняемый файл `hadoop`, который в зависимости от переданной подкоманды (`fs` или `dfs`) выполняет соответствующий код.

Первая отвечает за более общий вариант работы и предназначена для работы со всеми файловыми системами, с которыми Hadoop может взаимодействовать. Например, это может быть Amazon S3.

Вторая предназначена именно для работы с файлами в HDFS.

Ниже мы будем рассматривать второй вариант. Однако если везде заменять `hdfs dfs ...` на `hadoop fs ...` то это тоже будет корректно.

Примечание

Amazon S3 (Simple Storage Service) — облачный сервис объектного хранения данных от Amazon, который позволяет пользователям хранить и извлекать любые объемы данных из любого места в интернете. Он предлагает высокую масштабируемость, доступность, безопасность и производительность, а также удобный веб-интерфейс для управления данными. Сервис используется для резервного копирования, архивации и в приложениях, требующих работы с большими объемами данных.

Объектное хранилище означает, что данные хранятся в виде объектов, каждый из которых имеет уникальный идентификатор, метаданные и само содержимое.

Большинство команд работы с файлами имеют вид:

```
hdfs dfs -<command> [аргументы]
```

Команда `-<command>` в подавляющем большинстве случаев повторяет аналогичную команду консоли Linux. Например: `ls`.

Основные отличия работы с консолью Linux от работы с HDFS:

- В Linux можно отредактировать файл прямо в его местоположении. В HDFS это невозможно. Данные неизменяемы после записи. Стандартный вариант работы — скачать файл в локальную файловую систему, изменить и отправить новую версию обратно в HDFS.
- В Linux работа происходит относительно вашего текущего расположения, т. е. текущей папки. Переход между папками осуществляется командой `cd`. В HDFS отсутствует понятие текущей папки. Мы всегда находимся в корневой директории и поэтому всегда должны указывать полный путь к файлам и папкам.

• Навигация и просмотр

Команда	Описание	Пример
-ls	Просмотр содержимого папки	hdfs dfs -ls / hdfs dfs -ls /user/hadoop
-ls -R	Рекурсивный просмотр (показать все вложенные папки)	hdfs dfs -ls -R /data
-cat	Вывод содержимого файла на экран	hdfs dfs -cat /data/logs.txt
-tail	Вывод последней части файла (удобно для логов)	hdfs dfs -tail /data/logs.txt
-head	Вывод первой части файла	hdfs dfs -head /data/logs.txt
-du	Показать размер файлов/папок	hdfs dfs -du -h /data (-h для вывода в MB и GB)
-count	Показать кол-во файлов, папок и общий размер	hdfs dfs -count /user

• Управление файлами и папками

Команда	Описание	Пример
-mkdir	Создать папку	hdfs dfs -mkdir /my_new_data hdfs dfs -mkdir -p /prj/raw/data (-p создаст все родительские)
-put	Загрузить файл/папку из локальной FS в HDFS	hdfs dfs -put file.txt /data/ hdfs dfs -put dir/ /data/
-get	Скачать файл/папку из HDFS в локальную FS	hdfs dfs -get /data/hdfsfile.txt ./
-cp	Копирование внутри HDFS	hdfs dfs -cp /data/file.txt /data/backup/
-mv	Перемещение/переименование внутри HDFS	hdfs dfs -mv /data/oldname.txt /data/newname.txt
-rm	Удалить файл	hdfs dfs -rm /data/file.txt
-rm -r	Удалить папку dir и всё её содержимое	hdfs dfs -rm -r /data/dir/
-rmdir	Удалить пустую папку	hdfs dfs -rmdir /empty_dir

- **Выполнение типовых задач**

Загрузить данные в HDFS из локальной файловой системы узла:

```
hdfs dfs -put local_data.csv /user/your_name/input/
```

Посмотреть, что загрузилось:

```
hdfs dfs -ls -h /user/your_name/input/
```

Проверить размер данных:

```
hdfs dfs -du -h /user/your_name/input/
```

Просмотреть начало файла:

```
hdfs dfs -head /user/your_name/input/local_data.csv
```

Скопировать результат обработки в текущую папку локальной файловой системы узла:

```
hdfs dfs -get /user/your_name/output/result.csv ./
```

Удалить временные данные после работы:

```
hdfs dfs -rm -r /user/your_name/temp/
```

Получить справку по команде можно при помощи `-help`:

```
hdfs dfs -help ls
```

```
hdfs dfs -help put
```

Вопросы для конспекта

9. Что является базовым инструментом для работы с файлами в HDFS?
10. Опишите структуру команды работы с файлами в HDFS. В чём сходство и в чём различие команд работы с файлами в консоли Linux и команд HDFS?
11. Расскажите об основных командах работы с файлами в HDFS, при помощи которых осуществляется навигация и просмотр файлов.
12. Расскажите об основных командах работы с файлами в HDFS, при помощи которых производится управление файлами и папками.

Требования к форматам файлов в HDFS

В HDFS нет ограничений на форматы размещаемых файлов. Но существуют специальные форматы файлов, работа с которыми производится наиболее эффективным образом.

Файлы в HDFS хранятся в виде блоков, распределённых по разным узлам данных.

Обработка также производится по частям, непосредственно на узлах, без сборки фрагментов в целый файл (из-за того, что файлы большие).

Отсюда главное требование к файлам в HDFS — делимость.

- **Разделяемость** — возможность разделить файл на независимые фрагменты, которые могут быть обработаны независимо и параллельно на разных узлах кластера.



- **Если файл разделяем:** система может начать обработку с середины файла, не читая его с самого начала. Каждая задача обрабатывает свой фрагмент (например, блок HDFS размером 128 МБ), независимо от других.
- **Если файл НЕ разделяем:** весь файл должен быть обработан одной задачей. Его нельзя разбить на части, даже если физически он хранится в нескольких блоках HDFS на разных серверах.

Для экономии места очень желательно хранить файлы в сжатом виде (текстовые файлы в сжатом виде занимают примерно в десять раз меньший объём).



Требование разделяемости накладывает ограничение на используемые методы сжатия.

На уровне самой HDFS блоки не сжимаются. Сжатие нужно производить на уровне исходного файла, до разделения его на блоки.

Не все процедуры сжатия допускают разделение файла на фрагменты с возможностью их временной распаковки для обработки. Но существуют архиваторы, которые это поддерживают, например bzip2.

Если произвольный файл сжать одним из таких архиваторов и поместить в HDFS, системы распределённой обработки (например, Hadoop) всё равно не будут иметь информацию о возможности его разделения.



- **Сжатие** — формат файла должен быть таким, чтобы имелась возможность его сжатия без потери свойства разделимости. Это значит что сжатие выполняется не внешним архиватором, а включено в описание формата.

- **Оптимизация чтения**: так как размеры файлов предполагаются большими, формат должен быть таким, чтобы чтение производилось наиболее быстро



Как известно, наиболее эффективно обрабатываются структурированные данные. Но может также потребоваться эту структуру поменять.

- **Гибкая схема:** должна иметься возможность задать схему данных в файле но при этом также должна иметься возможность корректировать схему по мере необходимости без полной перезаписи файла.



- **Эффективность для запросов:** способность формата ускорять выполнение аналитических запросов (достигается, например, столбцовым хранением данных и хранением метаданных, например индексов).

- **Совместимость:** поддержка формата большим числом инструментов.

Принципиальное значение имеет разделяемость, так как без неё невозможна параллельная обработка файлов по частям. Другие свойства желательны и выполняются в разной степени.

Вопросы для конспекта

13. Какие форматы файлов допустимы в HDFS?
14. Какое требование к форматам файлов в HDFS является самым главным? Поясните, почему.
15. Каким образом должно быть реализовано сжатие файлов для их эффективного использования в HDFS?
16. Какие свойства желательны для форматов файлов в HDFS?

Форматы файлов, используемых в HDFS

- **Текстовые форматы.** В HDFS используются файлы, в которых данные хранятся в текстовом виде: CSV, JSON, XML.

Текстовые форматы плохо подходят для хранения основных данных.

Основные недостатки: высокие затраты на парсинг, отсутствие встроенной поддержки разделения файлов при сжатии.

Преимущества текстовых форматов. Не требуются специальные инструменты для просмотра, поддержка всеми инструментами работы с данными, минимальные накладные расходы для доступа к данным.

Сценарии использования. Сырые входные данные перед переносом в основное хранилище. Передача данных потребителям.

Прототипирование и исследовательский анализ.

- **Формат SequenceFile.** Это бинарный файл, состоящий из последовательности пар ключ-значение. Ключи и значения могут быть различных типов.

Файлы SequenceFile не имеют стандартного расширения и идентифицируются по маркеру в начале (Magic Bytes).

SequenceFile поддерживает сжатие данных.

Формат поддерживает разделение, в том числе при сжатии (в файл помещаются маркеры), что позволяет обрабатывать его параллельно.

Формат разработан специально для хранения и обработки данных в экосистеме Hadoop в MapReduce-задачах. Другие инструменты Hadoop также умеют работать с этим типом файлов.

Файлы SequenceFile применяют для решения проблемы маленьких файлов в HDFS: множество файлов собирают в один большой SequenceFile. Ключи — имена файлов, значения — содержимое.

SequenceFile был одним из первых эффективных бинарных форматов в Hadoop и широко использовался в ранних версиях экосистемы. В настоящее время уступает по популярности столбцовым форматам, таким как Parquet и ORC.

- **Формат Parquet.** Это бинарный формат хранения данных в форме таблиц. Требуется наличия строгой схемы данных.

В отличие от строчных форматов (как SequenceFile или CSV), он хранит данные по столбцам.

- Строчное хранение: [Иван, 25, Москва], [Анна, 30, СПб].

- Столбцовое хранение: [Иван, Анна], [25, 30], [Москва, СПб].

Файлы Parquet имеет сложную внутреннюю структуру.

Исходная таблица делится на строковые группы. Каждая строковая группы хранит заданное количество строк каждого столбца. Например, строки 1-10 — строковая группа один, 11-20 — строковая группа два и так далее.

Внутри группы данные хранятся по столбцам.

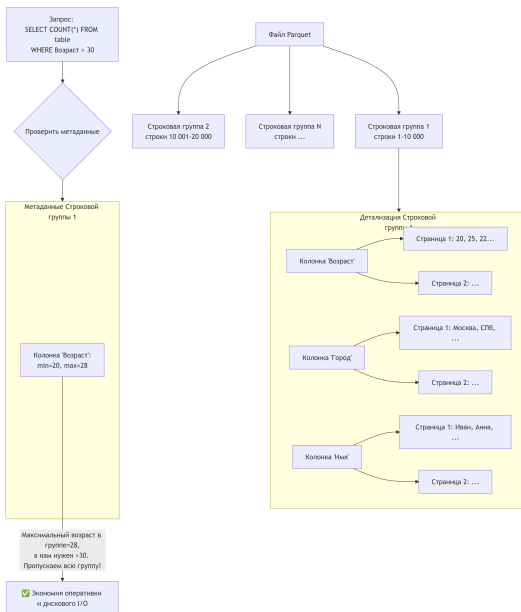
Каждый столбец поделён на блоки, называемые страницами. Обычные размер страницы — несколько десятков или сотен килобайт.

Страница — это единица сжатия и чтения. Иными словами каждый столбец сжат по частям. Сжатие фрагментов каждого столбца отдельно от других повышает эффективность, так как данные в одном столбце часто однородны и хорошо сжимаются.

Файл Parquet разработан для HDFS и поддерживает разделимость по границам строковых групп. Это позволяет обрабатывать его параллельно.

По этой причине при создании Parquet файла рекомендуется задавать размер строковой группы сопоставимым с размером блока HDFS.

В файле Parquet также хранятся метаданные: статистика по каждому столбцу в каждой странице (минимальное и максимальное значение и количество значений) и схема данных (имена полей, типы данных и ограничения на диапазоны).



Parquet поддерживается практически всеми современными инструментами обработки данных и является одним из самых популярных форматов для хранения данных.

В первую очередь он широко используется в аналитике данных — колоночная организация подходит для быстрого выполнения аналитических запросов.

Parquet не эффективен для записи потоковых данных (streaming). Эффективен только при записи данных большими блоками (batches).

Из-за сложной внутренней структуры запись выполняется более медленно, по сравнению с более простыми форматами, например, с текстами.

- **Формат ORC.** Также как и Parquet это бинарный файл с колоночным хранением табличных данных.

Также как и Parquet, ORC-файлы требуют строгой схемы данных.

Внутренняя структура ORC-файлов аналогична структуре файлов Parquet.

Строчным группам Parquet соответствуют полосы (Stripe) в ORC. Это обеспечивает разделимость и возможность параллельной обработки.

Аналогично Parquet в ORC данные внутри полос хранятся по столбцам и столбцы разделены на несколько фрагментов (Streams), которые сжимаются по отдельности.

Отличия ORC от Parquet

- **Индексация:** ORC в отличие от Parquet поддерживает развитую систему индексов для ускорения доступа к данным.
- **ACID-транзакции:** ORC изначально разрабатывался для Hive, поэтому он нативно поддерживает ACID-свойства. Parquet такой поддержки не имеет.
- **Экосистема:** Parquet был разработан для более широкой экосистемы (Spark, Impala) и стал де-факто стандартом для межплатформенного обмена данными. ORC изначально был сильнее заточен под Hive.

Индекс в базе данных — это отдельная структура данных, которая хранит отсортированные копии значений из одного или нескольких столбцов таблицы вместе с указателями на соответствующие строки в основной таблице.

Оба формата и Parquet, и ORC требуют строгой схемы данных. Это значит, что если данные не соответствуют схеме, запись в файл не происходит.

Однако они поддерживают хранение полуструктурированных данных благодаря поддержке не только примитивных типов данных (int, string), но и сложных: составных и вложенных.

- **Структура:** Аналог JSON-объекта. Позволяет создавать вложенные структуры. Пример:

```
{"address": {"city": "Москва", "street": "Арбат"}}
```

- **Массив:** Список значений. Значения в массиве могут быть как примитивными, так и сложными типами. Пример:

```
{"tags": ["пользователь", "москва", "vip"]}
```

- **Avro** — формат для сериализации данных и их эффективной передачи между системами, часто в реальном времени.

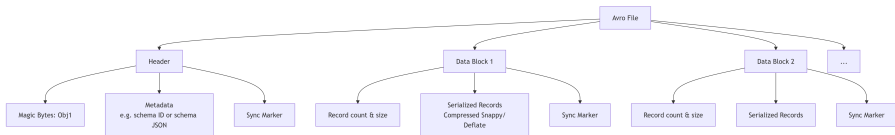
Примечание

Сериализация — это процедура преобразования сложного объекта или структуры данных (например, объекта в оперативной памяти программы) в последовательность байтов (поток байтов) или другой формат, пригодный для передачи по сети или сохранения на диск.

В Avro-файлах данные хранятся по строкам — такая компоновка оптимальна именно для работы со потоковыми данными (Streaming).

Файл Avro состоит из заголовка и блоков данных, разделённых специальными синхронизационными маркерами (Sync Marker).

Блоки содержат данные, они могут быть сжатыми. Допускается разделение по границам блоков.



В заголовке хранятся метаданные, в частности — схема данных в формате JSON.

```
{  
  "type": "record",  
  "name": "User",  
  "fields": [  
    {"name": "name", "type": "string"},  
    {"name": "age", "type": "int"},  
    {"name": "city", "type": "string"}  
  ]  
}
```

При сериализации в бинарный файл не записываются имена полей (name, age, city) и символы-разделители. Вместо этого данные записывают в том порядке, в котором они указаны в схеме, и только их значения.

При десериализации данные извлекаются по известным смещениям в байтах от начала записи.

Этим достигается максимальная компактность данных и скорость их передачи и обработки так как не требуется парсинг.

В Avro схема данных — это отдельная структура. Допустимо при чтении задать другую (совместимую) схему данных.

Основные сценарии использования Avro:

- получение потоковых данных
- сообщения в Kafka
- промежуточное хранение данных между этапами конвейера обработки данных
- долгосрочное хранение данных с часто изменяющейся структурой

Вопросы для конспекта

17. Для каких целей используются текстовые форматы в HDFS? Какие у них преимущества и недостатки?
18. Какую структуру имеют файлы SequenceFile? Для каких целей применяются эти файлы?
19. Какую структуру имеют файлы Parquet? Для каких целей применяются эти файлы? Для каких задач эти файлы менее эффективны?
20. Чем похожи и чем отличаются файлы ORC и Parquet?
21. Требуют ли наличия схемы данных файлы ORC и Parquet? Можно ли использовать эти форматы для хранения полуструктурированных данных?
22. Какую структуру имеют файлы Avro? Для каких целей применяются эти файлы?

Вопросы для конспекта

- 1 Дайте определение распределённой файловой системы.
- 2 Что такое блок данных в HDFS?
- 3 Что такое репликация блоков в HDFS?
- 4 Для чего в HDFS используется параметр осведомлённости о стойке?
- 5 Что происходит когда в кластере Hadoop отключаются или подключаются узлы данных?
- 6 Опишите процедуру чтения файла в HDFS.
- 7 Опишите процедуру записи файла в HDFS.
- 8 Почему HDFS плохо работает с небольшими файлами?
- 9 Что является базовым инструментом для работы с файлами в HDFS?
- 10 Опишите структуру команды работы с файлами в HDFS. В чём сходство и в чём различие команд работы с файлами в консоли Linux и команд HDFS?
- 11 Расскажите об основных командах работы с файлами в HDFS, при помощи которых осуществляется навигация и просмотр файлов.
- 12 Расскажите об основных командах работы с файлами в HDFS, при помощи которых производится управление файлами и папками.
- 13 Какие форматы файлов допустимы в HDFS?
- 14 Какое требование к форматам файлов в HDFS является самым главным? Поясните, почему.
- 15 Каким образом должно быть реализовано сжатие файлов для их эффективного использования в HDFS?
- 16 Какие свойства желательны для форматов файлов в HDFS?
- 17 Для каких целей используются текстовые форматы в HDFS? Какие у них преимущества и недостатки?

- 18 Какую структуру имеют файлы SequenceFile? Для каких целей применяются эти файлы?
- 19 Какую структуру имеют файлы Parquet? Для каких целей применяются эти файлы? Для каких задач эти файлы менее эффективны?
- 20 Чем похожи и чем отличаются файлы ORC и Parquet?
- 21 Требуют ли наличия схемы данных файлы ORC и Parquet? Можно ли использовать эти форматы для хранения полуструктурированных данных?
- 22 Какую структуру имеют файлы Avro? Для каких целей применяются эти файлы?

Лекция 9. Обработка данных с использованием MapReduce и Apache Hive

Общие принципы работы MapReduce

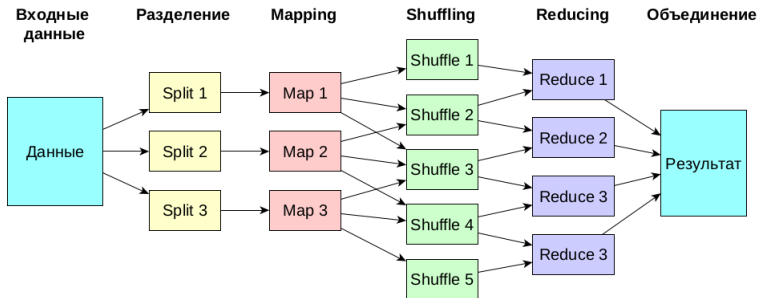
MapReduce — это модель распределённых вычислений, которая обеспечивает параллельную обработку больших данных, автоматически разделяя вычисления между узлами кластера.

Один из главных принципов организации вычислений в MapReduce — локальность данных. Данные хранятся распределённо на разных узлах и логика (программный код) доставляется непосредственно к данным.

Данные в MapReduce проходят следующие этапы обработки.

- **Разделение.** Входные данные разделяются на партии для их параллельной обработки на разных узлах (вспомним, что основное требование к файлам в HDFS — делимость). Партии располагаются на узлах данных и там же обрабатываются.
- **Mapping.** На стадии Mapping данные преобразуются в пары ключ-значение по некоторому закону, задаваемому функцией map.
- **Shuffling.** Фактически это сортировка по ключу: данные с разных узлов группируются по ключам, сортируются по ключу, и все пары с одинаковыми ключами направляются на один узел.

- **Reducing.** Пары с одинаковыми ключами преобразуются в одну пару ключ-значение по закону, задаваемому функцией reduce.
- **Объединение.** Данные с разных узлов объединяются.



Для реализации этого процесса требуется определить:

- функцию **map**, преобразующую данные в пары ключ-значение
- функцию **reduce**, которая агрегирует пары с одинаковым ключом

Другие операции не зависят от алгоритма обработки данных.

Пример команды запуска задачи MapReduce: здесь запускается задача подсчёта частоты слов в тексте, которая входит в число демонстрационных примеров Hadoop.

```
yarn jar \  
$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \  
wordcount \ # имя класса для решаемой задачи  
/user/data/words.txt \ # входной файл, находящийся в HDFS  
/user/output/jar_wordcount # путь к папке с результатами, находящейся в HDFS
```

- Первым указано имя jar-файла с кодом Map и Reduce. В этом файле могут быть упакованы несколько классов для решения разных MapReduce задач.
- Звёздочка в имени заменяет номер используемой версии файла.
- Следом идёт параметр `wordcount`, который указывает, какой именно класс нужно использовать.
- Следующие параметры — путь и имя входного файла и путь к папке, в которую нужно поместить результатами работы. Оба находятся в HDFS.

Вопросы для конспекта

1. Что такое MapReduce? Что является главным принципом для работы с данными на основе MapReduce?
2. Какие этапы обработки проходят данные в MapReduce? Какие из них стандартные, а какие зависят от реализуемого алгоритма?

Иллюстрация подхода MapReduce при помощи программы на Python

Чтобы разобраться с логикой работы MapReduce, мы рассмотрим программу на Python, которая обрабатывает данные согласно принципам MapReduce.

Мы реализуем классическую задачу, которую используют для иллюстрации работы MapReduce — подсчёт числа слов в тексте.

Действия YARN по разделению данных, сортировке и объединению мы смоделируем при помощи функций Python.

Работа выполняется по следующей схеме:

- Разделение текста на фрагменты по числу имеющихся исполнителей.
- Операция Map выделяет в тексте слова и возвращает пары (слово, 1) Здесь «слово» — это ключ, а «1» — значение.
- Операция Shuffling группирует данные по словам.
- Операция Reduce подсчитывает число повторов каждого слова.

```

from multiprocessing import Pool
import re

def mapper(text):
    """Функция Map: отделяет слова и возвращает список элементов 'слово\t1'
    """
    result = []
    words = re.findall(r'\w+', text.lower()) # Упрощенная токенизация текста
    for word in words:
        result.append(f"{word}\t{1}")
    return result

def reducer(key_value_list):
    """Функция Reduce: 'слово\t1', 'слово\t1', 'слово\t1' -> , 'слово\t3'
    """
    result, current_word, current_count = "", None, 0
    for line in key_value_list:
        word, count = line.split('\t', 1)
        count = int(count)
        if word == current_word:
            current_count += count
        else:
            if current_word:
                result += f"{current_word}\t{current_count}, "
            current_word = word
            current_count = count
    if current_word:
        result += f"{current_word}\t{current_count}, "
    return result.strip(", ") # Удалить, если нужно, последний пробел и запятую

```

```

def splitter(text, n_chunks):
    """Разделяем входной массив на n_chunks фрагментов
    """
    # Вычисляем предварительные размеры фрагментов
    # Len=17, n_chunks=5 -> [4,4,3,3,3]
    chunk_sizes = [len(text) // n_chunks] * n_chunks
    for n in range(len(text) % n_chunks):
        chunk_sizes[n] += 1
    # Делим входной массив на части
    chunks, n = [], 0
    for size in chunk_sizes:
        chunks.append(text[n:n+size])
        n += size
    return chunks

def words_repair(chunks):
    """Восстанавливаем разрезанные слова
    """
    for nn in range(1, len(chunks)):
        # Если есть пробел на границе фрагмента, слово не было разрезано
        if chunks[nn-1].endswith(' ') or chunks[nn].startswith(' '):
            continue
        # Восстанавливаем разрезанное слово
        chunks[nn-1], part0 = chunks[nn-1].rsplit(" ", 1)
        part1, chunks[nn] = chunks[nn].split(" ", 1)
        chunks[nn-1] += " " + part0+part1
    return chunks

```

```

if __name__ == "__main__":
    # Входные данные (большой текст)
    text = "Обработка данных важна а MapReduce упрощает обработку данных " * 10
    n_workers = 8 # Количество обработчиков данных
    n_chunks = n_workers # Количество партиций данных

    # Разделяем данные на n_chunks фрагментов
    chunks = words_repair(splitter(text, n_chunks=n_chunks))
    print("DEBUG chunks:", chunks)

    # Этап Mapping (параллельно обрабатываем фрагменты)
    # mapped_results - это список из n_chunks списков
    with Pool(processes=n_workers) as pool:
        mapped_results = pool.map(mapper, chunks)
    print("DEBUG mapped_results:", mapped_results)

    # Этап Shuffling (собираем фрагменты, группируем по ключу и сортируем)
    shuffle_results = sum(mapped_results, []) # [[a,b],[c,d]] -> [a,b,c,d]
    shuffle_results.sort()
    print("DEBUG shuffle_results:", shuffle_results)

    # Этап Reducing (агрегируем результаты)
    final_results = reducer(shuffle_results)

    print("RESULTS:", repr(final_results))

```

Здесь для простоты `shuffle_results` обрабатывается одним исполнителем. В реальности `reducer` также выполняется параллельно.

Вывод программы:

```
DEBUG chunks: ['Обработка данных важна а MapReduce упрощает обработку данных Обработка данных', ' важна а
↳ MapReduce упрощает обработку данных Обработка данных важна а MapReduce', 'упрощает обработку данных
↳ Обработка данных важна а MapReduce упрощает обработку', 'данных Обработка данных важна а MapReduce
↳ упрощает обработку данных Обработка', 'данных важна а MapReduce упрощает обработку данных Обработка
↳ данных', ' важна а MapReduce упрощает обработку данных Обработка данных важна а MapReduce', 'упрощает
↳ обработку данных Обработка данных важна а MapReduce упрощает обработку', 'данных Обработка данных важна
↳ а MapReduce упрощает обработку данных ']
DEBUG mapped_results: [['обработка\t1', 'данных\t1', 'важна\t1', 'а\t1', 'mapreduce\t1', 'упрощает\t1',
↳ 'обработку\t1', 'данных\t1', 'обработка\t1', 'данных\t1'], ['важна\t1', 'а\t1', 'mapreduce\t1',
↳ 'упрощает\t1', 'обработку\t1', 'данных\t1', 'обработка\t1', 'данных\t1', 'важна\t1', 'а\t1',
↳ 'mapreduce\t1'], ['упрощает\t1', 'обработку\t1', 'данных\t1', 'обработка\t1', 'данных\t1', 'важна\t1',
↳ 'а\t1', 'mapreduce\t1', 'упрощает\t1', 'обработку\t1'], ['данных\t1', 'обработка\t1', 'данных\t1',
↳ 'важна\t1', 'а\t1', 'mapreduce\t1', 'упрощает\t1', 'обработку\t1', 'данных\t1', 'обработка\t1'],
↳ ['данных\t1', 'важна\t1', 'а\t1', 'mapreduce\t1', 'упрощает\t1', 'обработку\t1', 'данных\t1',
↳ 'обработка\t1', 'данных\t1'], ['важна\t1', 'а\t1', 'mapreduce\t1', 'упрощает\t1', 'обработку\t1',
↳ 'данных\t1', 'обработка\t1', 'данных\t1', 'важна\t1', 'а\t1', 'mapreduce\t1'], ['упрощает\t1',
↳ 'обработку\t1', 'данных\t1', 'обработка\t1', 'данных\t1', 'важна\t1', 'а\t1', 'mapreduce\t1',
↳ 'упрощает\t1', 'обработку\t1'], ['данных\t1', 'обработка\t1', 'данных\t1', 'важна\t1', 'а\t1',
↳ 'mapreduce\t1', 'упрощает\t1', 'обработку\t1', 'данных\t1']]
DEBUG shuffle_results: ['mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1',
↳ 'mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1', 'mapreduce\t1', 'а\t1', 'а\t1', 'а\t1',
↳ 'а\t1', 'а\t1', 'а\t1', 'а\t1', 'а\t1', 'важна\t1', 'важна\t1', 'важна\t1', 'важна\t1', 'важна\t1',
↳ 'важна\t1', 'важна\t1', 'важна\t1', 'важна\t1', 'важна\t1', 'важна\t1', 'данных\t1', 'данных\t1',
↳ 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1',
↳ 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1', 'данных\t1',
↳ 'данных\t1', 'данных\t1', 'обработка\t1', 'обработка\t1', 'обработка\t1', 'обработка\t1',
↳ 'обработка\t1', 'обработка\t1', 'обработка\t1', 'обработка\t1', 'обработка\t1', 'обработка\t1',
↳ 'обработку\t1', 'обработку\t1', 'обработку\t1', 'обработку\t1', 'обработку\t1', 'обработку\t1',
↳ 'обработку\t1', 'обработку\t1', 'обработку\t1', 'обработку\t1', 'упрощает\t1', 'упрощает\t1',
↳ 'упрощает\t1', 'упрощает\t1', 'упрощает\t1', 'упрощает\t1', 'упрощает\t1', 'упрощает\t1',
↳ 'упрощает\t1', 'упрощает\t1']

RESULTS: 'mapreduce\t10, а\t10, важна\t10, данных\t20, обработка\t10, обработку\t10, упрощает\t10'
```


Преимущества MapReduce

- Хорошо отработанная технология.
- Автоматическое распараллеливание задач.
- Легко масштабируется добавлением новых узлов в кластер.
- Автоматически обрабатывает сбой узлов без потери данных.
- Работает на стандартном оборудовании, не требуя мощных серверов.

Недостатки MapReduce

- Ориентирован на пакетную обработку, а не на работу в реальном времени.
- Далеко не все алгоритмы обработки данных можно эффективно представить в виде MapReduce задачи. Неэффективная реализация — низкая производительность.

- В ходе выполнения задач делается много обращений к долговременной памяти (к диску). Это также сильно снижает производительность.
- За счёт того, что выполняются несколько отдельных этапов, программы для MapReduce достаточно сложно отлаживать.
- Даже простые задачи обработки данных требуют написания большого количества кода: «Я хочу просто выполнить JOIN двух таблиц, а не писать 3 класса на Java!».
- Плохо подходит для интерактивной работы с данными и прототипирования.

Таким образом мы видим, что MapReduce — это жёсткая схема решения любых задач обработки данных посредством последовательного выполнения трёх операций: Map, Shuffle, Reduce.

Пользователь подставляет в эту схему только процедуры Map и Reduce, а остальные преобразования выполняются автоматически.

Исходная идея — максимально абстрагироваться от деталей обработки данных на большом кластере. Но оказалось, что такой высокий уровень абстракции критически влияет на быстродействие.

Кроме того, так как операции Map, Shuffle и Reduce выполняются изолированно, в отдельных программных модулях, результат каждого шага приходится сохранять во временных файлах в долговременной памяти, которая работает медленно. Это также сказывается на общем быстродействии.

Без привязки к этим операциям часто можно найти решение более оптимальное по быстродействию и по расходу памяти.

В рассмотренной задаче подсчёта слов операция Map избыточна. Также можно обойтись без явной сортировки по словам.

Например, можно было бы вычислять для слов хеши, хранить их в виде односвязного списка и увеличивать счётчик разных слов по мере токенизации текста. Очевидно, что существуют и другие варианты эффективного решения этой задачи.

В настоящее время новые проекты рекомендуется создавать на основе других технологий, например Apache Spark.

Вопросы для конспекта

3. В чём состоят преимущества и недостатки MapRedcue?

Реализация MapReduce на языках, отличных от Java при помощи Hadoop Streaming

Стандартный язык для написания кода MapReduce-задач — Java.

Однако в настоящее время Java не используется широко для работы с данными.

Поэтому среди программистов Java мало специалистов по работе с данными, а среди специалистов по данным мало программистов на Java.

По этой причине использование MapReduce для бизнес-задач оказалось затруднено.

Для решения этой проблемы был разработан Hadoop Streaming.

При использовании Hadoop Streaming, функции Map и Reduce могут быть написаны на произвольном языке программирования.

В операционных системах, в частности в Linux, существуют понятия стандартного потока ввода и вывода, `stdin`, `stdout`.

По умолчанию это клавиатура и экран, соответственно, но при запуске программы эти потоки можно перенаправить.

Поток вывода одной программы можно «присоединить» к потоку ввода другой программы. В Linux это называется трубопроводом (pipe). Также поток вывода можно перенаправить в файл.

Hadoop Streaming работает на основе этого механизма.

Функции Map и Reduce пишутся как отдельные программы. Источником входных данных у них должен быть стандартный поток ввода, а выводить результаты работы они должны в стандартный поток вывода.

При запуске через Hadoop Streaming MapReduce-задачи, данные в Map и Reduce программы поставляются и получаются через перенаправление потоков ввода и вывода.

Это значит, что любая программа на любом языке программирования, которая корректно работает с потоками ввода и вывода, может быть встроена в этот трубопровод.

Тем не менее, эффективность трубопровода на основе Hadoop Streaming ниже чем у «родных» (нативных) задач, написанных непосредственно на Java.

Фактически трубопроводы реализуются через временные файлы — когда логически передача данных происходит через трубопровод, на самом деле программа источник пишет данные во временный файлы, а программа приёмник их оттуда читает. Это снижает эффективность работы задачи в целом.

Ниже приведён пример программ на Python выполняющих операции Map и Reduce для задачи подсчёта частот слов и которые могут быть встроены в трубопровод Hadoop Streaming.

При анализе кода Reducer нужно помнить, что на ему вход поступают данные, упорядоченные по ключу.

```

# Mapper
import re
import sys

WORD_RE = re.compile(r"[\w']+")

for line in sys.stdin:
    line = line.strip()
    words = WORD_RE.findall(line)
    for word in words:
        print("{}\t{}".format(word, 1))

# Reducer
import sys

current_word, current_count = None, 0

for line in sys.stdin:
    word, count = line.split('\t', 1)
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word:
            print("{}\t{}".format(current_word, current_count))
            current_word = word
            current_count = count
if current_word:
    print("{}\t{}".format(current_word, current_count))

```

Запуск MapReduce-задачи на основе Hadoop Streaming осуществляется следующей командной строкой:

```
yarn jar \
$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-files mapper.py, reducer.py \ # файлы, которые будут скопированы на узлы кластера
-mapper "python3 mapper.py" \ # командная строка запуска программы mapper
-reducer "python3 reducer.py" \ # командная строка запуска программы reducer
-input /user/data/words.txt \ # файл входных данных, находящийся в HDFS
-output /user/output/py_wordcount # папка в HDFS для выходных данных
```

- **JAR-файл** — Java класс для реализации трубопровода Hadoop Streaming. Звёздочка заменяет номер версии файла.
- **Опция** `-files` указывает, какие файлы должны быть скопированы на все узлы кластера, на которых будет выполняться обработка данных. Очевидно, что нужно скопировать сами программы Map и Reduce, а также здесь можно также указать необходимые им файлы данных.
- **Опции** `-mapper` и `-reducer` содержат командные строки, которые нужно выполнить, чтобы запустить соответствующие программы для операция Map и Reduce.

Вопросы для конспекта

4. Какой язык программирования является стандартным для MapRedcue? По какой причине в настоящее время это считается недостатком?
5. Расскажите о принципах работы Hadoop Streaming. Как реализуется возможность использовать произвольные языки программирования для разработки MapReduce-задач?

Основные сведения об Apache Hive

Предположим, что нужно проанализировать 1 Тб логов сервера и получить ответ на простой запрос: «подсчитать количество запросов от каждого IP-адреса».

Предположим, что в распоряжении аналитика данных, который решает эту задачу, имеется кластер Hadoop и инструменты MapReduce.

- Пишем код Mapper и Reducer на Java или на другом языке, если используется Hadoop Streaming.
- Отлаживаем код, имея дело с достаточно сложными и массивными сообщениями об ошибках.
- Запускаем задачу на исполнение.
- Ждём достаточно долго, возможно несколько часов, чтобы получить ответ.

Какие здесь возникают проблемы:

- **Низкоуровневость:** Приходится думать в категориях парадигмы MapReduce, а не о самой задаче.

- **Медленная разработка:** Написание, отладка и запуск кода на Java или Python для каждого нового вопроса занимает много времени.
- **Порог входа:** У аналитиков, которые виртуозно владеют SQL, но не Python/Java, нет возможности работать с данными в Hadoop.

Решение этих проблем: создать транслятор с SQL в MapReduce и предоставить аналитикам возможность готовить запросы на SQL.

● **Apache Hive** — это система управления данными в распределённых хранилищах, которая предоставляет SQL-подобный язык запросов (HiveQL) для чтения, записи и управления большими наборами данных.

Изначально Hive представлял собой набор скриптов поверх Hadoop Streaming, а позже развился в полноценный фреймворк для выполнения запросов к данным поверх MapReduce.

Ключевые особенности:

- **Не СУБД:** Hive не является системой управления базами данных. В отличие от СУБД, данные Hive — это обычные файлы в файловой системе. С ними можно работать напрямую, а Hive даёт возможность работы с ними через SQL.
- **Нет транзакций:** Hive не поддерживает транзакции как это делает PostgreSQL или MySQL.
- **Надстройка над Hadoop:** Hive транслирует SQL-запросы в набор заданий MapReduce (в настоящее время поддерживаются и другие низкоуровневые интерфейсы), которые выполняются в кластере Hadoop.
- **«Схема при чтении» (Schema-on-Read):** В отличие от традиционных СУБД, в которых схема данных применяется при записи («схема при записи»), Hive не проверяет данные при их загрузке в HDFS. Он применяет схему, когда производится чтение данных. Это обеспечивает гибкость и высокую скорость загрузки данных.

Работа Hive строится по следующей схеме:

- Данные хранятся в виде файлов, в принятых в HDFS форматах.
- Пользователь создаёт SQL запрос, который передаётся в Hive.
- Hive транслирует запрос в низкоуровневый код обработки данных и передаёт его YARN на исполнение.

Изначально Hive транслировал запросы в формат MapReduce задач.

Впоследствии, для достижения более высокого быстродействия, был разработан альтернативный низкоуровневый инструмент обработки данных: Apache Tez. Кроме того, в настоящее время Hive может также работать поверх Spark.

Примечание

Apache Tez — это высокопроизводительная, расширяемая платформа для пакетной и интерактивной обработки данных, разработанная в первую очередь для работы с Apache Hive и работающая в экосистеме Apache Hadoop под управлением YARN. Работа Tez основана на построении графов обработки данных (DAG, Directed Acyclic Graph) и обработки данных на их основе. Это обеспечивает более быстрый доступ к данным по сравнению с MapReduce.

Ключевые компоненты Hive:

- **HiveQL:** Диалект SQL. Очень похож на стандартный SQL, но с некоторыми расширениями для работы с Big Data.
- **Драйвер:** Принимает HiveQL-запрос, управляет его выполнением и отслеживает жизненный цикл.
- **Компилятор:** Выполняет трансляцию запроса HiveQL в низкоуровневые команды, организованные в виде графа исполнения — Directed Acyclic Graph, DAG.
- **Метахранилище (Metastore):** Это база данных (обычно MySQL/PostgreSQL), где хранится метаданные об обрабатываемых таблицах данных (иными словами — устанавливается соответствие между файлами в HDFS и их представлением в виде таблиц):
 - Структура таблиц (имена, типы столбцов).
 - Информация о партициях (Partitioning) таблиц.
 - Местоположение соответствующих файлов данных в HDFS.

Metastore — это то, что превращает файл в HDFS в структурированные таблицы.

- **Движок (Execution Engine):** Получает от компилятора план выполнения в виде DAG и выполняет его. По умолчанию использует для этого MapReduce, но может использовать более быстрые движки, как Apache Tez или Spark.

Примечание

Вспомним, как обеспечивается кроссплатформенность приложений на Java: сначала исходный код компилируется в платформенно-независимый байт-код, а затем этот байт-код выполняется виртуальной машиной Java, которая работает на разных платформах. Аналогичным образом работает связка Компилятор-Движок в Hive: компилятор преобразует запрос в промежуточный код, оформленный в виде DAG, а движок переносит DAG в конкретную среду исполнения: MapReduce, Tez или Spark.

Вопросы для конспекта

6. Опишите проблемы, с которыми сталкиваются аналитики данных при необходимости работать с MapReduce-задачами непосредственно?
7. Что собой представляет Apache Hive? Опишите ключевые особенности.
8. Опишите этапы работы с данными при использовании Apache Hive.
9. Перечислите и опишите ключевые компоненты Apache Hive.

Этапы работы с данными в Apache Hive: сопоставление файлам
схемы данных в виде таблицы

Предположим, что у нас есть CSV-файлы с логами веб-сервера `log0001.csv`, `log0002.csv`, `log0002.csv`, ...

Эти файлы расположены в HDFS в папке `/user/data/logs`.

Файлы содержат разделённые запятыми поля `ip_address` (тип `STRING`), `log_time` (тип `TIMESTAMP`) и `request` (тип `STRING`):

```
192.168.1.1, 2023-10-25 10:00:01, GET /index.html
192.168.1.2, 2023-10-25 10:00:02, POST /login
192.168.1.1, 2023-10-25 10:00:03, GET /style.css
```

Для работы с этими данными при помощи Hive нужно сначала файлам сопоставить таблицу, к которой потом можно будет делать запросы.

Для этого выполним следующий запрос:

```
CREATE EXTERNAL TABLE raw_logs (
    ip_address STRING, log_time TIMESTAMP, request STRING
)
ROW FORMAT DELIMITED
    FIELDS TERMINATED BY ','
LOCATION '/user/data/logs';
```

- Инструкция `CREATE EXTERNAL TABLE` означает, что данные будут храниться в файлах, которыми Hive не владеет — может их только читать. Альтернатива — таблица на основе файлов, созданных и управляемых самим Hive. Такие таблицы называются Managed Tables.
- Инструкция `ROW FORMAT` сообщает Hive, что поля разделены запятыми.
- Инструкция `LOCATION` указывает папку, в которой лежат файлы. В этой опции нельзя указать путь к конкретному файлу. Hive создаёт таблицу на основе всех файлов в указанной папке.

В принципе этого достаточно чтобы начать работать с данными используя SQL. Но для увеличения производительности рекомендуется выполнить их партиционирование.

Партиционирование в Hive может быть статическим и динамическим. Статическое — мы вручную распределяем данные по партициям,

выполняя запросы `INSERT` для каждой партии отдельно.

Динамическое — партии создаются автоматически на основе заданного критерия.

Создадим таблицу с партиционированием по дням

```
CREATE TABLE partitioned_logs (  
    ip_address STRING, request STRING, log_date STRING  
)  
PARTITIONED BY (log_date STRING)  
STORED AS ORC;
```

- Инструкция `CREATE TABLE` (без `EXTERNAL`) означает, что будет создана управляемая таблица (Managed Table) — данные в ней находятся под управлением Hive. Они также будут храниться в файлах, расположенных в системных папках Hive в HDFS. Эта область называется «warehouse» (переводится как хранилище).
- Инструкция `PARTITIONED BY` задаёт партиционирование.
- Инструкция `STORED AS ORC` указывает, что файлы данных должны создаваться в формате ORC.

Теперь переносим данные из исходной таблицы в таблицу с динамическим партиционированием.

```
INSERT OVERWRITE TABLE partitioned_logs PARTITION (log_date)
SELECT ip_address, request, DATE(log_time) AS log_date
FROM raw_logs;
```

В результате этой операции Hive создаст в своей системной области «warehouse» отдельные папки для каждой даты:

```
/user/hive/warehouse/partitioned_logs/log_date=2023-10-07/
/user/hive/warehouse/partitioned_logs/log_date=2023-10-08/
/user/hive/warehouse/partitioned_logs/log_date=2023-10-09/
```

Перенос данных при статическом партиционировании выполняется так:

```
INSERT INTO TABLE partitioned_logs PARTITION (log_date = '2023-10-07')
SELECT ip_address, request, DATE(log_time) AS log_date
FROM raw_logs
WHERE DATE(log_time) = '2023-10-07';
```

```
INSERT INTO TABLE partitioned_logs PARTITION (log_date = '2023-10-08')
SELECT ip_address, request, DATE(log_time) AS log_date
FROM raw_logs
WHERE DATE(log_time) = '2023-10-08';
```

- Опция **PARTITION** (`log_date = ...`) указывает в какую партицию писать данные.
- Инструкция **WHERE DATE**(`log_time`) = ... задаёт условие отбора данных в таблице-источнике.

В итоге также будут созданы папки с файлами данных в области warehouse.

Таким образом, мы создали таблицу `partitioned_logs`. Описание структуры этой таблицы хранится в метастранище (Metastore) Hive. Данные записаны в виде файлов в HDFS.

Вопросы для конспекта

10. В чём состоит подготовка к использованию данных в Apache Hive, если они изначально заданы в виде файлов?
11. Что такое внешние (external) и управляемые (managed) таблицы в Apache Hive? В чем их отличия?
12. Для чего нужно партиционирование данных в таблицах? Как оно выполняется в Apache Hive, если данные изначально заданы в виде файлов?

Этапы работы с данными в Apache Hive: выполнение запроса к таблице

Пусть в созданной таблице логов нам нужно подсчитать количество обращений с каждого IP-адреса, которые произошли 25 октября 2023 года.

Мы готовим следующий запрос:

```
SELECT ip_address, COUNT(*)  
FROM partitioned_logs  
WHERE log_date = '2023-10-25'  
GROUP BY ip_address;
```

Обработкой этого запроса занимается компонент Hive, который называется драйвер.

В запросе мы обращаемся к таблице `partitioned_logs`, но как говорилось выше, на самом деле все данные хранятся в виде файлов в HDFS.

За сопоставление файлов данных и таблиц отвечает метастранилище.

Поэтому после получения запроса драйвер первым делом обращается в метастранилище и получает следующую информацию:

- Где в HDFS лежат данные для таблицы `partitioned_logs`?
- Какого типа столбцы `ip_address` и `log_date`?
- Разбита ли таблица на партии по дате? Какое имя у партии, в которой находятся нужные данные?

Эту информацию вместе с запросом драйвер передаёт компилятору, а затем движку которые транслируют запрос в MapReduce-код на Java (если Hive работает с движком MapReduce).

Для нашего запроса логика Mapper будет выглядеть примерно так (для большей наглядности использован Python):

```
for line in data_batch: # получаем очередную строку
    if extract_log_date(line) == '2023-10-25':
        ip = extract_ip(line)
        print f"({ip}, 1)" # формируем пару ключ-значение
```

Логика Reducer будет стандартной — после Shuffle (сортировки по ключу `ip`) посчитать количество повторяющихся кортежей.

Вопросы для конспекта

13. Опишите этапы выполнения запроса на выборку данных при работе с Apache Hive поверх MapReduce.

Вопросы для конспекта

- 1 Что такое MapReduce? Что является главным принципом для работы с данными на основе MapReduce?
- 2 Какие этапы обработки проходят данные в MapReduce? Какие из них стандартные, а какие зависят от реализуемого алгоритма?
- 3 В чём состоят преимущества и недостатки MapReduce?
- 4 Какой язык программирования является стандартным для MapReduce? По какой причине в настоящее время это считается недостатком?
- 5 Расскажите о принципах работы Hadoop Streaming. Как реализуется возможность использовать произвольные языки программирования для разработки MapReduce-задач?
- 6 Опишите проблемы, с которыми сталкиваются аналитики данных при необходимости работать с MapReduce-задачами непосредственно?
- 7 Что собой представляет Apache Hive? Опишите ключевые особенности.
- 8 Опишите этапы работы с данными при использовании Apache Hive.
- 9 Перечислите и опишите ключевые компоненты Apache Hive.
- 10 В чём состоит подготовка к использованию данных в Apache Hive, если они изначально заданы в виде файлов?
- 11 Что такое внешние (external) и управляемые (managed) таблицы в Apache Hive? В чем их отличия?
- 12 Для чего нужно партиционирование данных в таблицах? Как оно выполняется в Apache Hive, если данные изначально заданы в виде файлов?
- 13 Опишите этапы выполнения запроса на выборку данных при работе с Apache Hive поверх MapReduce.

Лекция 10. Общие сведения об Apache Spark

Предназначение Apache Spark и его место среди систем для работы с большими данными

Apache Spark — это универсальная и высокопроизводительная платформа, предназначенная **для организации распределённых вычислений** на кластерах, в том числе над большими данными.

Работа над Spark началась в 2009 году. Spark с самого начала создавался с целью преодоления неэффективности Hadoop MapReduce для итеративных и интерактивных вычислительных задач.

Spark и большая часть его библиотек написаны на Scala, код выполняется JVM (виртуальной Java-машиной).

Примечание

Scala (SCAlable LAnguage) — это, мультипарадигмальный язык программирования, который сочетает объектно-ориентированные и функциональные подходы. Он компилируется в байт-код и работает на виртуальной машине Java (JVM), что обеспечивает совместимость с Java и высокая производительность. Scala позволяет использовать Java-библиотеки и легко интегрироваться в существующие Java-проекты.

Scala был разработан с целью устранить некоторые недостатки Java и поэтому достаточно сильно похож на Java. Scala более лаконичен чем Java. Кроме того, Scala также имеет сильное влияние со стороны функциональных языков, таких как Haskell.

Быстрые вычисления. Одно из главных качеств — обработка данных происходит существенно быстрее чем в MapReduce (в некоторых случаях до ста раз).

Это достигается за счёт отказа от универсальной схемы вычислений, реализуемой в MapReduce, а также за счёт минимизации обращений к долговременной памяти (стараясь не создавать временные файлы, всё храним в ОЗУ, настолько, насколько это возможно).

Универсальность. Spark охватывает широкий диапазон рабочих нагрузок, которые прежде требовали создания отдельных распределенных систем:

- пакетная обработка данных
- итерационные алгоритмы обработки данных
- интерактивные запросы (вопросы-ответы с короткой задержкой)
- обработка потоковых данных

Spark поддерживает разные виды задач с помощью общего для всех сценариев вычислительного движка (computational engine, часть ядра Spark).

Гибкость. Spark может работать на кластерах, управляемых Hadoop YARN, Kubernetes, Apache Mesos, а также включает в себя инструменты для развёртывания собственного кластера.

Примечание

Kubernetes (часто сокращают до K8s, происходит от греческого слова «рулевой») предназначен для автоматизации развёртывания, управления и масштабирования контейнеризированными приложениями (например, при помощи Docker). Kubernetes запускает контейнеры на большом количестве хостов, контролирует их состояние, автоматически перезапускает в случае выхода из строя, обеспечивает масштабирование кластера при необходимости.

Родственное приложение Docker Compose предназначено для управления многоконтейнерными приложениями на одном хосте, а Kubernetes управляет контейнерами, работающими в кластере из нескольких машин.

Примечание

Apache Mesos — это универсальная система управления кластерами, которая может управлять разнообразными рабочими нагрузками, включая контейнеры, виртуальные машины и другие приложения. Разрабатывался при участии создателей Spark.

Mesos был создан раньше Kubernetes (в 2009 году, а K8s в 2014) но проиграл ему в конкуренции. В 2025 году развитие прекращено.

Кластер Spark может быть развёрнут на локально или в облаке.

Также Spark может быть использован как обычная библиотека на локальном компьютере вообще без развёртывания кластера.

Поддержка разных языков программирования. Spark поддерживает разработку приложений для обработки данных на многих языках программирования: Scala, Java, Python, R и SQL. Приложения взаимодействуют со Spark через его API.

Примечание

R — язык программирования для статистической обработки данных и работы с графикой, а также свободная программная среда вычислений с открытым исходным кодом. Он широко используется для обработки больших массивов данных, машинного обучения и визуализации. R отличается уникальным синтаксисом, оптимизированным для работы с данными, и содержит множество встроенных функций и алгоритмов.

Вспомним, что MapReduce тоже позволяет использовать разные языки программирования. Однако там менее эффективно, так как реализуется через трубопроводы (pipelines), что сводится к передачи данных между отдельными приложениями через временные файлы.

Унификация и интеграция исполнения задач. Одна из основных идей, реализуемых Spark — тесная интеграция и унификация исполнения различных вычислительных задач (запросы SQL, машинное обучение, потоковая обработка и т. д.) по аналогии с тем как в одном программном проекте различные задачи решаются за счёт разных библиотек.

Это достигается за счёт того, что всё исполнение в конечном счёте происходит в универсальном вычислительном движке.

Это упрощает и удешевляет объединение разных видов обработки, которые часто необходимо выполнять в едином конвейере.

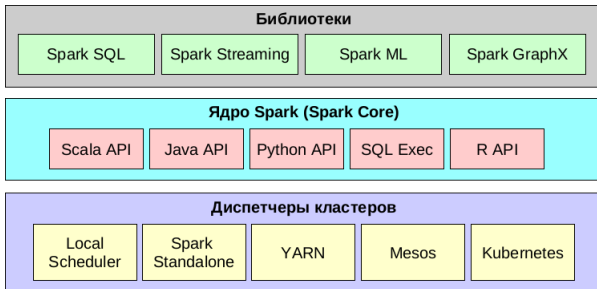
Кроме того, унифицированный подход снижает затраты на обслуживание инфраструктуры.

Вопросы для конспекта

1. Что такое Apache Spark? Для каких видов рабочих нагрузок он предназначен?
2. За счёт чего обеспечивается быстроедействие Spark-приложений?
3. Какие существуют варианты развёртывания кластера Apache Spark?
4. Какие языки программирования поддерживает Apache Spark?
5. За счёт чего обеспечивается унификация и интеграция исполнения задач на платформе Apache Spark?

Основные компоненты Spark

Стек Spark состоит из трёх слоёв: диспетчер кластера (управление «железом»), ядро, высокоуровневые библиотеки.



- **Диспетчер кластера.** Кластер, на котором выполняются задачи Spark, находится под управлением диспетчера. Диспетчер может быть сторонним, встроенным и отсутствовать вообще для локального запуска Spark.

Поддерживаемые сторонние диспетчеры кластера:

- Hadoop YARN
- Apache Mesos (разработка Mesos прекращена)
- Kubernetes

Встроенный диспетчер кластера называется Spark Standalone. По сравнению с другими, Spark Standalone обеспечивает только базовый функционал.

Spark Standalone прост в установке и настройке. Подходит для управления не очень большими кластерами.

Spark Standalone хорошо интегрирован с приложениями Spark, однако сторонние, не Spark приложения, поддерживаются не достаточно хорошо, по сравнению, например, с YARN.

Spark Standalone поддерживает восстановление после сбоев, но будет не так надёжен, как другие диспетчеры. Имеет Master-узел — это точка отказа кластера. При выходе его из строя кластер перестаёт работать.

Spark Standalone поддерживает базовые инструменты планирования выполнения задач, но у других диспетчеров они более развитые.

Spark Standalone поддерживает основные инструменты обеспечения безопасности. Они не так развиты как у других диспетчеров.

В Spark нет собственной поддержки распределённого хранения файлов, как в HDFS. Но для поддержки распределённых вычислений выполняется разделение файлов данных на партии, которые распределяются между узлами-исполнителями. На узлах партии хранятся в виде обычных файлов локальной файловой системы.

Для работы с большими данными необходимо разворачивать Standalone кластер поверх распределённой файловой системы, например HDFS.

Spark можно запускать на локальном компьютере. Приложения, исполняемые локально и на кластере используют один и тот же API.

Код, написанный и протестированный локально на небольшом наборе данных, может быть без модификаций перенесён на большой кластер.

Примеры сценариев использования Spark с разными диспетчерами кластера

- **Локальна установка** — прототипирование и тестирование кода.
- **Standalone** — небольшой кластер для распределённых вычислений. Не для больших данных. Простота в развёртывании, но ограниченные возможности по управлению задачами, не очень высокая отказоустойчивость.
- **Standalone поверх HDFS** — все преимущества и недостатки Standalone, но возможность распределённого хранения данных.
- **Kubernetes** — не очень большой кластер с большими по сравнению со Standalone возможностями по управлению его работой.
- **Hadoop YARN и HDFS** — дорогое, тяжеловесное и максимально мощное решения для работы с по настоящему большим данным.

- **Механизмы хранения данных для Spark.** Для реализации распределённого хранения файлов Spark опирается на сторонние решения, такие как HDFS или Amazon S3.

Spark может создавать распределённые наборы данных для их параллельной обработки из любых файлов, хранящихся в HDFS или в других системах хранения данных, включая локальную файловую систему.

Spark поддерживает текстовые файлы, файлы SequenceFile, Avro, Parquet и любые другие форматы, поддерживаемые Hadoop.

- **Ядро Spark.** Проект Spark включает множество тесно связанных компонентов. Их интеграцию и взаимодействие обеспечивается ядром Spark, Spark Core. Ядро обеспечивает выполнение API-функций, планирование заданий, управление памятью, обработку ошибок, взаимодействие с системами хранения данных.

- **Spark SQL** — высокоуровневая библиотека для работы со структурированными данными на основе SQL-запросов.

Поддерживается чистый SQL и его Hive-диалект Hive Query Language (HQL).

Поддерживает множество источников данных, включая таблицы Hive, файлы Parquet и JSON.

В дополнение к интерфейсу SQL компонент Spark SQL позволяет смешивать в одном приложении запросы SQL с программными конструкциями на Python, Java и Scala, и таким способом комбинировать SQL со сложной аналитикой.

- **Spark Streaming** — компонент Spark для обработки потоковых данных. Примеры источников таких: файлы журналов, заполняемые действующими веб-серверами, или очереди сообщений, посылаемых пользователями веб-служб.
- **Spark ML** (в ранних версиях Spark MLlib) — библиотека для решения задач машинного обучения.

Изначальной основной структурой данных в Apache Spark был RDD (Resilient Distributed Dataset, надежная распределенная коллекция).

Поэтому в ранних версиях Spark MLlib работал именно с API RDD, будучи упакован в пакет `spark.mllib`.

Но в впоследствии более популярными стали высокоуровневые структуры данных `DataFrame`. Начиная с версии 2.0, выпущенной в 2016 году, API для машинного обучения на основе `DataFrame` стали называть Spark ML, а библиотека MLlib на базе RDD перешла в режим обслуживания.

Это означает, что в API MLlib на основе RDD больше не добавляются новые функции, а только исправляются выявленные ошибки.

(Структуры данных RDD и `DataFrame` обсуждаются ниже.)

В Spark ML реализованы алгоритмы классического машинного обучения.

Работа с нейронными сетями также поддерживается в Spark, но на основе дополнительных библиотек, например `TensorFlowOnSpark`.

- **Spark GraphX** — библиотека для обработки графов и выполнения параллельных вычислений.

Подобно компонентам Spark Streaming и Spark SQL, GraphX дополняет GraphX поддерживает разнообразные операции управления графами и библиотеку обобщенных алгоритмов работы с графами.

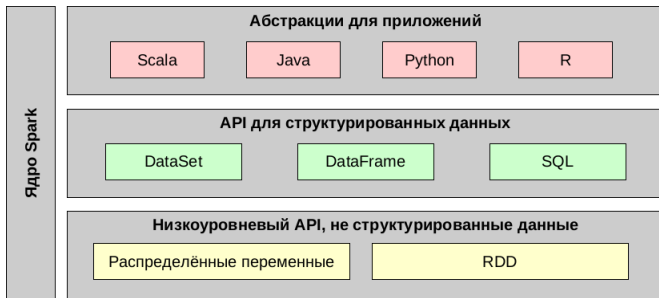
Вопросы для конспекта

6. Опишите состав стека Apache Spark.
7. Какие диспетчеры кластера поддерживаются в Apache Spark?
8. Расскажите про диспетчер кластера Spark Standalone. В чём его преимущества и недостатки?
9. Приведите примеры сценариев использования Apache Spark с разными диспетчерами кластера.
10. Расскажите про механизмы хранения данных в Apache Spark.
11. Какие функции выполняют ядро Spark и вычислительный движок?
12. Перечислите и опишите высокоуровневые библиотек Apache Spark.

Базовые структуры данных, используемые в Spark

Архитектура ядра Spark построена вокруг обработки данных, которые поддерживают разделимость (вспомним лекцию про типы данных в HDFS).

Ядро Spark содержит многоуровневый API для работы с данными



- Первый, наиболее низкий уровень — работа неструктурированным разделимыми данными: распределённые или общие переменные (предназначены для обмена данными между различными задачами в распределённой системе), RDD (распределённая коллекция объектов).
- Второй уровень — API для работы со структурированными данными, специфическим для Spark: DataSet (коллекция объектов), DataFrame (таблицы данных), поддержка SQL. DataFrame и DataSet в Spark построены на основе RDD, но представляют собой более высокоуровневые, оптимизированные абстракции.
- Третий уровень — работа с данными через языки программирования, поддержка абстракций этих языков (массивы, списки и т. д.).

- **RDD** (Resilient Distributed Datasets, устойчивые распределённые наборы данных) — основная структура данных в Spark. Представляет собой коллекцию произвольных объектов, распределённых по ОЗУ рабочих узлов кластера.

Обратить внимание: RDD — это не файлы. Если сравнить RDD с «обычными» данными языков программирования, то это больше похоже на массивы — работа с ними производится в оперативной памяти компьютера.

Но если массив всегда находится в оперативной памяти одного компьютера, то RDD — это обычно распределённая оперативная память. (Есть вариант локальной работы со Spark и тогда все RDD находятся в оперативной памяти одного компьютера.)

За счёт разделения, RDD позволяют выполнять операции параллельной обработки как локально, так и на кластере машин.

Фактически, вся работа Spark заключается в совершении операций над RDD.

Свойства RDD.

- **Коллекция объектов** — RDD могут содержать объекты любого типа на Python, Java или Scala, включая экземпляры пользовательских классов.
- **Распределённость** — Spark автоматически распределяет данные в наборах RDD между компьютерами в кластере и распараллеливает выполнение операций над ними.
- **Неизменяемость** — все вычисления, производимые над RDD сохраняются в виде новых RDD. Модификация существующих структур «на месте» (inplace) не возможна.
- **Отказоустойчивость** — обеспечивается за счёт неизменяемости RDD и за счёт того, что информации о происхождении данных и о выполненных над ними операциях протоколируется. Это позволяет восстанавливать RDD в случае сбоя, пересчитывая их из исходных данных.

- **Промежуточные результаты.** RDD — это формат данных, находящихся в процессе обработки. Исходные данные и результаты вычислений хранятся в виде обычных файлов. Данные загружаются в RDD перед началом работы и выгружаются в другие форматы после окончания работы.

Сравним подход Spark с подходом Hadoop:

- Обработка данных в Hadoop построена вокруг работы с файлами. Приложения Hadoop умеют работать с разными форматами файлов, но для поддержки распределённых вычислений требуется, чтобы файлы поддерживали делимость. Для этого были разработаны специальные типы файлов, например, Parquet.
- Spark строит свою работу не вокруг файлов, а вокруг данных в оперативной памяти. Задан базовый формат данных RDD с поддержкой делимости, и вокруг него выстроена вся вычислительная инфраструктура.

Работая непосредственно с RDD программист имеет максимальный доступ к сырым данным и управляет каждым объектом в RDD.

Это даёт максимальную гибкость и позволяет реализовать любую, даже самую сложную и нереляционную логику. Например, можно работать неструктурированными данными, например с потоком текста.

В силу отсутствия схемы данных, автоматическая оптимизация операций, хотя и выполняется, но не может достаточно «сильной».

Для повышения эффективности в Spark имеются структуры данных для работы со структурированными данными.

- **DataFrame** — распределенная коллекция данных, которая организована в именованные столбцы, подобно таблице в реляционной базе данных. Использует схему данных (заданы имена столбцов и их типы).

$\text{DataFrame} = \text{RDD} + \text{Схема данных}.$

DataFrame в Spark создавалась по аналогии с DataFrame в Pandas: совпадают принципы выполнения преобразований данных и названия методов.

Однако в целом Pandas предоставляет больше инструментов по работе с данными чем Spark DataFrame.

В настоящее время DataFrame — это основной формат данных для большинства задач.

Недостаток DataFrame — отсутствие статической типизации.

Схема данных задана, но соответствие данных схеме проверяется только в момент их использования.

Это значит, что нет возможности найти ошибки на этапе компиляции. Они всплывут только на этапе исполнения (ошибки могут проявиться только после того, как код отправлен в продакшен).

Такой подход характерен для Python, в котором типизация динамическая.

В Scala и Java статическая типизация поддерживается.

Чтобы задействовать преимущества статической типизации, в Spark имеется ещё одна структура данных, `DataSet`. С ней можно работать только в Scala и Java. В PySpark она не поддерживается.

- **DataSet** — распределённая коллекция объектов одного, заданного при создании, типа. Используется схема данных со статической типизацией — соответствие данных схеме проверяется на этапе компиляции.

`DataSet` = `RDD` + Схема данных + Статическая типизация.

Вследствие строгой типизации, с `DataSet` можно работать только в Scala и Java, так как у Python типизация только динамическая.

Отличие `DataSet` от `DataFrame` можно проиллюстрировать на простом примере, написанном на Scala.

Сначала вариант с `DataFrame`. Создаём `DataFrame`:

```
// Импортируем всё необходимое
import spark.implicits._

// Исходный DataFrame
val peopleDF = Seq(("Alice", 25), ("Bob", 30), ("Charlie", 35))
                .toDF("name", "age")
```

```
peopleDF.show()
// +-----+----+
// |  name|age|
// +-----+----+
// | Alice| 25|
// |   Bob| 30|
// |Charlie| 35|
// +-----+----+
```

Допустим, мы хотим увеличить возраст каждого человека. Но по ошибке попробуем прибавить строку к числу.

```
// Неверный запрос: прибавляем строку "1" к числовому столбцу "age"
val incorrectDF = peopleDF.select($"name", $"age" + "1")

// Этот код УСПЕШНО скомпилируется...
// ... но упадет с ошибкой ВО ВРЕМЯ ВЫПОЛНЕНИЯ (Runtime):
// org.apache.spark.sql.AnalysisException: cannot resolve '(`age` + '1')'
```

Компилятор не знает структуры данных и типов, поэтому он не может найти эту ошибку заранее. Оно проявится только во время выполнения.

Теперь преобразуем наш DataFrame в строго типизированный DataSet.

```
// Case class определяет схему и типы на уровне компиляции  
case class Person(name: String, age: Int)  
  
// Преобразуем наш DataFrame в типизированный DataSet  
val peopleDS = peopleDF.as[Person] // DataSet[Person]
```

Теперь попробуем выполнить ту же ошибочную операцию, но используя типизированные преобразования (например, map).

```
// Пытаемся сделать то же самое с помощью map  
val incorrectDS = peopleDS.map(person => person.name + (person.age + "1"))  
  
// ЭТОТ КОД НЕ СКОМПИЛИРУЕТСЯ!  
// [ERROR] ... type mismatch;  
// found    : String  
// required: Int  
//      person.age + "1")  
//      ^
```

Компилятор знает, что `person.age` имеет тип `Int`, и видит, что мы пытаемся сложить `Int` и `String`. Он немедленно сообщает об ошибке, исключая возможность запуска некорректного кода.

Общие рекомендации по использованию структур данных:

- `DataFrame` — основная структура для большинства задач. Максимальная оптимизация операций и отсутствие необходимости «возиться» со статической типизацией (Написание кода со статической типизацией более трудоёмко — ср. Python и C++.)
- `DataSet` — используется когда нужна максимальная надёжность. Требуется выявить все потенциальные ошибки как можно раньше.
- `RDD` — работа с неструктурированными и полуструктурированными данными, решение нестандартных задач.

- **Ленивые (отложенные) вычисления и автоматическая оптимизация.** Операции над наборами данных всех типов (RDD, DataFrame, DataSet) выполняются лениво.

Инструкции не выполняются непосредственно в момент их появления. Spark их протоколирует и создаёт на их основе DAG (Directed Acyclic Graph, ориентированный ациклический граф) операций.

Выполнение операций DAG производится только когда запрашивается результат преобразований.

Благодаря ленивыми вычислениям, имеется возможность выполнить оптимизацию графа операций перед его выполнением.

Оптимизация для DataSet и DataFrame более эффективна чем для RDD благодаря наличию схемы данных.

Иногда достигается 10-100x ускорение по сравнению с RDD. Также можно получить до 90% экономии памяти.

Оптимизация выполняется автоматическая, без участия пользователя.

- **Партиционирование.** Все структуры данных, RDD, DataFrame и DataSet, автоматически разделяются на партии, которые размещаются в оперативной памяти рабочих узлов кластера. Каждый узел получает и обрабатывает некоторое подмножество партий.

Это соответствует партиционированию таблиц по строкам в СУБД. Разделение по столбцам, как в столбцовых базах, не производится.

Spark выполняет партиционирование автоматически, в соответствии с заданными параметрами. Также существуют возможность выполнить партиционирование явным образом, вручную.

- **Распределённые переменные.** Spark производит обработку данных в распределённом режиме — код выполняется на разных узлах кластера. Распределённые переменные играют роль глобальных переменных в обычных программах: все фрагменты исполняемого кода могут читать и записывать эти переменные и таким образом обмениваться информацией.

Вопросы для конспекта

13. Расскажите про многоуровневый API для работы с данными Apache Spark.
14. Что такое RDD в Apache Spark? Перечислите и опишите основные характеристики.
15. Сравните подход к работе с данными в Apache Spark на основе RDD и универсальный подход, реализуемый в Hadoop.
16. Опишите DataFrame в Apache Spark. За счёт чего операции с ним будут выполняться быстрее, чем с RDD?
17. Опишите DataSet в Apache Spark. Сравните его с DataFrame.
18. Как выполняется разделение данных между узлами кластера в Apache Spark?

Структура Spark кластера и архитектура приложения

Структура кластера Spark зависит от используемого менеджера кластера. Если используется менеджер Spark Standalone, то потребуются узлы двух типов.

- **Мастер-узел (Master)** — узел, который выполняет роль менеджера кластера. Управляет ресурсами кластера, отслеживает состояние рабочих узлов, координирует выполнение приложений.
- **Рабочие узлы (Workers)**. Хранят в оперативной памяти партиции (части) обрабатываемых наборов данных, хранят в долговременной (дисковой) памяти кеши обрабатываемых данных, запускают исполнителей. Исполнители — это процессы, которые выполняют задачи по обработке данных. На одном рабочем узле могут быть запущены несколько исполнителей.
- **Сервер истории (History Server)**. Не обязательный узел, который хранит логи выполнения приложений.

Если Spark развернут на кластере Hadoop (YARN + HDFS), то мастер-узел Standalone кластера не используется. Его функцию выполняет YARN.

Рабочие узлы разворачиваются на узлах данных DataNode (хранение данных HDFS) на которых также работают менеджеры узла NodeManager (вычисления, управляемые YARN).

Вопросы для конспекта

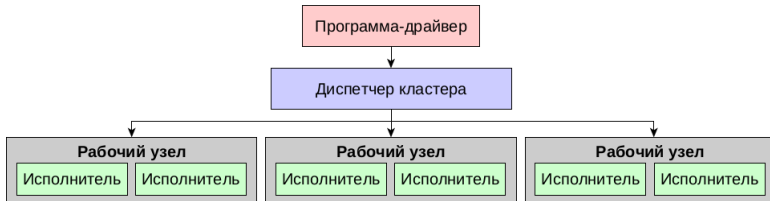
19. Из каких узлов состоит кластер Apache Spark под управлением Spark Standalone?

20. Из каких узлов состоит кластер Apache Spark под управлением YARN?

Spark-приложение

Spark-приложение состоит из двух компонентов:

- драйвер;
- исполнители.



● **Драйвер** — это процесс, в котором выполняется функция `main()` (в случае Scala и Java) Spark-приложения. Драйвер — это не программа пользователя. Это компонент Spark, который получает исходный код программы. Драйвер работает следующим образом:

- создаёт и поддерживает сессию Spark;
- получает исходный код программы и транслирует его в план параллельного исполнения на основе DAG;
- формирует на основе исходного кода задания исполнителям;
- распределяет задачи между исполнителями;
- собирает и сохраняет результирующие данные.

● **Исполнители** — это компоненты Spark; их процессы запускаются на рабочих узлах в кластере Spark и отвечают за обработку фрагментов данных, размещённых на этом узле в соответствии с заданиями драйвера. Каждый экземпляр приложения Spark запускает своих собственных исполнителей.

Количество исполнителей зависит от используемого менеджера кластера и режима его работы.

В кластере Spark Standalone число исполнителей задаётся при запуске приложения. Оно выбирается равны суммарному числу вычислительных ядер на рабочих узлах.

В кластере YARN число исполнителей может быть фиксированным, а может меняться в зависимости от нагрузки кластера (это рекомендованный режим).

- **Запуск Spark-приложения.** Spark-приложение запускается командой `spark-submit`. Существует несколько режимов (`deploy-mode`).

Режим клиента

```
spark-submit --deploy-mode client --master yarn app.py
```

Здесь `yarn` — это URL узла, на котором запущен YARN, `app.py` — имя файла приложения.

Драйвер запускается на той же машине, откуда выполняется команда `spark-submit`, например на пользовательском ноутбуке. Легче в отладке, так как логи видны напрямую. Остановка клиентской машины приводит к остановке приложения.

Режим кластера

```
spark-submit --deploy-mode cluster --master yarn app.py
```

Драйвер запускается внутри кластера на одном из рабочих узлов (менеджер кластера выбирает узел). Для отладки нужно смотреть логи через веб-интерфейс. Клиент можно отключить после запуска.

Локальный режим

```
spark-submit --master local[4] app.py
```

Здесь 4 — число исполнителей.

Драйвер и исполнители работают в одном процессе JVM на локальной машине.

Если работа со Spark производится в Jupyter через PySpark, то Spark работает в интерактивном режиме, в режиме клиента. Команда `spark-submit` в этом случае не используется. За запуск приложений отвечает `pyspark`.

- **Сценарий запуска приложения Spark.**

- Запуск приложения осуществляется консольной командой `spark-submit` или `pyspark` при работе в Python / Jupyter.
- Драйвер связывается с менеджером кластера (YARN, Spark Standalone) и запрашивает ресурсы для исполнителей.
- Менеджер кластера находит подходящие рабочие узлы и запускает на них процессы исполнителей.
- После запуска исполнитель регистрируется в драйвере, сообщая о своей готовности выполнять задачи. Далее исполнители и драйвер взаимодействуют друг с другом напрямую.

● **Spark-контекст (SparkContext)** — одна из базовых концепций в Spark, представляет собой точку входа для взаимодействия с кластером и управления всеми его ресурсами. В программе представлен в виде объекта. Функции контекста Spark:

- Координирует распределённые процессы в кластере Spark.
- Управляет подключением к кластеру и распределением ресурсов.
- Является посредником между пользовательской программой и кластером Spark.

Взаимодействие программы со Spark происходит через атрибуты и методы объекта, представляющего контекст.

Разные подсистемы Spark предоставляют собственные контексты: контекст для RDD и низкоуровневых операций, контекст для работы с SQL, для взаимодействия с Hive и т.д.

```
from pyspark import SparkContext, SparkConf
# Создание конфигурации
conf = SparkConf().setAppName("MyApp").setMaster("spark://master-node:7077")

# Создание SparkContext - именно в этот момент мы подключаемся к кластеру
sc = SparkContext(conf=conf)

# Создание RDD
data = sc.parallelize([1, 2, 3, 4, 5])

# RDD из файла в файловой системе
data2 = sc.textFile("hdfs:///data/file.txt")

# SparkContext преобразует это в:
# - Разбиение данных на партиции
# - Распределение партиций по исполнителям
# - Планирование выполнения операций
result = data.map(lambda x: x * 2).collect()
```

В настоящее время рекомендуется в качестве точки подключения приложения к Spark используется не отдельный контекст, а контейнер контекстов, называемый сессией Spark.

SparkSession — это унифицированный интерфейс, появившийся в версии 2.0. Он заменяет собой необходимость создавать отдельные контексты для разных API. SparkSession объединяет в себе:

- SparkContext (для RDD и низкоуровневых операций);
- SQLContext (для работы с SQL и DataFrames);
- HiveContext (для работы с Hive);
- StreamingContext (для стриминговых операций).

Многие методы отдельных контекстов перенесены непосредственно в объект сессии, например методы работы с DataFrame.

```
from pyspark.sql import SparkSession # Обратить внимание: pyspark.sql
spark = SparkSession.builder.appName("MyApp").master("spark://master-node:7077") \
    .getOrCreate()
```

```
# SparkContext доступен через сессию
sc = spark.sparkContext
```

```
# Но для работы с DataFrame лучше использовать саму сессию
df = spark.read.csv("data.csv")
```

Сравнение: создание контекстов и создание сессии

```
# Создаём несколько контекстов
from pyspark import SparkContext
from pyspark.sql import SQLContext

# Разные объекты для разных задач
sc = SparkContext(conf=conf)
sqlContext = SQLContext(sc)
hiveContext = HiveContext(sc)
rdd = sc.parallelize([1, 2, 3])
df = sqlContext.read.json("data.json")
```

```
# Создаём сессию
from pyspark.sql import SparkSession

# Один объект для всего
spark = SparkSession.builder\
    .getOrCreate()
rdd = spark.sparkContext\
    .parallelize([1, 2, 3])
df = spark.read.json("data.json")
df_sql = spark.sql("SELECT * FROM table")
```


Вопросы для конспекта

21. Из каких компонент состоит приложение Apache Spark? Опишите их предназначение и функции.
22. Как запускается приложение Spark? Какие режимы запуска существуют?
23. Опишите сценарий запуска приложения Spark.
24. Что такое SparkContext? Для чего нужен, как представлен в приложении?
25. Что такое SparkSession? В чём преимущество SparkSession по сравнению со SparkContext?

Вопросы для конспекта

- 1 Что такое Apache Spark? Для каких видов рабочих нагрузок он предназначен?
- 2 За счёт чего обеспечивается быстроедействие Spark-приложений?
- 3 Какие существуют варианты развёртывания кластера Apache Spark?
- 4 Какие языки программирования поддерживает Apache Spark?
- 5 За счёт чего обеспечивается унификация и интеграция исполнения задач на платформе Apache Spark?
- 6 Опишите состав стека Apache Spark.
- 7 Какие диспетчеры кластера поддерживаются в Apache Spark?
- 8 Расскажите про диспетчер кластера Spark Standalone. В чём его преимущества и недостатки?
- 9 Приведите примеры сценариев использования Apache Spark с разными диспетчерами кластера.
- 10 Расскажите про механизмы хранения данных в Apache Spark.
- 11 Какие функции выполняет ядро Spark и вычислительный движок?
- 12 Перечислите и опишите высокоуровневые библиотеки Apache Spark.
- 13 Расскажите про многоуровневый API для работы с данными Apache Spark.
- 14 Что такое RDD в Apache Spark? Перечислите и опишите основные характеристики.
- 15 Сравните подход к работе с данными в Apache Spark на основе RDD и универсальный подход, реализуемый в Hadoop.
- 16 Опишите DataFrame в Apache Spark. За счёт чего операции с ним будут выполняться быстрее, чем с RDD?
- 17 Опишите DataSet в Apache Spark. Сравните его с DataFrame.
- 18 Как выполняется разделение данных между узлами кластера в Apache Spark?
- 19 Из каких узлов состоит кластер Apache Spark под управлением Spark Standalone?
- 20 Из каких узлов состоит кластер Apache Spark под управлением YARN?

- 21 Из каких компонент состоит приложение Apache Spark? Опишите их предназначение и функции.
- 22 Как запускается приложение Spark? Какие режимы запуска существуют?
- 23 Опишите сценарий запуска приложения Spark.
- 24 Что такое SparkContext? Для чего нужен, как представлен в приложении?
- 25 Что такое SparkSession? В чём преимущество SparkSession по сравнению со SparkContext?

Лекция 11. Работа с данными в Spark с использованием RDD

Общие сведения о RDD

- **RDD** (Resilient Distributed Datasets, устойчивые распределённые наборы данных) — основная структура данных в Spark.

Зачем изучать RDD, если есть высокоуровневые DataFrames и DataSets?

- **Фундаментальное понимание:** RDD — основа Spark. DataFrames и DataSet — это RDD с дополнительными качествами.
- **Гибкость:** Позволяют выполнять операции, которые сложно или невозможно выразить в декларативном стиле DataFrames.
- **Низкоуровневый контроль:** Использование RDD позволяет максимально контролировать расположение данных и вычисления.

Характеристики RDD

- **Неизменяемость:** RDD нельзя изменить после создания. Можно только создать новое RDD при помощи преобразований.
- **Распределённость:** Данные разбиты на партии, которые располагаются на разных узлах кластера.
- **Хранение в ОЗУ:** Партии RDD — это не фрагменты файлов, на разных узлах как в HDFS. Партии одного объекта RDD расположены в оперативной памяти разных узлов кластера.
- **Отказоустойчивость:** RDD помнит линию наследования — последовательность преобразований, которые были применены для его получения. Если партия на одном узле потеряна, Spark может пересчитать её заново по этой линии наследования, а не восстанавливать с бэкапа в долговременной памяти.
- **Ленивые вычисления:** Преобразования над RDD не выполняются немедленно. Spark строит план вычислений (DAG — Directed Acyclic Graph). Вычисления запускаются только при выполнении операций, называемых действиями.

RDD — это максимально общий формат, который может содержать произвольные данные.

RDD могут содержать объекты любого типа на Python, Java или Scala, включая экземпляры пользовательских классов.

Главное и практически единственное требование к RDD — делимость.

При создании RDD задаётся разделение данных на партии так, чтобы каждая партия могла быть обработана на отдельном узле кластера.

Разделение прозрачно для пользователя Spark, который работает с RDD как с единым целым.

Вопросы для конспекта

1. Что собой представляет Spark RDD?
2. Объясните, почему важно понимать принципы работы со Spark RDD даже несмотря на наличие высокоуровневых структур данных DataFrame и DataSet.
3. Опишите основные характеристики Spark RDD.
4. Какой главное требование предъявляется к данным, загружаемым в Spark RDD?

Принципы работы с RDD

RDD может быть создан двумя способами:

- при конвертировании «обычных» коллекций данных в программе-драйвере — метод `.parallelize`
- при загрузке данных из внешних источников, например метод `.textFile` и аналогичные для других типов

Пример создания RDD:

```
# Создаём сессию
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("BigD-Course-App") \
    .master("spark://spark-master:7077") \
    .getOrCreate()

# Получаем контекст
sc = spark.sparkContext

# Конвертирование коллекции данных
data = range(1000000)
rdd2 = sc.parallelize(data)

# Загрузка из файла
rdd1 = sc.textFile('/opt/spark/data/words.txt')
```

Создание RDD из обычного массива методом `.parallelize(data)` обычно применяют для тестирования, обучения и проверки идей.

Стандартный способ — загрузка данных из внешнего источника.

Основные источники данных:

- файлы и файловые системы;
- структурированные данные в файлах через Spark SQL (доступ к Hive или полностью структурированным JSON);
- базы данных, например MySQL, PostgreSQL или MongoDB.

При необходимости можно сохранить RDD в долговременную память (на диск). При этом создаётся каталог, в который каждая партиция записывается в виде отдельного файла.

Загрузка данных и сохранение RDD производятся в файловую систему кластера (см. далее).

Вся работа с RDD производится в оперативной памяти.

После того, как RDD создан, над ними можно совершать операции двух видов: преобразования (transformations) и действия (actions).

Преобразования — операции над RDD в результате выполнения которых создаются новые RDD на основе существующих. Пример преобразования — фильтрация по заданному критерию.

```
rdd1 = sc.textFile('/opt/spark/data/words.txt')  
rdd2 = rdd1.filter(lambda line: line[:3] == "adv")
```

При выполнении инструкции кода с операцией преобразования фактически обработка данных не производится. Вместо этого операция записывается в DAG (Directed Acyclic Graph) операций. По мере накопления операций производится их оптимизация.

Действия — вычисляют результат, не создавая новых RDD, и возвращают его программе-драйверу или сохраняют во внешнем хранилище (например, в HDFS). Пример действия — метод возвращающий первый элемент набора `.first()`

```
rdd1 = sc.textFile('/opt/spark/data/words.txt')  
rdd2 = rdd1.filter(lambda line: line[:3] == "adv")  
res = rdd2.first()
```

После исполнения этого примера обычная строчная переменная `res` будет хранить первую строку набора `rdd2`. Например, это может быть слово «adventure».

При выполнении инструкции кода с операцией действия происходит фактическое исполнение операций, собранных в DAG.

Вычисления только в момент обращения к результату называются ленивыми или отложенными вычислениями.

Жадные или немедленные вычисления кода из примера предполагают, что сначала в оперативную память будет загружен файл «words.txt», затем в памяти будет создана новая структура, в которую будут помещены отфильтрованные данные и далее из них будет выбран только первый элемент. Остальные данные не нужны.

Такое вычисление приведёт к значительному расходу оперативной памяти. Если файл большой, он просто не уместится в памяти целиком.

Вместо этого Spark соберёт операции преобразований в DAG и выполнив оптимизацию «увидит», что достаточно просканировать файл и вернуть первое найденное слово, начинающееся на «adv».

Вся цепочка операций, собранная в DAG выполняется всякий раз заново при выполнении очередного действия.

```
rdd1 = sc.textFile('/opt/spark/data/words.txt')
rdd2 = rdd1.filter(lambda line: line[:3] == "adv")
res = rdd2.first()
cnt = rdd2.count()
```

В этом примере весь DAG операций (начиная с чтения файла) будет выполнен дважды — первый раз чтобы выполнить действие `.first` и второй раз для подсчёта числа элементов методом `.count`.

Если предполагается использовать один и тот же RDD для выполнения нескольких действий, то можно потребовать от Spark сохранить (кешировать) его вызовом метода `.persist()`. Кеширование обычно выполняется в оперативной памяти но можно указать, что при нехватки места в ОЗУ данные нужно записать на диск (уровни сохранения задаются аргументом метода `.persist()`).

После вычисления набора RDD в первый раз Spark сохранит его содержимое в памяти или на диске и будет использовать при выполнении последующих действий.

```
rdd1 = sc.textFile('/opt/spark/data/words.txt')
rdd2 = rdd1.filter(lambda line: line[:3] == "adv")
rdd2.persist()
res = rdd2.first()
cnt = rdd2.count()
```

Общая схема работы в Spark с данными на основе RDD:

- 1 Создаются некоторые исходные наборы RDD из внешних данных.
- 2 На их основе создаются новые наборы с применением преобразований, таких как `.filter()`
- 3 Для любых промежуточных наборов, которые потребуются впоследствии, вызывается метод `.persist()`.
- 4 Запускаются действия, такие как `.count()` и `.first()`, которые оптимизируются и выполняются фреймворком Spark параллельно на нескольких компьютерах в кластере.

Вопросы для конспекта

5. Каким способами можно создавать Spark RDD? Приведите примеры.
6. Какие основные источники данных используются в Spark?
7. Операции каких типов применяются к Spark RDD? В чём их принципиальное отличие?
8. Что такое ленивые вычисления? Как ленивые вычисления используются в Spark?
9. Что такое DAG? Каким образом он создаётся?
10. Как Spark производит вычисления, если после необходимых преобразований выполняются несколько действий над одним и тем же RDD? Как можно повысить эффективность этих вычислений?

Общие правила загрузки RDD из файлов и сохранения в файлы

Простой пример загрузки данных в RDD из текстового файла:

```
rdd = sc.textFile('/opt/spark/data/words.txt')
```

При загрузке внешних данных из файла указывается путь к файлу, размещённому в файловой системе кластера (не в локальной файловой системе узла, через который идёт работа!).

Если Spark работает поверх Hadoop, то аргументом команды `sc.textFile('path/file.txt')` должен быть путь к файлу внутри HDFS.

У собственного диспетчера кластера Spark Standalone нет собственной распределённой файловой системы и обмен данными производится через общие сетевые папки.

Примечание

Общие («расшаренные», от share) папки — один из узлов сети даёт возможность другим узлам подмонтировать папку в свои файловые системы.

Под Windows протокол создания общих папок называется SMB (Server Message Block). Ранняя версия SMB также называлась CIFS (Common Internet File System).

Под Linux нативный протокол называется NFS (Network File System). Но также может использоваться SMB.

В кластере Spark Standalone аргумент `sc.textFile('path/file.txt')` — это путь к файлу в общей сетевой папке, доступной всем узлам кластера.

При создании RDD данные разбиваются на партии, чтобы их можно было обрабатывать на разных узлах кластера.

Число партий выбирается автоматически но может быть задано и скорректировано после создания вручную.

Есть много факторов, влияющих на то, сколько Spark выберет партий.

В самом простом случае число партий в автоматическом режиме будет выбрано равным суммарному количеству вычислительных ядер на рабочих узлах (workers).

Например, если в кластере 3 рабочих узла и каждый имеет двухядерный процессор, то число партий будет задано равным 6.

Нужно понимать, что деление на партии соответствуют распределению этих партий между рабочими узлами.

Поэтому если в RDD загружается файл, который разделён на фрагменты и они хранятся на разных узлах кластера, например файл из HDFS, то число партиций будет равно числу имеющихся фрагментов файла.

Spark ориентирован на работу с большими данными. Поэтому при чтении файла драйвер не читает данные непосредственно, а создаёт план чтения:

- анализирует метаданные файла (размер, блоки);
- создает план разделения на партиции;
- каждой партиции сопоставляет физическое расположение данных

План чтения передаётся исполнителям.

- Каждый исполнитель получает задание прочитывать определённые блоки файла.
- Исполнитель подключается напрямую к узлам данных (если это HDFS) или к общей папке.
- Каждый исполнитель напрямую считывает собственную часть файла и формирует партиции для дальнейшей обработки.

Вопросы для конспекта

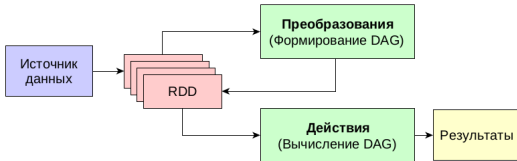
11. Где должен быть расположен файл, чтобы его можно было загрузить в Spark RDD?
12. Для чего выполняется партиционирование Spark RDD?
13. Предположим, что Spark RDD создаётся самым простым способом, на основе некоторого числового массива, предварительно вычисленного средствами Python? Что нужно знать, что бы найти число создаваемых Spark партиций для такого RDD?
14. Предположим, что данные в Spark RDD загружаются из файла, сохранённого в HDFS. Сколько будет создано партиций для такого RDD?

Выполнение DAG. Стадии и задачи

Обработка RDD в Spark производится на основе плана в виде DAG (Directed Acyclic Graph). Выполнение DAG состоит из нескольких преобразований, которые заканчиваются действием.

- **Преобразования (transformations)** — это операции над RDD в DAG в результате которых создаются новые RDD. Эти операции не выполняются немедленно при исполнении пользовательского кода, а записываются в DAG.

- **Действия (actions)** — это операции над RDD, которые вычисляют результат, не создавая новых RDD. Результат возвращается программе-драйверу в виде обычного, не распределённого набора данных или сохраняется в файл.

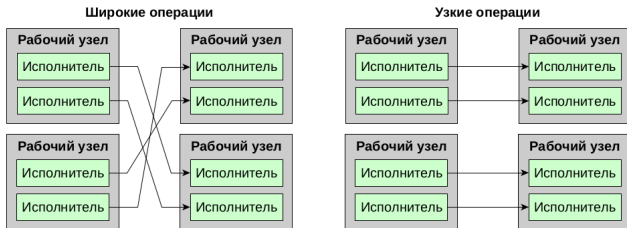


DAG создается для каждого действия отдельно. Каждое действие запускает создание и выполнение своего собственного DAG, который включает все преобразования, необходимые для вычисления этого конкретного действия.

С точки зрения вычислительной эффективности преобразования RDD делятся на перемешивающие (shuffle) и не перемешивающие.

- **Перемешивающие (shuffle) преобразования** — преобразования, которые требуют перераспределения данных между партициями. Это очень ресурсоёмкие операции.
 - При выполнении такой операции исполнители сначала записывают свои данные в локальные файлы — замедление из-за дискового ввода-вывода.
 - Затем данные передаются по сети между узлами — замедление из-за дискового ввода-вывода.
- **Узкие преобразования (narrow transformations)** — преобразования, которые не содержат перемешиваний.

- **Широкие преобразования (wide transformations)** — преобразования, в которых задействованы перемешивания (shuffle).



Выполнение DAG разделено на стадии, которые включают в себя исполнение задач.

- **Задача (task)** — единица выполнения кода в Spark. Каждая задача
 - представляет собой последовательность узких преобразований;
 - выполняется на одном ядре процессора;
 - обрабатывает одну партицию данных;
 - запускается на одном исполнителе.

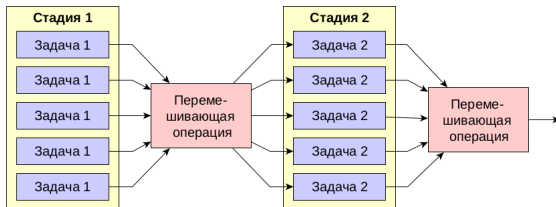
Характеристики задач:

- Одинаковый код: Все задачи в одной стадии выполняют одинаковый код.
 - Разные данные: Каждая задача работает со своей партицией данных.
 - Параллельное выполнение: Задачи выполняются параллельно на разных исполнителях.
- **Стадия (stage)** — это набор задач, которые могут выполняться параллельно без обмена данными между исполнителями.

Стадия — это последовательность узких преобразований. Из них формируется код задачи и выполнение стадии сводится к параллельному исполнению этого кода на каждой из партиций.

Границы стадий — это перемешивающие операции (shuffle) или действия. Иными словами, каждая операция, требующая передачи данных между партициями, создаёт границу между стадиями.

Стадии выполняются последовательно, одна за другой.



Стадии бывают двух типов:

- **Промежуточные (Shuffle Map Stage).** Это стадии всегда предшествуют перемешивающим операциям. Они подготавливают данные для перемешивающей операции, записывают их в файлы, которые будут прочитаны следующей стадией.
- **Финальная (Result Stage).** Завершает выполнение DAG. Не имеет последующих стадий. Эта стадия вычисляет данные, которые затем будут использованы операцией типа «действие».

Вопросы для конспекта

15. Что собой представляет операция над RDD типа «преобразование» в Spark?
16. Что собой представляет операция над RDD типа «действие» в Spark?
17. Какие операции входят в DAG, какой операцией заканчивается?
18. Чем широкие преобразования RDD в Spark отличаются от узких? Какие выполняются медленнее и почему?
19. Что такое задачи в Spark?
20. Что такое стадии в Spark? Из чего состоят? Что является границей стадии?
21. Какие существуют типы стадий в Spark?

Механизм взаимодействия драйвера и исполнителей

Для обеспечения работы Spark-приложения драйвер и исполнители должны обмениваться информацией друг с другом.

Существует два типа обмена: между драйвером и исполнителями и между исполнителями.

• Ситуации, когда производится обмен информацией драйвер-исполнители:

- Передача от драйвера исполнителям кода задачи.
- Передача от драйвера исполнителям данных, которые нужны всем (например, команды управления, планы чтения файлов данных). (Вспомним: драйвер не пересылает непосредственно обрабатываемые данные, исполнители сами читают свои фрагменты в соответствии с полученным планом.)
- Пересылка значений распределённых переменных. (Вспомним: распределённые переменные играют роль глобальных переменных, доступных одновременно всем исполнителям.)
- Возврат от исполнителей результатов.

Всё это взаимодействие происходит посредством обмена сетевыми сообщениями. Промежуточные файлы почти никогда не используются. Исключение — временная буферизация данных большого объёма.

- **Обмен информацией между исполнителями** производится когда выполняются перемешивающие (shuffle) операции.

Это делается при помощи общих (доступных всем узлам) временных файлов.

Причина, по которой перемешивание выполняется при помощи файлов в том, что Spark работает с большими данными и поэтому типичный сценарий — в таких операциях задействован большой объём данных.

Вопросы для конспекта

22. В каких случаях драйвер и исполнители обмениваются информацией? Посредством чего происходит передача?
23. В каких случаях исполнители обмениваются информацией друг с другом? Каким образом это производится?

Пример Spark-приложения и визуализация DAG

За выполнением Spark-приложения можно следить при помощи веб-интерфейса.

- **<IP узла драйвера>:4040** — веб-интерфейс драйвера (Spark Application UI). Доступен от момента создания сессии Spark и до её остановки командой `sc.stop()`. Предоставляет визуализацию DAG'а в целом, его фрагментов для каждой стадии, состояния выполняемых и выполненных задач.
- **<IP сервера истории>:18080** — веб-интерфейс сервера истории (History Server UI). Сюда переносится информация из UI драйвера когда сессия закончена. Также позволяет видеть состояние текущей, незавершённой сессии.
- **<IP мастера>:8080** — веб-интерфейс мастера (Standalone Master UI). Представляет логи исполнителей, работающих на рабочих узлах.
- **<IP рабочего узла>:8081** — веб-интерфейс рабочего узла (Standalone Worker UI). Представляет более детальное описание состояния исполнителей на данном узле.

Напишем Spark-приложение, которое загружает три текстовых файла (приведены ниже) и подсчитывает в них количество длинных строк, количество коротких строк, количество длинных слов.

В этом приложении стадии зачисляются перемешивающими (широкими) операциями `reduceByKey` и `groupByKey`.

Файл «Пушкин.txt»

Любви, надежды, тихой
славы Недолго нежил
нас обман, Исчезли
юные забавы, Как сон,
как утренний туман; Но
в нас горит еще желанье,
Под гнетом власти
роковой Нетерпеливою
душой Отчизны внемлем
призыванье. Мы ждем с
томленьем упованья
Минуты вольности
святой, Как ждет
любовник молодой
Минуты верного
свиданья.

Файл «Есенин.txt»

Вы помните, Вы всё,
конечно, помните, Как я
стоял, Приблизившись к
стене, Взволнованно
ходили вы по комнате И
что-то резкое В лицо
бросали мне. Вы
говорили: Нам пора
расстаться, Что вас
измучила Моя шальная
жизнь, Что вам пора за
дело приниматься, А
мой удел — Катиться
дальше, вниз.

Файл «Тютчев.txt»

Есть в осени
первоначальной
Короткая, но дивная
пора — Весь день стоит
как бы хрустальный, И
лучезарны вечера. . . Где
бодрый серп гулял и
падал колос, Теперь уж
пусто все — простор
езде, — Лишь паутины
тонкий волос Блестит на
праздной борозде. . .
Пустеет воздух, птиц не
слышно боле, Но далеко
еще до первых зимних
бурь — И льется чистая
и теплая лазурь На
отдыхающее поле. . .

```

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("BigD-Course-Poems") \
    .master("spark://spark-master:7077").getOrCreate()
sc = spark.sparkContext

# Читаем три текстовых файла в отдельные RDD
pushkin_rdd = sc.textFile("/opt/spark/data/Пушкин.txt")
esenin_rdd = sc.textFile("/opt/spark/data/Есенин.txt")
tyutchev_rdd = sc.textFile("/opt/spark/data/Тютчев.txt")

# Добавляем метку поэта к каждой строке
pushkin_labeled = pushkin_rdd.map(lambda line: ("Пушкин", line))
esenin_labeled = esenin_rdd.map(lambda line: ("Есенин", line))
tyutchev_labeled = tyutchev_rdd.map(lambda line: ("Тютчев", line))

# Объединяем все данные
all_poems_rdd = pushkin_labeled.union(esenin_labeled.union(tyutchev_labeled))

# ВЕТВЬ 1: Анализ длинных строк
long_lines_analysis = (all_poems_rdd
    .filter(lambda x: len(x[1]) > 30) # Длинные строки
    .map(lambda x: (x[0], 1)) # Преобразуем в пары (поэт, 1)
    .reduceByKey(lambda a, b: a + b)) # Считаем количество длинных строк

# long_lines_analysis=[('Тютчев', 6), ('Есенин', 2)]

```

ВЕТВЬ 2: Анализ коротких строк

```
short_lines_analysis = (all_poems_rdd
    .filter(lambda x: len(x[1]) < 20)    # Короткие строки
    .map(lambda x: (x[0], 1))            # Преобразуем в пары (поэт, 1)
    .reduceByKey(lambda a, b: a + b))    # Считаем количество коротких строк
```

```
# short_lines_analysis=[('Тютчев', 2), ('Есенин', 8), ('Пушкин', 1)]
```

ВЕТВЬ 3: Анализ слов

```
words_analysis = (all_poems_rdd
    .flatMap(
        lambda x: [(x[0], word) for word in x[1].split()]) # Разбиваем на слова
    .filter(lambda x: len(x[1]) > 5)    # Слова длиннее 5 символов
    .map(lambda x: (x[0], 1))          # Преобразуем в пары (поэт, 1)
    .reduceByKey(lambda a, b: a + b))  # Считаем количество длинных слов
```

```
# words_analysis=[('Тютчев', 27), ('Есенин', 20), ('Пушкин', 26)]
```

ОБЪЕДИНЕНИЕ: Собираем все результаты вместе

Сначала преобразуем все RDD к одинаковому формату

```
long_lines_formatted = long_lines_analysis.map(
    lambda x: (x[0], ("long_lines", x[1])))
```

```
# long_lines_formatted=[('Тютчев', ('long_lines', 6)),
#    ('Есенин', ('long_lines', 2))]
```



```
short_lines_formatted = short_lines_analysis.map(
    lambda x: (x[0], ("short_lines", x[1])))

# short_lines_formatted=[('Тютчев', ('short_lines', 2)),
# ('Есенин', ('short_lines', 8)), ('Пушкин', ('short_lines', 1))]

words_formatted = words_analysis.map(
    lambda x: (x[0], ("long_words", x[1])))

# words_formatted=[('Тютчев', ('long_words', 27)),
# ('Есенин', ('long_words', 20)), ('Пушкин', ('long_words', 26))]

# Объединяем все результаты
all_results = long_lines_formatted.union(
    short_lines_formatted).union(words_formatted)

# Группируем по поэту для финального вывода
final_results = all_results.groupByKey().mapValues(list)

# Запускаем вычисление и выводим результат
results = final_results.collect()
for poet, stats in results:
    print(f"{poet}: {stats}")

# Останавливаем SparkContext
sc.stop()
```

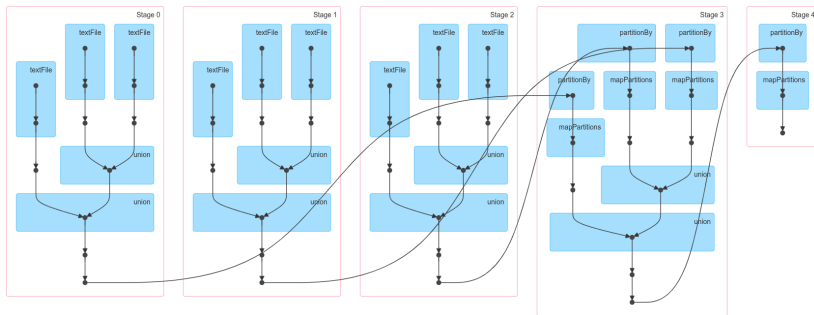
Результат:

Есенин: [('long_lines', 2), ('long_words', 20), ('short_lines', 8)]

Тютчев: [('short_lines', 2), ('long_lines', 6), ('long_words', 27)]

Пушкин: [('short_lines', 1), ('long_words', 26)]

DAG этого приложения состоит из 4 стадий.



В этом приложении `all_poems_rdd` используется три раза. Видно, что при выполнении каждой ветви этот RDD перевычисляется.

Для повышения эффективности имеет смысл его закешировать.

...

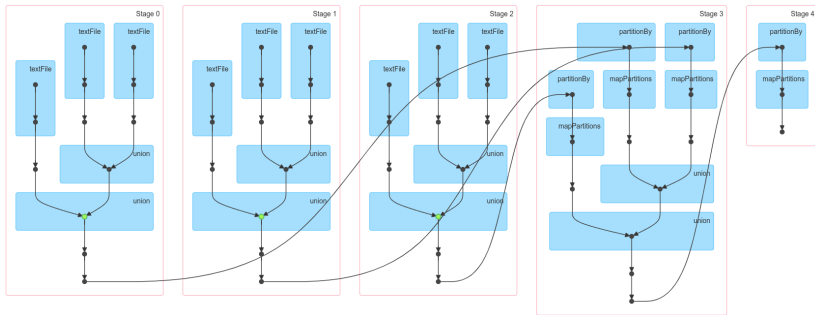
Объединяем все данные

```
all_poems_rdd = pushkin_labeled.union(esenin_labeled.union(tyutchev_labeled))
```

Сохраняем этот RDD в памяти

```
all_poems_rdd.persist()
```

...



Новый DAG устроен точно также, но точка кеширования выделена зелёным. Теперь `all_poems_rdd` вычисляется только один раз.

Вопросы для конспекта

24. На каких портах предоставляют веб-интерфейс компоненты кластера Spark? Какую информацию можно получить?

Вопросы для конспекта

- 1 Что собой представляет Spark RDD?
- 2 Объясните, почему важно понимать принципы работы со Spark RDD даже несмотря на наличие высокоуровневых структур данных DataFrame и DataSet.
- 3 Опишите основные характеристики Spark RDD.
- 4 Какое главное требование предъявляется к данным, загружаемым в Spark RDD?
- 5 Какими способами можно создавать Spark RDD? Приведите примеры.
- 6 Какие основные источники данных используются в Spark?
- 7 Операции каких типов применяются к Spark RDD? В чём их принципиальное отличие?
- 8 Что такое ленивые вычисления? Как ленивые вычисления используются в Spark?
- 9 Что такое DAG? Каким образом он создаётся?
- 10 Как Spark производит вычисления, если после необходимых преобразований выполняются несколько действий над одним и тем же RDD? Как можно повысить эффективность этих вычислений?
- 11 Где должен быть расположен файл, чтобы его можно было загрузить в Spark RDD?
- 12 Для чего выполняется партиционирование Spark RDD?
- 13 Предположим, что Spark RDD создаётся самым простым способом, на основе некоторого числового массива, предварительно вычисленного средствами Python? Что нужно знать, чтобы найти число создаваемых Spark партиций для такого RDD?
- 14 Предположим, что данные в Spark RDD загружаются из файла, сохранённого в HDFS. Сколько будет создано партиций для такого RDD?
- 15 Что собой представляет операция над RDD типа «преобразование» в Spark?
- 16 Что собой представляет операция над RDD типа «действие» в Spark?
- 17 Какие операции входят в DAG, какой операцией заканчивается?

- 18 Чем широкие преобразования RDD в Spark отличаются от узких? Какие выполняются медленнее и почему?
- 19 Что такое задачи в Spark?
- 20 Что такое стадии в Spark? Из чего состоят? Что является границей стадии?
- 21 Какие существуют типы стадий в Spark?
- 22 В каких случаях драйвер и исполнители обмениваются информацией? Посредством чего происходит передача?
- 23 В каких случаях исполнители обмениваются информацией друг с другом? Каким образом это производится?
- 24 На каких портах предоставляют веб-интерфейс компоненты кластера Spark? Какую информацию можно получить?

Лекция 12. Операции над RDD

Загрузка и чтение текстовых файлов TXT

Текстовый файл в формате TXT не имеет внутренней структуры за исключением деления на строки.

Загрузка из текстового файла выполняется методом `.textFile()`:

```
# Загрузить один файл  
rdd = sc.textFile('/opt/spark/data/words.txt')
```

Обратить внимание: это операция относится к типу преобразований и не выполняется сразу же. Если указанный файл не существует, то сообщение об ошибке появится только при выполнении операции типа действия над данными из этого RDD.

При чтении текстового файла через `.textFile()` партиционирование делается по строкам. Слова разорваны не будут.

Можно собрать в один RDD несколько текстовых файлов. Пути к ним нужно записать в строке через запятую:

```
# Загрузить несколько файлов в один RDD  
rdd = sc.textFile('/opt/spark/data/files/w1.txt,/opt/spark/data/files/w2.txt')
```

Можно указать путь к папке с текстовыми файлами:

```
# Загрузить все файлы из папки  
rdd = sc.textFile('/opt/spark/data/files')
```

Если нужно сохранить информацию о распределении текстов по файлам, можно использовать метод `wholeTextFiles()`. Этот метод возвращает RDD в виде набора пар ключ-значение, где ключи — это имена файлов, а значения — это содержание файлов.

```
# Загрузить все файлы из папки в виде пар ключ-значение  
rdd = sc.wholeTextFiles('/opt/spark/data/files')
```

Обратить внимание: между партициями распределяются файлы целиком, каждый файл целиком принадлежит только одной партиции.

Чтобы в это убедиться, выполним действие `.collect()`. Этот метод соберёт все партиции вместе и возвратит их как объект Python:

```
print(rdd.collect())  
[('file:/opt/spark/data/files/w3.txt', 'whale\nred\nwhale\nwhale\n'),  
 ('file:/opt/spark/data/files/w2.txt', 'adventure\nred\nwhale\nwhale\n\n'),  
 ('file:/opt/spark/data/files/w1.txt', 'whale\nsea\nwhale\nwhale\nsea\nsea\n\n\n')]
```

Видно, что одна запись RDD — это пара путь-текст (ключ-значение). Деление на партии не разрывает записи и каждая запись оказывается в одной партии целиком.

Очевидно, что такой метод загрузки подходит, когда нужно обработать большое число небольших файлов и при этом каждый файл требуется обрабатывать как одно целое, без деления на партии.

Для сохранения RDD в виде текстового файла используется метод `.saveAsTextFile()`. Аргумент интерпретируется как путь к папке, в которой будут сохранены партии RDD.

```
# Сохранить RDD в текстовые файлы  
rdd.saveAsTextFile('/opt/spark/data/my_rdd')
```

После выполнения этой команды создаётся папка `my_rdd`, в которой будут размещены файлы:

```
part-00000  
.part-00000.crc  
part-00001  
.part-00001.crc  
part-00002  
.part-00002.crc  
_SUCCESS  
._SUCCESS.crc
```

Файлы `part-*` — это текстовые файлы, содержание которых повторяют содержание исходных файлов.

Эти файлы создают рабочие узлы — каждый сохраняет в указанную папку данные своих партиций RDD.

Файлы начинающиеся с точки и с расширением CRC, `.*.crc`, — это скрытые файлы контрольных сумм.

Файл `_SUCCESS` — это пустой файл индикатор успешного завершения операции.

Если RDD был создан не из текстовых данных, то при выполнении `.saveAsTextFile()` будут созданы текстовые файлы, содержащие текстовое представление элементов RDD.

Если папка, в которую сохраняем, в примере это `my_rdd`, уже существует, то его содержание не переписывается и операция завершается с ошибкой.

Если в виде текста сохранить RDD, созданный при помощи `sc.wholeTextFiles()`, содержимым файлов `part-*` будут пары ключ-значение:

`part-00000:`

```
('file:/opt/spark/data/files/w3.txt', 'whale\nred\nwhale\nwhale\n')  
( 'file:/opt/spark/data/files/w2.txt', 'adventure\nred\nwhale\nwhale\n\n')
```

`part-00001:`

```
('file:/opt/spark/data/files/w1.txt', 'whale\nsea\nwhale\nwhale\nsea\nsea\n\n')
```

Вопросы для конспекта

1. Если при загрузке в Spark RDD данных из файла в отвечающий за это метод передать неверные путь или имя файла, то метод выполнится без ошибок. Почему? В какой момент вычисления будут прерваны с сообщением «Файл не найден»?
2. В каких местах внутри текстового файла Spark может, а в каких не может разделять его на партии при загрузке в RDD?
3. Какой метод отвечает за загрузку текстового файла в Spark RDD? Как с его помощью загрузить сразу несколько файлов? Как загрузить содержимое целого каталога?
4. Как в Spark RDD загрузить несколько текстовых файлов с сохранением путей к ним?
5. Как загрузить в Spark RDD много небольших текстовых файлов так, чтобы обрабатывать каждый отдельный файл как единое целое, т. е. с гарантией, что части одного файла не окажутся в разных партициях?

6. Какие объекты файловой системы создаёт Spark при сохранении RDD в виде текста?

Загрузка и чтение файлов JSON и CSV

Для загрузки данных в формате JSON нужно сначала загрузить их как текстовый файл, а выполнить преобразование при помощи парсера JSON.

```
# Импортировать библиотеку работы с JSON
import json
# Загрузить файл как текст
rdd_text = sc.textFile('/opt/spark/data/mtcars.json')
# Выполнить парсинг
rdd = rdd_text.map(lambda x: json.loads(x))
```

В результате чтения JSON-документа как текста получается коллекция строк:

```
print(rdd_text.collect())

['{"model": "Mazda RX4", "mpg": 21, "cyl": 6, "disp": 160, "hp": 110, "drat": 3.9,
  "wt": 2.62, "qsec": 16.46, "vs": 0, "am": 1, "gear": 4, "carb": 4}',
 '{"model": "Mazda RX4 Wag", "mpg": 21, "cyl": 6, "disp": 160, "hp": 110, "drat": 3.9,
  "wt": 2.875, "qsec": 17.02, "vs": 0, "am": 1, "gear": 4, "carb": 4}',
 '{"model": "Datsun 710", "mpg": 22.8, "cyl": 4, "disp": 108, "hp": 93, "drat": 3.85,
  "wt": 2.32, "qsec": 18.61, "vs": 1, "am": 1, "gear": 4, "carb": 1}',
 ...]
```

После парсинга получаем RDD как коллекцию словарей

```
print(rdd.collect())
```

```
[{'model': 'Mazda RX4', 'mpg': 21, 'cyl': 6, 'disp': 160, 'hp': 110, 'drat': 3.9,
  'wt': 2.62, 'qsec': 16.46, 'vs': 0, 'am': 1, 'gear': 4, 'carb': 4},
 {'model': 'Mazda RX4 Wag', 'mpg': 21, 'cyl': 6, 'disp': 160, 'hp': 110, 'drat': 3.9,
  'wt': 2.875, 'qsec': 17.02, 'vs': 0, 'am': 1, 'gear': 4, 'carb': 4},
 {'model': 'Datsun 710', 'mpg': 22.8, 'cyl': 4, 'disp': 108, 'hp': 93, 'drat': 3.85,
  'wt': 2.32, 'qsec': 18.61, 'vs': 1, 'am': 1, 'gear': 4, 'carb': 1},
 ...]
```

При записи сначала JSON конвертируется в текстовые строки а затем записывается как текстовый файл.

```
# Преобразуем в текст
```

```
rdd_text = rdd.map(lambda x: json.dumps(x))
```

```
# Пишем в файл
```

```
rdd_text.saveAsTextFile('/opt/spark/data/my_json')
```

В папке my_json появятся файлы part-* со следующим содержимым:

```
{"model": "Mazda RX4", "mpg": 21, "cyl": 6, "disp": 160, "hp": 110, ...}
{"model": "Mazda RX4 Wag", "mpg": 21, "cyl": 6, "disp": 160, "hp": 110, ...}
{"model": "Datsun 710", "mpg": 22.8, "cyl": 4, "disp": 108, "hp": 93, ...}
...
```

Идея работы с CSV такая же: сначала читаем как текстовый файл, затем выполняем парсинг.

Самый простой вариант — выполнить сплит по разделителю:

```
# Загрузить файл как текст и выполнить сплит по разделителю  
rdd = sc.textFile('/opt/spark/data/happiness_score.csv').map(lambda x: x.split(','))
```

В результате получаем коллекцию списков:

```
print(rdd.collect())  
[['Country', 'Region', 'Happiness Score'],  
 ['Denmark', 'Western Europe', '7.526'],  
 ['Switzerland', 'Western Europe', '7.509'],  
 ['Iceland', 'Western Europe', '7.501'],  
 ...]
```

Обратить внимание: исходный документ CSV содержит строку заголовка и после сплита она стала первым элементом набора.

Для её удаления можно применить фильтрацию:

```
# Читаем как текст  
rdd_text = sc.textFile('/opt/spark/data/happiness_score.csv')  
# Считываем первую строку  
header_string = rdd_text.first()  
# Отфильтровываем её  
rdd_text = rdd_text.filter(lambda x: x != header_string)
```

Далее можно получить RDD в виде коллекции словарей:

```
header = header_string.split(',')  
rdd = rdd_text.map(lambda x: x.split(',')).map(lambda x: dict(zip(header,x)))
```

Вот так буде выглядеть результат:

```
print(rdd.collect())
```

```
[{'Country': 'Denmark', 'Region': 'Western Europe', 'Happiness Score': '7.526'},  
 {'Country': 'Switzerland', 'Region': 'Western Europe', 'Happiness Score': '7.509'},  
 {'Country': 'Iceland', 'Region': 'Western Europe', 'Happiness Score': '7.501'},  
 ...]
```

Такой RDD можно записать как JSON, а можно при сохранении вернуться к CSV формату, превратив предварительно набор словарей в набор строк:

```
rdd_text = rdd.map(lambda d: ','.join([str(x) for x in d.values()]))  
rdd_text.saveAsTextFile('/opt/spark/data/my_csv')
```

В папке my_csv в файлах part-* будут следующие записи:

```
Denmark,Western Europe,7.526  
Switzerland,Western Europe,7.509  
Iceland,Western Europe,7.501  
...
```

Вопросы для конспекта

7. Как выполняется загрузка JSON и CSV файлов в Spark RDD. Приведите пример.
8. Как сохранить текстовый файл в виде JSON или CSV. Приведите пример.
9. Как при загрузке в Spark RDD CSV файла удалить из RDD строку заголовка. Приведите пример.
10. Как загрузить в Spark RDD CSV файл в виде коллекции словарей?

Операции преобразования RDD

- **Фильтрация (filter)** — преобразование, возвращает RDD с элементами, удовлетворяющими заданному критерию. Узкое преобразование.

```
rdd0 = sc.parallelize([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])  
rdd1 = rdd0.filter(lambda x: x % 2) # Отбираем нечётные значения  
print(rdd1.collect())  
[1, 3, 5, 7, 9]
```

- **Отображение (map)** — преобразование, которое применяет заданную функцию к каждому элементу RDD. Новый RDD содержит вычисленные значения функции. Узкое преобразование.

```
rdd0 = sc.parallelize([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])  
rdd1 = rdd0.map(lambda x: x*x) # Возводим каждый элемент в квадрат  
print(rdd1.collect())  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- **Векторное отображение (flatMap)** — преобразование, применяет заданную функцию к элементам RDD, возвращает вместо единственного элемента вектор (итерируемый объект, например, список) и затем сглаживает (flatten) результирующий набор.

Не вызывает перемешивания данных — **узкое преобразование**.

```
# Исходный текст, разделённый на строки (или его можно было загрузить из файла)
raw_text = """Я к вам пишу - чего же боле?
```

```
Что я могу еще сказать?
```

```
Теперь, я знаю, в вашей воле
```

```
Меня презреньем наказать.""".split('\n')
```

```
print(raw_text)
```

```
['Я к вам пишу - чего же боле?', 'Что я могу еще сказать?',  
 'Теперь, я знаю, в вашей воле', 'Меня презреньем наказать.']
```

```
# Создаём RDD - RDD сохраняет деление по строкам
```

```
rdd0 = sc.parallelize(raw_text)
```

```
print(rdd0.collect())
```

```
['Я к вам пишу - чего же боле?', 'Что я могу еще сказать?',  
 'Теперь, я знаю, в вашей воле', 'Меня презреньем наказать.']
```

```
# Токенизация - делим каждую строку на слова по пробелам
```

```
rdd1 = rdd0.flatMap(lambda x: x.split(" "))
```

```
print(rdd1.collect())
```

```
['Я', 'к', 'вам', 'пишу', '-', 'чего', 'же', 'боле?', 'Что', 'я', 'могу', 'еще', 'сказать?',  
 'Теперь,', 'я', 'знаю,', 'в', 'вашей', 'воле', 'Меня', 'презреньем', 'наказать.']
```

• **Семплирование (sample)** — преобразование, возвращает случайную выборку из набора. **Узкое преобразование**. Имеет три аргумента:

- withReplacement — если истина, то выборка с возвратами;
- fraction — примерное значение доли возвращаемых элементов;
- seed — начальное состояние генератора случайных чисел.

```
rdd1 = sc.parallelize(range(10))  
# Выборка без возвратов  
rdd2 = rdd1.sample(False, 0.6); print(rdd2.collect())  
# Выборка с возвратами  
rdd3 = rdd1.sample(True, 0.6); print(rdd3.collect())  
  
[2, 3, 4, 6]  
[0, 2, 2, 5, 6, 6, 8, 8]
```

Обратить внимание: fraction — это вероятность, что данный элемент будет выбран. Поэтому размер выборки будет только примерно равен ($\text{fraction} \times N$). Так сделано, чтобы выборку можно было делать независимо на каждом рабочем узле (узкое преобразование).

Примечание

Представим себе ящик с 10 шарами, на которых написаны номера от 0 до 9.

Выборка без возвратов — после выбора очередного шара он не возвращается в ящик и его номер не может попасть в выборку дважды.

Выборка с возвратами — шар возвращается и поэтому его номер может появиться в выборке несколько раз.

Параметр `seed` — если задан (не равен `None`), то генератор случайных чисел будет выдавать одну и ту же последовательность. Выборки будут получаться одинаковыми.

```
rdd1 = sc.parallelize(range(10))
# seed явно не задан (равен None): при каждом семплировании новая выборка
rdd2 = rdd1.sample(False, 0.5); print(rdd2.collect())
rdd2 = rdd1.sample(False, 0.5); print(rdd2.collect())

[0, 1, 2, 5, 7, 8]
[1, 4, 5, 7, 8]

# При заданном seed выборка всегда одна и таже
rdd2 = rdd1.sample(False, 0.5, 42); print(rdd2.collect())
rdd2 = rdd1.sample(False, 0.5, 42); print(rdd2.collect())

[0, 7]
[0, 7]
```

• **Объединение (`union`)** — преобразование, объединяющее два RDD.

Узкое преобразование.

```
rdd1 = sc.parallelize(['январь', 'февраль', 'март', 'апрель'])
rdd2 = sc.parallelize(['март', 'апрель', 'май', 'июнь'])
# Объединяем - второй RDD добавляется в конец к первому
rdd3 = rdd1.union(rdd2); print(rdd3.collect())

['январь', 'февраль', 'март', 'апрель', 'март', 'апрель', 'май', 'июнь']
```

● **Удаление дубликатов (distinct)** — преобразование, возвращает RDD без повторов. **Широкое преобразование.**

```
# В RDD из примера выше удаляем повторы  
rdd4 = rdd3.distinct(); print(rdd4.collect())  
['июнь', 'апрель', 'январь', 'март', 'февраль', 'май']
```

● **Пересечение (intersection)** — преобразование, возвращающее общие элементы. Повторы удаляются. **Широкое преобразование.**

```
# В этих наборах общие элементы 'март', 'апрель', 'апрель'  
rdd1 = sc.parallelize(['январь', 'февраль', 'март', 'апрель', 'апрель'])  
rdd2 = sc.parallelize(['март', 'апрель', 'май', 'июнь', 'апрель'])  
# Повтор 'апрель' будет удалён  
rdd3 = rdd1.intersection(rdd2); print(rdd3.collect())  
['апрель', 'март']
```

● **Вычитание (subtract)** — преобразование, удаляющее из первого RDD элементы, которые есть во втором. **Широкое преобразование.**

```
# Общие элементы: 'март' и 'апрель'  
rdd1 = sc.parallelize(['январь', 'февраль', 'март', 'апрель', 'апрель'])  
rdd2 = sc.parallelize(['март', 'апрель', 'май', 'июнь'])  
# rdd3 = rdd1 без rdd2 - удаляем 'март' и 'апрель'  
rdd3 = rdd1.subtract(rdd2)  
print(rdd3.collect())  
['январь', 'февраль']
```

- **Декартово произведение (cartesian)** — преобразование, которое возвращает все возможные пары элементов из двух наборов.

Широкое преобразование.

```
rdd1 = sc.parallelize(['январь', 'февраль', 'март'])
rdd2 = sc.parallelize([2025, 2026])
rdd3 = rdd1.cartesian(rdd2)
print(rdd3.collect())

[('январь', 2025),
 ('январь', 2026),
 ('февраль', 2025),
 ('февраль', 2026),
 ('март', 2025),
 ('март', 2026)]
```

Вопросы для конспекта

11. Расскажите об операции filter над Spark RDD.
12. Расскажите об операции map над Spark RDD.
13. Расскажите об операции flatMap над Spark RDD.
14. Расскажите об операции sample над Spark RDD.
15. Расскажите об операции union над Spark RDD.
16. Расскажите об операции distinct над Spark RDD.
17. Расскажите об операции intersection над Spark RDD.
18. Расскажите об операции subtract над Spark RDD.
19. Расскажите об операции cartesian над Spark RDD.

Операции действия над RDD

В отличие от преобразований, действия не создают новые RDD, а возвращают результат драйверу или записывают данные в файл.

- **Подсчитать (count)** — сообщить драйверу число элементов в RDD.
- **Получить (collect)** — получить в драйвере все значения из набора RDD в виде списка.

Выполнение этой операции требует, чтобы все данные умещались в памяти одного компьютера, на котором запущен драйвер. Обычно используется при отладке и тестировании программ Spark.

- **Получить n значений (take(n))** — в отличие от collect собирает не все, а только первые n элементов из RDD и возвращает их в виде массива на драйвер.
- **Получить n наибольших значений (top(n))** — значения будут получены в порядке убывания.

```
rdd = sc.parallelize(range(10))  
print(rdd.top(3))
```

```
[9, 8, 7]
```


Функция имеет второй аргумент — функция сравнения элементов.

```
rdd = sc.parallelize(['январь', 'февраль', 'март', 'апрель', 'май', 'июнь'])  
# Три самые короткие названия месяцев  
print(rdd.top(3, lambda x: -len(x)))  
['май', 'март', 'июнь']
```

- **Применить к каждому (foreach)** — применяет функцию к каждому элементу RDD. Похоже на map. Но это действие, которое выполняется немедленно. Здесь функция не возвращает результат, а выполняется ради побочных эффектов, например для записи в файл или отправки данных в хранилище.

- **Редукция (reduce)** — свести коллекцию RDD к единственному значению. Принимает единственный аргумент op:

- reduce — функция редукции $\text{reduce}(X, Y) \Rightarrow Z$. Аргументы X, Y и результат Z должны быть того же типа, что и элементы RDD. Применяется внутри партиций, а затем для объединения результатов партиций.

Если RDD пустой, выбрасывается исключение.

Пример: суммирование элементов RDD. Для определённости явно разобьём данные на две партии:

```
# Второй аргумент parallelize задаёт две партии
rdd = sc.parallelize([1,2,3,4,5,6], 2)
print(rdd.reduce(lambda x,y: x+y))
# Партия 1: [1, 2, 3] -> reduce(1,2)=3, reduce(3,3)=6
# Партия 2: [4, 5, 6] -> reduce(4,5)=9, reduce(9,6)=15
# Объединение: reduce(6,15)=21
```

21

- **Складывание (fold)** — похожа на reduce, но имеет два аргумента.
 - zeroValue — нулевое значение, используется в первом вызове функции редукции;
 - fold — функция редукции, применяется внутри партий, а затем для объединения результатов партий. Типы zeroValue, аргументов и результата функции редукции должны совпадать с типом элементов RDD.

Для получения корректного результата zeroValue должно быть нейтральным элементом по отношению к функции редукции: для любого X должно быть $\text{fold}(\text{zeroValue}, X) == X$.

Если функция выполняет суммирование, нейтральный элемент — ноль, если умножение — единица.

```
rdd = sc.parallelize([1,2,3,4,5,6])  
print(rdd.fold(0, lambda x,y: x+y)) # zeroValue=0
```

21

Нулевое значение участвует в вычислениях $N + 1$ раз, где N — число партиций. Если `zeroValue` не нейтральный элемент, тогда результат зависит от числа партиций.

Пример с двумя партициями и `zeroValue=10`

```
# Создаём RDD с явным указанием, что число партиций равно 2  
rdd = sc.parallelize([1,2,3,4,5,6], 2)  
# Пусть zeroValue = 10  
print(rdd.fold(10, lambda x,y: x+y))  
# Партиция 1: [1, 2, 3] -> fold(10,1)=11, fold(11,2)=13, fold(13,3)=16  
# Партиция 2: [4, 5, 6] -> fold(10,4)=14, fold(14,5)=19, fold(19,6)=25  
# Объединение: fold(10, 16)=26, fold(26,25)=51
```

51

Увеличим число партиций до трёх — результат на 10 больше:

```
rdd = sc.parallelize([1,2,3,4,5,6], 3)  
print(rdd.fold(10, lambda x,y: x+y))
```

61

- **Агрегация (aggregate)** — обобщение fold, принимает три аргумента:
 - асс — аккумулятор, объект для накопления информации, тип может не совпадать с типом элементов RDD;
 - `aggInner` — добавить новое значение `val` к аккумулятору:
`acc=aggInner(acc, val)`
 - `aggOuter` — попарная редукция аккумуляторов партиций:
`acc1=accOuter(acc1, acc2)`

Отличие от fold:

- Тип объекта асс может быть любым, а `zeroValue` в fold должен иметь тип, как у элементов RDD.
- Внутри партиций и между ними используются разные функции.

Операция `aggregate` позволяет выполнять более сложные действия над RDD по сравнению с `reduce` и `fold`.

Например, при помощи `reduce` и `fold` нельзя одним действием вычислить среднее значение. Можно просуммировать значения, а затем подсчитать число элементов при помощи действия `count`.

Пример: вычисление среднего значения при помощи aggregate.

```
rdd = sc.parallelize([1,2,3,4,5,6])
sumCnt = rdd.aggregate((0, 0), # Начальное значение аккумулятора
                        (lambda acc, val: (acc[0]+val, acc[1]+1)),
                        (lambda acc1, acc2: (acc1[0]+acc2[0], acc1[1]+acc2[1])))

print(sumCnt)
print(sumCnt[0]/sumCnt[1])

(21, 6)
3.5
```

Пример: вычисление минимального и максимального значений.

```
rdd = sc.parallelize([1,2,3,4,5,6])
minMax = rdd.aggregate((float('inf'), float('-inf')),
                        (lambda acc, val: (min(acc[0], val), max(acc[1], val))),
                        (lambda acc1, acc2: (min(acc1[0], acc2[0]),
                                              max(acc1[1], acc2[1]))))

print(minMax)

(1, 6)
```

Вопросы для конспекта

- 20. Расскажите об операциях `count`, `collect`, `take` и `top` над Spark RDD.
- 21. Расскажите об операции `foreach` над Spark RDD. В чём её отличие от операции `map`?
- 22. Расскажите об операции `reduce` над Spark RDD.
- 23. Расскажите об операции `fold` Spark RDD.
- 24. Расскажите об операции `aggregate` над Spark RDD.

Кеширование

Коллекции данных RDD обрабатываются в Spark в ленивом режиме. При этом один и тот же набор может использоваться многократно. Если не предпринять никаких дополнительных мер, то при каждом использовании он будет вычисляться заново.

Чтобы избежать многократного вычисления, можно потребовать от Spark кешировать вычисленные RDD. Уровни кеширования:

- **MEMORY_ONLY** — в память.
- **MEMORY_AND_DISK** — на диск, то что не поместилось в памяти.
- **DISK_ONLY** — только на диск.

Кеширование в память работает значительно быстрее, но если данные очень большие их не получится кешировать в памяти.

Если какой-то узел потерпит аварию и кеш будет потерян, данные будут вычислены заново, когда потребуются.

Если места в кеше не хватает, из кеша вытесняются наиболее старые данные — они начинают перевычисляться.

Стандартные дистрибутивы Spark не умеют автоматически кешировать данные.

Кеширование выполняется «вручную», при помощи операций `persist` и `cache`. `cache` не имеет параметров и кеширует всегда в память. `persist` по умолчанию также кеширует в память.

Для удаления данных из кеша используется метод `unpersist`.

```
from pyspark import StorageLevel
rdd1 = sc.parallelize(['январь', 'февраль', 'март', 'апрель', 'май', 'июнь'])
rdd2 = rdd1.map(lambda x: x[:3]).filter(lambda x: x[0] == 'м')
#rdd2.cache() # Кешируем данные одним из способов
#rdd2.persist(StorageLevel.MEMORY_ONLY)
#rdd2.persist(StorageLevel.MEMORY_AND_DISK)
rdd2.persist(StorageLevel.DISK_ONLY)
print(rdd2.collect())
print(rdd2.count())
rdd2.unpersist() # Удалить данные из кеша

['март', 'май']
2
```

Вопросы для конспекта

- 25. Для чего в Spark используется кеширование данных?
- 26. Какие уровни кеширования могут быть использованы в Spark?

Глобальные переменные и работа механизма замыкания в Spark

Многие операции над RDD задействуют пользовательские функции (например, `map` или `filter`).

При создании пользовательских функций можно использовать внешние переменные через стандартный механизм замыкания.

Примечание

Замыкание (closure) — механизм передачи в функцию информации о переменных вне её тела.

Также замыканием часто называют саму функцию, которая запоминает и имеет доступ к переменным из своей внешней (охватывающей) области видимости, даже после того, как внешняя функция завершила выполнение.

```
months = ['январь', 'февраль', 'март', 'апрель', 'май']
```

```
N = 4  # Внешняя переменная задаёт число символов
```

```
def takeN(s):  
    return s[:N]  # N существует вне функции
```

```
rdd1 = sc.parallelize(months)  
rdd2 = rdd1.map(takeN)  
print(rdd2.collect())  
['янва', 'февр', 'март', 'апре', 'май']
```

Операции над RDD выполняются в параллельном режиме, независимо несколькими исполнителями.

Для этого драйвер отправляет исполнителям код функций и копии захваченных ими внешних переменных. В нашем примере — это `N`.

Если, например, приложения выполняется шестью исполнителями, то будут созданы шесть копий переменной `N`.

Поэтому если пользовательская функция изменяет значение захваченной переменной, драйвер это не видит.

Пример (бессмысленный, только для иллюстрации) на простом Python — функция меняет захваченную переменную, используя `global`:

```
N = 4  # Внешняя переменная
def takeN(s):
    global N  # Захватываем N
    N = 3  # Меняем N
    return s[:N]

print(takeN('январь'), "N =", N)

январь N = 3
```

Тоже самое на Spark не работает:

```
months = ['январь', 'февраль', 'март', 'апрель', 'май']
N = 4
def takeN(s):
    global N # Захватываем N
    N = 3 # Меняем N
    return s[:N] # Новое значение работает - отделяем по 3 символа

print(N) # Драйвер видит только исходное значение 4
rdd1 = sc.parallelize(months)
rdd2 = rdd1.map(takeN)
print(rdd2.collect())

4
['январь', 'февраль', 'март', 'апрель', 'май']
```

Вопросы для конспекта

27. Что происходит, когда пользовательская функция, передаваемая в операцию над RDD, использует переменную, определённую вне её тела?
28. Что происходит, если функция изменяет внешнюю захваченную переменную во время работы исполнителя.

Аккумуляторы в Spark

- **Аккумуляторы** — это объекты для накопления информации в исполнителях и передачи её драйверу.

Аккумуляторы реализуют то, что нельзя сделать с обычными глобальными переменными в Spark: в Spark нельзя захватить глобальную переменную, изменить её в функции и сделать эти изменения видимыми вне функции.

Обычно модификация внешней захваченной переменной внутри функции считается плохой практикой. Но при обработки данных подобный механизм может быть полезен, например, для отладки.

Исполнителям аккумуляторы доступны только для записи. Запрет на чтение гарантирует отсутствие конфликтов доступа: для обеспечения корректного чтения аккумуляторов потребовалось бы выполнять синхронизацию их значений между исполнителями. Это была бы очень ресурсоёмкая операция.

Единственная допустимая операция с аккумулятором в исполнителе — инкремент.

Значения аккумуляторов накапливаются только при выполнении операции действия над RDD (ленивые вычисления).

Накопления выполняются каждым исполнителем отдельно, а затем их значения суммируются драйвером и результат передаётся пользователю.

Пример использования аккумулятора — подсчёт числа пустых строк.

```
months = ['январь', '', 'февраль', '', 'март', 'апрель', '', 'май']
```

```
blankLines = sc.accumulator(0) # Создаём аккумулятор
```

```
def takeN(s):  
    global blankLines # Захватили аккумулятор  
    if (s == ""):  
        blankLines += 1 # Подсчитываем пустые строки  
    return s[:3]
```

```
rdd1 = sc.parallelize(months)  
rdd2 = rdd1.map(takeN)  
print(rdd2.collect())  
print("blankLines =", blankLines.value)  
['январь', '', 'февраль', '', 'март', 'апрель', '', 'май']  
blankLines = 3
```

Аккумуляторы могут быть вычислены повторно при пересчёте RDD, вытеснении RDD и кеша и сбросе и восстановлении/замещении узла.

Поэтому их следует использовать с осторожностью.

Пример, в котором над одним RDD выполняются два действия и аккумулятор вычисляется дважды

```
months = ['январь', '', 'февраль', '', 'март', 'апрель', '', 'май']
blankLines = sc.accumulator(0)
def takeN(s):
    global blankLines    # Захватили аккумулятор
    if (s == ""):
        blankLines += 1  # Подсчитываем пустые строки
    return s[:3]

rdd1 = sc.parallelize(months)
rdd2 = rdd1.map(takeN)
print(rdd2.collect())    # Действие collect()
print("blankLines =", blankLines.value)
print(rdd2.count())      # Действие count()
print("blankLines =", blankLines.value)

['январь', '', 'февраль', '', 'март', 'апрель', '', 'май']
blankLines = 3
8
blankLines = 6
```

Для предотвращения повторного накопления аккумулятора, RDD нужно закешировать перед выполнением действий. Тогда операции преобразований не будут перевычисляться для каждого действия.

```
months = ['январь', '', 'февраль', '', 'март', 'апрель', '', 'май']
```

```
blankLines = sc.accumulator(0)
```

```
def takeN(s):
```

```
    global blankLines    # Захватили аккумулятор
```

```
    if (s == ""):
```

```
        blankLines += 1    # Подсчитываем пустые строки
```

```
    return s[:3]
```

```
rdd1 = sc.parallelize(months)
```

```
rdd2 = rdd1.map(takeN)
```

```
rdd2.persist()    # Кешируем - теперь map не будет перевычисляться
```

```
print(rdd2.collect())    # Действие collect()
```

```
print("blankLines =", blankLines.value)
```

```
print(rdd2.count())    # Действие count()
```

```
print("blankLines =", blankLines.value)
```

```
['янв', '', 'фев', '', 'мар', 'апр', '', 'май']
```

```
blankLines = 3
```

```
8
```

```
blankLines = 3
```

Вопросы для конспекта

- 29. Для каких целей в Spark используются аккумуляторы?
- 30. Какие действия с аккумуляторами разрешены в исполнителях в Spark? Почему все другие запрещены?
- 31. Что происходит с аккумуляторами, когда в приложении Spark выполняются несколько действий над одним и тем же RDD? Как сделать, чтобы вычисления аккумуляторов оставались корректными в таких случаях?

Широковещательные переменные

Передача информации от драйвера к исполнителям может осуществляться через обычные глобальные переменные и механизм замыкания.

Но если исполнителям требуется передать большой объём информации (например, справочную таблицу, в соответствии с которой нужно выполнять фильтрацию данных), то этот механизм становится крайне неэффективным.

При каждом запуске задачи с замыканием, которое захватывает объект большой ёмкости, этот объект сериализуется в байты, отправляется **с каждой задачей** каждому исполнителю, десериализуется на каждом исполнителе.

Если захватываемый объект имеет объём 1 Мб и имеется 1000 задач на 100 исполнителях, то 1 Мб данных будет отправлен 1000 раз через сеть — будет передан 1 Гб данных.

- **Широковещательные переменные** — объекты, при помощи которых от драйвера к исполнителям передаётся информация большого объёма.

В отличие от простых глобальных переменных, данные передаются не каждой задаче, а **на каждый рабочий узел** кластера и только один раз. Затем они хранятся в памяти или на диске этого узла для всех задач.

Кроме этого, при распространении широковещательной переменной по узлам кластера используется оптимизированный протокол, работающий по принципу точка-точка (peer-to-peer).

Это значит, что данные передаются не только от драйвера к исполнителям. Кроме этого узлы могут напрямую обмениваться между собой фрагментами недостающих данных.

В примере ниже создаётся широковещательная переменная со списком названий месяцев и с её помощью вычисляется, сколько раз в тексте встречаются названия месяцев.

Вместо широковещательной переменной можно было бы взять простой список. Но так эффективнее.

```
поем = ""Июнь, Июль сгущают зной, Август - ягодой лесной.  
Сентябрь вносит желтизну, Октябрь красит всю листву.  
Декабрь, Январь метут метель, Февраль скрипит на льду постель.  
Март ручьи нам принесёт, Апрель - подснежника найдёт.  
А за Май, цветущий, ясный, Год целуется с прекрасной!""
```

```
# Широковещательная переменная - мог быть просто список, но так эффективнее  
months = sc.broadcast(['январь', 'февраль', 'март', 'апрель', 'май',  
                        'июнь', 'июль', 'август', 'сентябрь', 'октябрь', 'ноябрь', 'декабрь'])
```

```
def if_month(s):  
    return 1 if s.lower() in months.value else 0
```

```
# Чистим текст и превращаем в RDD  
rdd1 = sc.parallelize(poem.replace('\n', ' ').replace(',','').  
                      .replace('.', '').split(' '))
```

```
# Проверяем, что получилось
```

```
print(rdd1.take(5))
```

```
# Заменяем месяцы на 1, другие слова на 0 и суммируем
```

```
print(rdd1.map(if_month).sum())
```

```
['Июнь', 'Июль', 'сгущают', 'зной', 'Август']
```

```
11
```

Вопросы для конспекта

32. Каким способами драйвер приложения Spark может передавать исполнителям переменные?

33. В чём преимущество использования широковещательных переменных Spark по сравнению с простыми глобальными переменными?

Вопросы для конспекта

- 1 Если при загрузке в Spark RDD данных из файла в отвечающий за это метод передать неверные путь или имя файла, то метод выполнится без ошибок. Почему? В какой момент вычисления будут прерваны с сообщением «Файл не найден»?
- 2 В каких местах внутри текстового файла Spark может, а в каких не может разделять его на партиции при загрузке в RDD?
- 3 Какой метод отвечает за загрузку текстового файла в Spark RDD? Как с его помощью загрузить сразу несколько файлов? Как загрузить содержимое целого каталога?
- 4 Как в Spark RDD загрузить несколько текстовых файлов с сохранением путей к ним?
- 5 Как загрузить в Spark RDD много небольших текстовых файлов так, чтобы обрабатывать каждый отдельный файл как единое целое, т. е. с гарантией, что части одного файла не окажутся в разных партициях?
- 6 Какие объекты файловой системы создаёт Spark при сохранении RDD в виде текста?
- 7 Как выполняется загрузка JSON и CSV файлов в Spark RDD. Приведите пример.
- 8 Как сохранить текстовый файл в виде JSON или CSV. Приведите пример.
- 9 Как при загрузке в Spark RDD CSV файла удалить из RDD строку заголовка. Приведите пример.
- 10 Как загрузить в Spark RDD CSV файл в виде коллекции словарей?
- 11 Расскажите об операции filter над Spark RDD.
- 12 Расскажите об операции map над Spark RDD.
- 13 Расскажите об операции flatMap над Spark RDD.
- 14 Расскажите об операции sample над Spark RDD.
- 15 Расскажите об операции union над Spark RDD.
- 16 Расскажите об операции distinct над Spark RDD.
- 17 Расскажите об операции intersection над Spark RDD.

- 18 Расскажите об операции `subtract` над Spark RDD.
- 19 Расскажите об операции `cartesian` над Spark RDD.
- 20 Расскажите об операциях `count`, `collect`, `take` и `top` над Spark RDD.
- 21 Расскажите об операции `foreach` над Spark RDD. В чём её отличие от операции `map`?
- 22 Расскажите об операции `reduce` над Spark RDD.
- 23 Расскажите об операции `fold` Spark RDD.
- 24 Расскажите об операции `aggregate` над Spark RDD.
- 25 Для чего в Spark используется кеширование данных?
- 26 Какие уровни кеширования могут быть использованы в Spark?
- 27 Что происходит, когда пользовательская функция, передаваемая в операцию над RDD, использует переменную, определённую вне её тела?
- 28 Что происходит, если функция изменяет внешнюю захваченную переменную во время работы исполнителя.
- 29 Для каких целей в Spark используются аккумуляторы?
- 30 Какие действия с аккумуляторами разрешены в исполнителях в Spark? Почему все другие запрещены?
- 31 Что происходит с аккумуляторами, когда в приложении Spark выполняются несколько действий над одним и тем же RDD? Как сделать, чтобы вычисления аккумуляторов оставались корректными в таких случаях?
- 32 Каким способами драйвер приложения Spark может передавать исполнителям переменные?
- 33 В чём преимущество использования широковебательных переменных Spark по сравнению с простыми глобальными переменными?